

RĪGAS VALSTS TEHNIKUMS

DATORIKAS NODAĻA

Izglītības programma: Programmēšana

KVALIFIKĀCIJAS DARBS

“Astronomijas izglītības platforma”

Paskaidrojošais raksts 58 lpp.

Audzēknis:

Aleksandrs Vodņevs DP4-2

Prakses vadītājs:

Kaspars Baldiņš

Nodaļas vadītājs:

Normunds Barbāns

Rīga, 2026

ANOTĀCIJA

Darba tēma: Astronomijas izglītības platforma - tīmekļa lietojumprogramma.

Platforma ir izstrādāta, lai nodrošinātu lietotājiem ātru un saprotamu piekļuvi astronomijas informācijai, jaunumiem un kopienas saturam. Sistēmas izstrādē izmantotas modernās tehnoloģijas: Node.js v22 (backend), React v19 (frontend), PostgreSQL v18.3 (datu bāze), Sequelize v6.37 (ORM), Express.js v5.1, JSON Web Token un bcrypt autentifikācijai. Izstrādes vide - Visual Studio Code, versiju kontrole - Git un GitHub.

Kvalifikācijas darbs ietver ievadu, uzdevuma nostādni, prasību specifikāciju (ieejas un izejas informācijas aprakstu, funkcionālās un nefunkcionālās prasības), uzdevuma risināšanas līdzekļu izvēles pamatojumu, programmatūras produkta modelēšanu un projektēšanu (sistēmas arhitektūru, ER modeli un datu plūsmu diagrammas), datu struktūru aprakstu, lietotāja ceļvedi, programmatūras lietojamības testēšanu un nobeigumu. Ievadā ir aprakstīta astronomijas izglītības platformas nepieciešamība un aktualitāte. Prasību specifikācija satur ieejas un izejas informācijas aprakstu, funkcionālās un nefunkcionālās prasības. Sistēmas modelēšanas daļā ir aprakstīta sistēmas arhitektūra ar apakšsistēmu sadalījumu, ER modelis ar entitijām un to saistībām, kā arī septiņas datu plūsmu diagrammas. Datu struktūru aprakstā ir norādīta datu bāzes tabulu struktūra. Lietotāja ceļvedī ir aprakstītas sistēmas prasības, instalācija un programmas lietošanas pamācība. Programmatūras testēšanas nodaļā uzrādīti funkcionālo prasību testēšanas pārskati.

Kvalifikācijas darbs sastāv no aptuveni 58 lappusēm (bez pielikumiem), kurās ietilpst tabulas un teksts. Pielikumos iekļautas sistēmas diagrammas, ekrānattēlu paraugi un programmas pirmkods.

ANNOTATION

Topic: Astronomy Education Platform - a web application. The platform has been developed to provide users with quick and easy access to astronomy information, news, and community content. The system was built using modern technologies: Node.js v22 (backend), React v19 (frontend), PostgreSQL v18.3 (database), Sequelize v6.37 (ORM), Express.js v5.1, JSON Web Token and bcrypt for authentication. The development environment is Visual Studio Code, with version control via Git and GitHub.

The qualification thesis includes an introduction, task statement, requirements specification (input/output data description, functional and non-functional requirements), justification of tools chosen, software product modelling and design (system architecture, ER model, and data flow diagrams), data structure description, user guide, software usability testing, and conclusion. The introduction describes the need for and relevance of an astronomy education platform. The requirements specification contains input and output data descriptions and both functional and non-functional requirements. The modelling section describes system architecture with subsystem breakdown, ER model with entities and relationships, and seven data flow diagrams. The data structure section details the database table structures. The user guide covers system requirements, installation, and program usage instructions. The testing chapter presents functional requirements testing reports.

The qualification thesis consists of approximately 58 pages (excluding appendices), containing tables and text. Appendices include system diagrams, screenshot samples, and the program source code.

SATURS

IEVADS.....	5
1. UZDEVUMA NOSTĀDNE	6
2. PRASĪBU SPECIFIKĀCIJA	8
2.1. Ieejas un izejas informācijas apraksts	8
2.1.1. Ieejas informācijas apraksts	8
2.1.2. Izejas informācijas apraksts	9
2.1.3. Ārējas informācijas apraksts	10
2.2. Funkcionālās prasības	11
2.3. Nefunkcionālās prasības.....	15
2.4. Skices	16
3. UZDEVUMA RISINĀŠANAS LĪDZEKĻU IZVĒLES PAMATOJUMS	18
3.1. Izmantoto rīku detalizēts apraksts	18
4. PROGRAMMATŪRAS PRODUKTA MODELĒŠANA UN PROJEKTĒŠANA	20
4.1. Sistēmas struktūras modelis	20
4.1.1. Sistēmas arhitektūra	20
4.1.2. Sistēmas ER modelis.....	21
4.2. Funkcionālais sistēmas modelis	23
4.2.1. Datu plūsmu modelis.....	23
5. DATU STRUKTŪRU APRAKSTS	31
6. LIETOTĀJA CEĻVEDIS	37
6.1. Sistēmas prasības aparatūrai un programmatūrai	37
6.2. Sistēmas instalācija un palaišana.....	37
6.3. Programmas apraksts.....	38
7. PROGRAMMATŪRAS LIETOJAMĪBAS TESTĒŠANA	50
NOBEIGUMS.....	54
INFORMĀCIJAS AVOTI.....	55
PIELIKUMI.....	57

IEVADS

Kvalifikācijas darba mērķis ir izstrādāt tīmekļa platformu, kas sniedz lietotājiem iespēju ātri un saprotami iegūt informāciju par astronomiju. Mūsdienās interesenti bieži sastopas ar problēmu - materiāli par astronomiju ir izklaidēti dažādos avotos, daļa ir sarežģīti iesācējiem, bet citi nav pietiekami uzticami. Tas apgrūtina mācīšanās procesu un rada nevajadzīgu laika patēriņu.

Tēma ir aktuāla, jo arvien vairāk cilvēku vēlas apgūt astronomijas pamatus, sekot līdzi kosmosa jaunumiem un saprast, kā izmantot teleskopus vai veikt novērojumus. Pašlaik šim nolūkam izmanto dažādus atsevišķus resursus - ziņu portālus, forumus, sociālos tīklus. Tie sniedz informāciju, bet neveido pilnu mācību un kopienas pieredzi.

Izstrādātais risinājums apvieno astronomijas ziņu sekciju, kopienas forumu ar reģistrētu lietotāju sistēmu, mācību materiālus ar Markdown atbalstu, profilu pārvaldību ar attēlu augšupielādi, interaktīvu 3D Saules sistēmas vizualizāciju un NASA ikdienas astronomijas attēlu sākumlapā. Platforma nodrošina divvalodīgu (LV/EN) saskarni, e-pasta verifikāciju reģistrācijas laikā un paroles atjaunošanu pa e-pastu. Tā ir orientēta uz iesācējiem un entuziastiem, nodrošinot vienkāršu un viegli saprotamu piekļuvi astronomijas saturam.

Tirgus izpete liecina, ka analoģu risinājumu ir maz, un neviens pilnvērtīgi neapvieno visas nepieciešamās funkcijas. Stellarium (stellarium.org) ir populāra bezmaksas teleskopa un debess kartes lietotne, kas piedāvā 3D debess vizualizāciju, taču tai nav foruma, ziņu vai mācību materiālu. Sky & Telescope (skyandtelescope.org) ir profesionāls angļu valodas ziņu portāls ar rakstiem un attēliem, taču nav kopienas sadaļas un nav pieejams latviski. Astronomy.com piedāvā zināscaronā rakstus un jaunumus, taču tāpat nav interaktīvas vizualizācijas vai kopienas funkciju. Neviena no šīm platformām nepiedāvā apvienojumu: latviešu valodas atbalstu, kopienas forumu, mācību materiālus un interaktīvu 3D Saules sistēmu vienā vietā.

1. UZDEVUMA NOSTĀDNE

Sistēmā tiks realizētas vairākas funkcionālās prasības, kas nodrošinās pilnvērtīgu platformas darbību un ļaus lietotājiem piekļūt informācijai par astronomiju, kā arī sazināties savā starpā.

1.2. Funkcionālās prasības

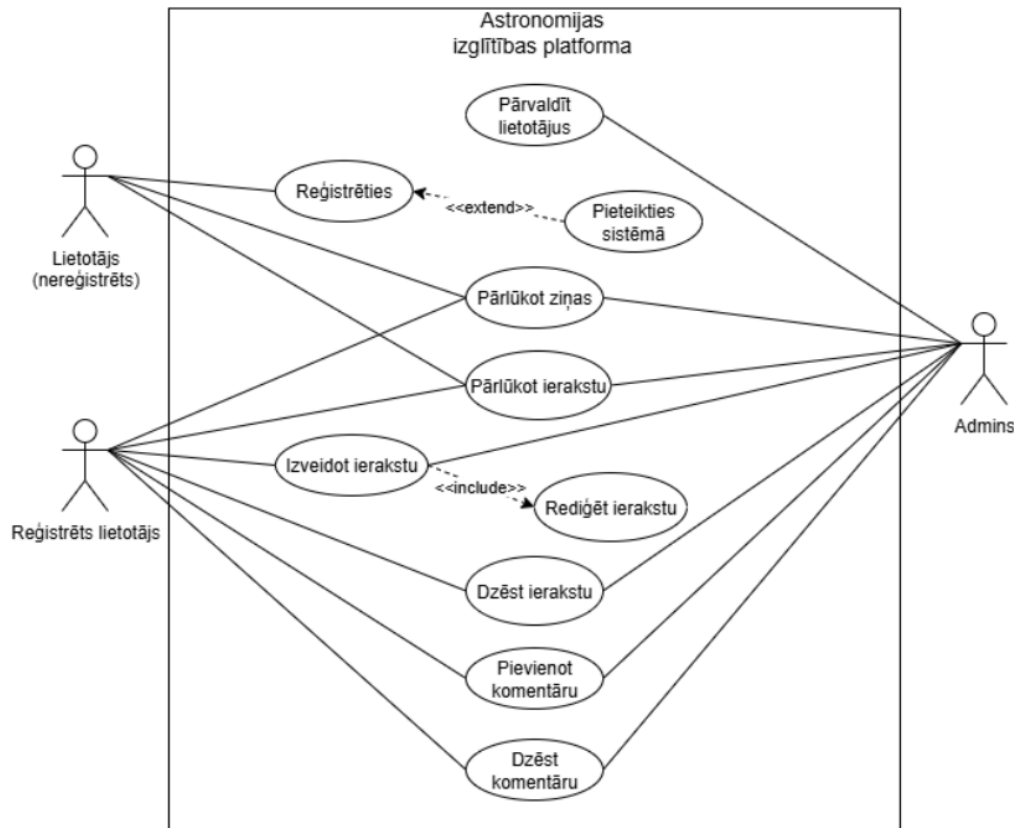
- Ziņu apskate - visi lietotāji varēs skatīt astronomijas jaunumus un rakstus.
- Postu apskate platformā tiks attēloti kopienas veidotie raksti un pieredzes stāsti.
- Postu izveide (reģistrētajiem lietotājiem) - reģistrēti lietotāji varēs publicēt savu saturu.
- Komentāru pievienošana - reģistrēti lietotāji varēs komentēt postus.
- Profila pārvaldība - katrs reģistrētais lietotājs varēs papildināt un labot savu profilu.
- Administratora paneļa funkcijas - administrators varēs pārvaldīt lietotājus, forumu un saturu.
- Neregistrēta lietotāja režīms - viesis varēs skatīt publisko saturu un reģistrēties

1.3. Lietotāju lomas

- Viesis - var apskatīt ziņas, postus un mācību materiālus ierobežotā režīmā, kā arī reģistrēties.
- Reģistrēts lietotājs - var izveidot postus, rakstīt komentārus, rediģēt savu profilu un piekļūt paplašinātam saturam.

- Administrators - atbild par visu sistēmas datu pārvaldību, lietotāju lomām, ziņu ievietošanu un satura moderēšanu.

Aleksandrs Vodņevs, DP4-2, Astronomijas izglītības platforma



1.att. Lietošanas gadījumu (Use Case) diagramma

2. PRASĪBU SPECIFIKĀCIJA

2.1. Ieejas un izejas informācijas apraksts

2.1.1. Ieejas informācijas apraksts

Ieejas informācija ir dati, kurus lietotājs vai administrators ievada sistēmā, lai nodrošinātu tās funkcionalitāti. Ievaddati tiek apstrādāti, validēti un saglabāti datu bāzē.

Informācija par lietotāja reģistrāciju sastāvēs no šādiem datiem:

Vārds - lietotāja personvārds - teksta lauks līdz 30 simboliem, tikai burti (piem., Aleksandrs).

Uzvārds - lietotāja uzvārds - teksta lauks līdz 30 simboliem.

E-pasts - unikāla lietotāja e-pasta adrese - jāatbilst standarta e-pasta formātam.

Parole - drošības parole konta izveidei - vismaz 8 simboli.

Informācija par lietotāju autentifikāciju sastāvēs no šādiem datiem:

E-pasts - lietotāja elektroniskā pasta adrese autorizācijai sistēmā - burtu, ciparu un simbolu kombinācija, kas satur simbolu “@” un domēna nosaukumu (piem., user@email.com). E-pasts sistēmā nedrīkst atkārtoties.

Parole - drošības parole piekļuvei sistēmai - burtu un ciparu kombinācija, vismaz 8 simboli. Parole tiek saglabāta šifrētā veidā un tiek izmantota tikai lietotāja autentifikācijai.

Autentifikācijas ievade - lietotāja ievadāmie dati pieteikšanās formā:

- e-pasts - lietotāja e-pasta adrese pieteikšanās laukā.
-

Loma - lietotāja tiesību līmenis sistēmā - tiek piešķirta sistēmas pusē pēc veiksmīgas autentifikācijas - (piem., Reģistrēts lietotājs, Administrators).

Informācija par ieraksta (posta) izveidi sastāvēs no šādiem datiem:

Virsraksts - ieraksta nosaukums - teksta lauks līdz 100 rakstzīmēm (piem., “Kas ir melnais caurums?”).

Saturs - ieraksta galvenais teksts - teksta lauks līdz 3000 rakstzīmēm.

Kategorija - ieraksta tematiskā piederība - izvēle no saraksta (piem., Planētas, Galaktikas, Kosmos, Astrofizika).

Publicēšanas statuss - ieraksta redzamības statuss - izvēle: melnraksts vai publicēts.

Informācija par komentāra pievienošanu sastāvēs no šādiem datiem:

Komentāra saturs - lietotāja ievadīts komentārs - teksta lauks līdz 500 rakstzīmēm.

Publicēšanas datums - sistēmas automātiski ģenerēts datums un laiks.

Informācija par lietotāja profila rediģēšanu sastāvēs no šādiem datiem:

Profila apraksts - lietotāja īss pašraksturojums - teksta lauks līdz 300 rakstzīmēm.

Interesu joma - lietotāja izvēlētā astronomijas tematika - izvēle no saraksta.

Zināšanu līmenis - lietotāja pašnovērtētais līmenis - izvēle no variantiem: Iesācējs,

Vidējs, Padziļināts.

Informācija par ziņu pievienošanu (administratora funkcija) sastāvēs no šādiem datiem:

Ziņas virsraksts - informatīvā paziņojuma nosaukums - teksta lauks līdz 150 rakstzīmēm.

Ziņas saturs - pilns ziņas teksts - teksta lauks bez stingra simbolu ierobežojuma.

Avota saite ārējās informācijas resurss - URL formāts, kas sākas ar “https://”.

2.1.2. Izejas informācijas apraksts

Izejas informācija ir dati, kurus sistēma attēlo lietotājam vai administratoram pēc datu apstrādes. Informācija tiek attēlota tīmekļa saskarnē vai administrācijas panelī.

Informācija par lietotāja autentifikācijas rezultātu sastāvēs no šādiem datiem:

Autentifikācijas statuss - sistēmas atbilde uz ievadītajiem datiem - paziņojums ekrānā (piem., “Pieteikšanās veiksmīga” vai “Nepareizs e-pasts vai parole”).

Lietotāja loma - pēc veiksmīgas autentifikācijas sistēma piešķir atbilstošu piekļuves līmeni un ielādē attiecīgo interfeisu.

Informācija par ziņu attēlošanu sastāvēs no šādiem datiem:

Ziņas virsraksts - attēlots sarakstā.

Publicēšanas datums - sistēmas ģenerēts datums.

Ziņas saturs - pilns teksts detalizētajā skatā.

Informācija par ierakstu attēlošanu sastāvēs no šādiem datiem:

Virsraksts - ieraksta nosaukums.

Autors - lietotāja vārds un uzvārds.

Publicēšanas datums - sistēmas saglabāts datums.

Komentāru skaits - sistēmas aprēķināts skaits.

Informācija par komentāru attēlošanu sastāvēs no šādiem datiem:

Komentāra saturs - lietotāja ievadītais teksts.

Autors - lietotāja vārds.

Publicēšanas datums un laiks - sistēmas ģenerēta vērtība.

Informācija administratora panelī sastāvēs no šādiem datiem:

Lietotāju saraksts - reģistrēto lietotāju dati (vārds, e-pasts, loma).

Ierakstu pārvaldība - visu sistēmas ierakstu saraksts ar iespēju dzēst vai rediģēt.

Komentāru pārvaldība - komentāru saraksts ar iespēju dzēst neatbilstošu saturu.

2.1.3. Ārējas informācijas apraksts

Astronomijas izglītības platforma apmainās ar datiem ar trim ārējām sistēmām:

Cloudinary attēlu glabāšanai, Resend e-pasta sūtīšanai un NASA APOD API satura iegūšanai.

Cloudinary (attēlu glabāšana)

Cloudinary ir mākoņpakalpojums attēlu glabāšanai un apstrādei. Platforma nosūta attēlu uz Cloudinary, izmantojot HTTP POST pieprasījumu ar multipart/form-data formātu. Cloudinary atgriež JSON atbildi ar attēla publisko URL (`secure_url`), kas tiek saglabāts datubāzē. Savienojums notiek caur backend serveri, izmantojot Cloudinary SDK. Nodoto datu saraksts: attēla binārais saturs (`buffer`), mapes nosaukums (`folder`), publiskais identifikators (`public_id`). Sagamaie dati: attēla HTTPS URL, publiskais ID, attēla izmērs un formāts.

Resend (e-pasta sūtīšana)

Resend ir e-pasta nosūtīšanas pakalpojums, ko sistēma izmanto divu veidu e-pastu sūtīšanai: (1) e-pasta verificācijas kods reģistrācijas laikā (6 ciparu kods ar 15 minūšu derīgumu) un (2) paroles atjaunošanas saite (tokens ar 1 stundas derīgumu). Backend serveri veido HTTP POST pieprasījumu uz Resend API (`api.resend.com/emails`), nodod sūtītāja adresi (`noreply@astrolv.com`), saņēmēja adresi, tēmu un HTML saturu. Resend atgriež JSON ar nosūtīšanas statusu un ziņojuma ID.

NASA APOD API (ikdienas astronomijas attēls)

NASA Astronomy Picture of the Day (APOD) API ir bezmaksas publisks NASA pakalpojums, kas katru dienu nodrošina jaunas astronomijas fotogrāfijas ar aprakstu. Backend serveri nosūta HTTP GET pieprasījumu uz `api.nasa.gov/planetary/apod` ar API atslegas

parametru. Sagamaie JSON dati: title (attēla nosaukums), url (attēla adrese), hdurl (augstas izskirtspējas URL), explanation (apraksts angļu valodā), date (publicēšanas datums), media_type (image vai video). Atbilde tiek kešota servera pusē 24 stundas, lai nepatērētu API limitu. Savienojums notiek tikai no backend servera - API atslegas netiek nodota klientpusei.

2.2 Funkcionālās prasības

1. Lietotāja reģistrācija sistēmā

2.1.tabula

Identifikators	FP 2.2.1
Ievads	
	Funkcija paredzēta, lai jauns lietotājs varētu izveidot kontu platformā un iegūtu piekļuvi reģistrēta lietotāja funkcionalitātei.
Ievaddati	
	1. Vārds - obligāts lauks, teksts līdz 30 rakstzīmēm (piem., Aleksandrs). 2. Uzvārds - obligāts lauks, teksts līdz 30 rakstzīmēm (piem., Bērziņš). 3. E-pasts - unikāla e-pasta adrese, jāsaturs simbolu "@" un domēnu (piem., user@inbox.lv). 4. Parole - vismaz 8 simboli. 5. Paroles apstiprinājums - jāsakrīt ar ievadīto paroli.
	2. Uzvārds - obligāts lauks, teksts līdz 30 rakstzīmēm (piem., Bērziņš).
	3. E-pasts - unikāla e-pasta adrese, jāsaturs simbolu "@" un domēnu (piem., user@inbox.lv).
	4. Parole - vismaz 8 simboli, var saturēt burtus un ciparus.
	5. Paroles apstiprinājums - jāsakrīt ar ievadīto paroli.
Apstrāde	
	1. Tiek pārbaudīts, vai visi obligātie lauki ir aizpildīti.
	2. Tiek pārbaudīts e-pasta formāts.
	3. Tiek pārbaudīts, vai e-pasts nav jau reģistrēts sistēmā.
	4. Tiek pārbaudīts paroles garums (min. 8 simboli).
	5. Parole tiek hešēta ar bcrypt algoritmu.
	6. Jaunais lietotājs tiek saglabāts datu bāzē ar lomu "user".
Izvaddati	
	1. Veiksmīgas reģistrācijas gadījumā tiek izsniegts JWT tokens.
	2. Lietotājs tiek novirzīts uz sākumlapu ar aktīvu sesiju.
Kļūdas paziņojumi	
	1. "Visi lauki ir obligāti" - ja kāds lauks nav aizpildīts.
	2. "Nepareizs e-pasta formāts" - ja e-pasts neatbilst standartam.
	3. "Lietotājs ar šādu e-pastu jau eksistē" - ja e-pasts aizņemts.
	4. "Parole ir pārāk īsa (min. 8 simboli)" - ja parole par īsu.
	5. "Paroles nesakrīt" - ja apstiprinājums neatbilst.
Izsaucamās prasības	
	Nav izsaucamo prasību.

2. Lietotāja autentifikācija (pieteikšanās)

2.2.tabula

Identifikators	FP 2.2.2
Ievads	
Funkcija paredzēta, lai reģistrēts lietotājs varētu pieteikties sistēmā un izmantot tās pilno funkcionalitāti.	
Ievaddati	
1. E-pasts - lietotāja reģistrācijas e-pasta adrese.	
2. Parole - lietotāja konta parole.	
Apstrāde	
1. Tiek pārbaudīts, vai lietotājs ar šādu e-pastu eksistē datu bāzē.	
2. Ievadītā parole tiek salīdzināta ar saglabāto heša vērtību.	
3. Tiek pārbaudīts konta statuss (aktīvs / bloķēts).	
4. Veiksmīgas autentifikācijas gadījumā tiek ģenerēts JWT tokens ar derīgumu 7 dienas.	
Izvaddati	
1. Lietotājs tiek novirzīts uz sākumlapu ar aktīvu sesiju.	
2. Navigācijas joslā parādās lietotāja vārds un poga "Iziet".	
Kļūdas paziņojumi	
1. "Nepareizs e-pasts vai parole" - ja dati neatbilst.	
2. "Jūsu konts ir bloķēts" - ja administrators ir bloķējis kontu.	
3. "E-pasts un parole ir obligāti" - ja lauki nav aizpildīti.	
Izsaucamās prasības	
Nav izsaucamo prasību.	

3. Ieraksta izveide un publicēšana

2.3.tabula

Identifikators	FP 2.2.3
Ievads	
Funkcija paredzēta, lai autentificēts lietotājs varētu izveidot un publicēt jaunu ierakstu astronomijas forumā.	
Ievaddati	
1. Virsraksts - obligāts, teksts līdz 100 rakstzīmēm.	
2. Saturs - obligāts, teksts (nav ierobežojuma garumam).	
3. Kategorija - izvēle no saraksta: Planētas, Galaktikas, Kosmos, Astrofizika, Novērojumi.	
4. Statuss - izvēle: Publicēts vai Melnraksts.	
Apstrāde	
1. Tiek pārbaudīta lietotāja autentifikācija (JWT tokens).	
2. Tiek pārbaudīts, vai obligātie lauki ir aizpildīti.	
3. Saturs tiek sanitizēts (aizsardzība pret XSS uzbrukumiem).	
4. Ieraksts tiek saglabāts datu bāzē ar autora ID un publicēšanas laiku.	

5. Ja statuss ir "Publicēts" - ieraksts ir redzams forumā.
Izvaddati
1. Ieraksts parādās foruma sarakstā (ja statuss "Publicēts").
2. Lietotājs tiek novirzīts uz jaunā ieraksta lapu.
Kļūdas paziņojumi
1. "Virsraksts ir obligāts lauks" - ja virsraksts nav aizpildīts.
2. "Saturs ir obligāts" - ja saturs nav ievadīts.
3. HTTP 401 un novirzīšana uz pieteikšanās lapu - ja sesija nav aktīva.
Izsaucamās prasības
FP 2.2.2 (lietotājam jābūt autentificētam).

2.3.tabulas turpinājums

4. Komentāra pievienošana

2.4.tabula

Identifikators	FP 2.2.4
Ievads	
	Funkcija paredzēta, lai autentificēts lietotājs varētu pievienot komentāru pie jebkura publicēta ieraksta.
Ievaddati	
	1. Komentāra teksts - obligāts, no 1 līdz 500 rakstzīmēm.
	2. Ieraksta identifikators - automātiski no sistēmas (nav redzams lietotājam).
Apstrāde	
	1. Tiek pārbaudīta lietotāja autentifikācija.
	2. Tiek pārbaudīts komentāra teksta garums.
	3. Teksts tiek sanitizēts.
	4. Komentārs tiek saglabāts ar autora ID, ieraksta ID un publicēšanas laiku.
	5. Komentāru skaita rādītājs pie ieraksta tiek atjaunināts.
Izvaddati	
	1. Komentārs parādās ieraksta lapā zemāk par esošajiem komentāriem.
	2. Rādītājs "Komentāri (N)" tiek atjaunināts.
Kļūdas paziņojumi	
	1. "Komentārs nevar būt tukšs" - ja teksta lauks ir tukšs.
	2. "Komentārs nevar pārsniegt 500 rakstzīmes" - ja pārāk garš.
	3. HTTP 401 - ja lietotājs nav autentificēts.
Izsaucamās prasības	
	FP 2.2.2 (lietotājam jābūt autentificētam).

5. Lietotāju pārvaldība (administrators)

2.5.tabula

Identifikators	FP 2.2.5
Ievads	
Funkcija paredzēta, lai administrators varētu pārvaldīt sistēmas lietotāju kontus - mainīt lomas un bloķēt vai atbloķēt kontus.	
Ievaddati	
1. Lietotāja identifikators - automātiski no sistēmas.	
2. Jaunā loma - izvēle: Lietotājs vai Administrators.	
3. Jaunais statuss - izvēle: aktīvs vai bloķēts.	
Apstrāde	
1. Tiek pārbaudīta administratora autentifikācija un loma.	
2. Tiek atrasts attiecīgais lietotājs datu bāzē.	
3. Tiek atjaunināta loma vai statuss.	
4. Ja konts tiek bloķēts - lietotājs nevar pieteikties sistēmā.	
Izvaddati	
1. Lietotāju sarakstā tiek atjaunināta informācija par lomu/statusu.	
2. Bloķētam lietotājam pieteikšanās atteikta ar attiecīgu paziņojumu.	
Kļūdas paziņojumi	
1. HTTP 403 Forbidden - ja pieprasījumu veic nevis administrators.	
2. "Kļūda mainot lomu / statusu" - servera kļūdas gadījumā.	
Izsaucamās prasības	
FP 2.2.2 (administratoram jābūt autentificētam ar lomu "admin").	

6. Paroles atjaunošana pa e-pastu

2.6.tabula

Identifikators	FP 2.2.6
Ievads	
Funkcija paredzēta, lai lietotājs, kurš aizmirsis paroli, varētu to atjaunot, izmantojot reģistrācijā norādīto e-pasta adresi.	
Ievaddati	
1. E-pasta adrese - obligāts lauks (paroles atjaunošanas veidlapā).	
2. Jaunā parole - obligāts lauks, min. 8 simboli.	
3. Paroles apstiprinājums - jāsakrīt ar jauno paroli.	
Apstrāde	
1. Tiek pārbaudīts, vai e-pasts eksistē datu bāzē.	
2. Tiek ģenerēts unikāls 32 baitu tokens ar 1 stundas derīgumu.	
3. Tokens tiek saglabāts password_resets tabulā.	

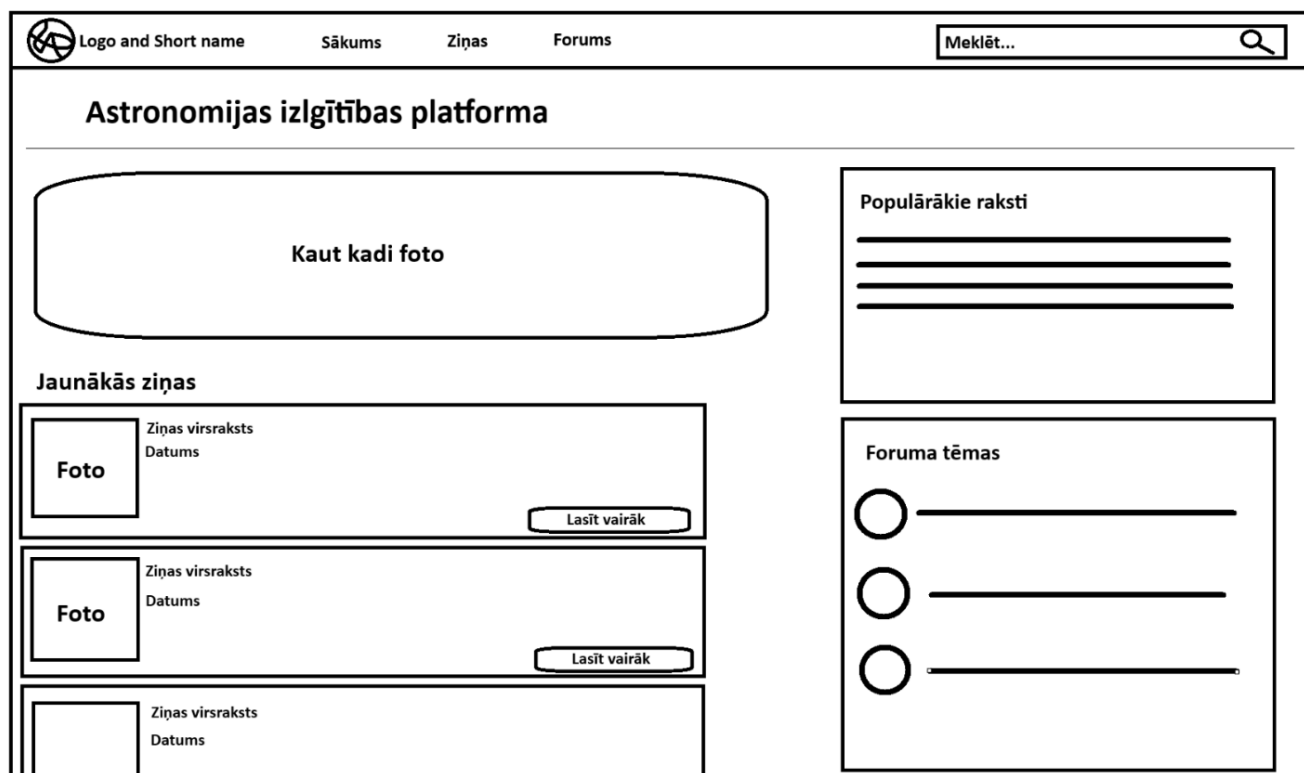
4. Uz e-pastu tiek nosūtīta atjaunošanas saite caur Resend API.
5. Pēc tokena validācijas jaunā parole tiek hešēta; tokens atzīmēts kā izmantots.
Izvaddati
1. Apstiprinājums: "Pārbaudiet savu e-pastu."
2. Pēc veiksmīgas atjaunošanas - novirzīšana uz pieteikšanās lapu.
Kļūdas paziņojumi
1. "Nederīgs vai novecojis tokens" - ja saite beigusi derīgumu.
2. "Parole ir pārāk īsa" - ja jaunā parole īsāka par 8 simboliem.
Izsaucamās prasības
Nav izsaucamo prasību.

2.3. Nefunkcionālās prasības

- Valoda: Visas sistēmas saskarnes teksti jābūt latviešu valodā. Kļūdu paziņojumiem, pogām, etiķetēm un navigācijas elementiem jābūt noformētiem latviski.
- Responsīvs dizains: Sistēmai automātiski jāpielāgojas dažādiem ekrāna izmēriem - datoram (min. 1024px), planšetdatoram (768px-1024px) un mobilajām ierīcēm (līdz 767px). Izkārtojums nedrīkst lauzt vai pārklāties nevienā no šiem izmēriem.
- Pārlūku saderība: Sistēmai jādarbojas korekti šādos pārlūkos - Google Chrome 89+, Mozilla Firefox 86+, Microsoft Edge 89+, Safari 14+. Visas galvenās funkcionalitātes jābūt pieejamām katrā no tiem.
- Ātrdarbība: Galveno lapu ielādes laiks normālos apstākļos nedrīkst pārsniegt 2 sekundes. API atbildes laikam jābūt zem 500 milisekundēm.
- Drošība: Lietotāju paroles jāglabā hešētā formātā (bcrypt). API galapunkti, kas prasa autentifikāciju, jāaizsargā ar JWT tokenu validāciju. Visi lietotāja ievaddati jāsanitizē, lai novērstu XSS un SQL injekcijas uzbrukumus.
- Pieejamība: Interfeisam jābūt intuitīvam un viegli saprotamam iesācējiem. Kļūdu paziņojumiem jābūt skaidriem un informatīviem. Formas laukiem jābūt ar aprakstošām etiķetēm.
- Uzturēšana: Koda bāzei jābūt organizētai pēc MVC principiem (backend) un komponentu arhitektūras (frontend). Kods jāglabā versiju kontroles sistēmā Git.

2.4. Skices

Pirmajā attēlā (skat. 2. att.) ir redzams tīmekļa vietnes sākulapas sākotnējais uzmetums, kurā definēta lapas struktūra: navigācijas josla ar meklēšanas funkciju, galvenais informācijas bloks "Astronomijas izglītības platforma" un satura izkārtojums ar jaunāko ziņu un foruma sadaļām. Savukārt pārējās divas skices (skat. 3. att.) attēlo lietotāja autorizācijas procesa vizuālo izskatu, ietverot reģistrācijas formu ar datu ievades laukiem un pieteikšanās logu ar "Atcerēties mani" un paroles atjaunošanas funkcijām.



2.att. Sākotnējais dizains lapai

Registration

Enter your name

Enter your email

Create a password

Confirm a password

☐ I accept all terms & conditions

Register Now

Already have an account? Login now

Login

Enter your email

Confirm a password

☐ Remember me

Forgot Password?

Login Now

Don't have an account? Signup now

3.att. Pieteikšanās un reģistrācijas forma

3. UZDEVUMA RISINĀŠANAS LĪDZEKĻU IZVĒLES PAMATOJUMS

Sistēmas izstrādei tiks izmantotas mūsdienīgas un populāras tehnoloģijas, kas nodrošina stabilu darbību un ērtu izstrādi.

Astronomijas izglītības platforma ir tīmekļa lietojumprogramma ar divu daļu arhitektūru - klientpuse (frontend) un serverpuse (backend) - ko savieno REST API. Datu glabāšanai tiek izmantota relāciju datu bāze. Šāda arhitektūra nodrošina skaidru atbildību sadalījumu, ērtāku uzturēšanu un labu veiktspēju. Sīkāks rīku apraksts ar versijām un izmantošanu sistēmā ir atrodams tabulā 3. nodaļas noslēgumā.

Serverpuses (backend) izstrādei tika izvēlēts Node.js v22 LTS kopā ar Express.js v5.1 ietvaru. Node.js ir ātra, notikumu virzīta JavaScript izpildes vide, kas labi piemērota REST API veidošanai ar biežu datu apmaiņu. Express.js nodrošina vienkāršu maršrutu un starpprogrammatūras (middleware) pārvaldību. Salīdzinājumā ar Python/Django, Node.js ir vieglāks konfigurēšanā un pilnībā integrējas ar to pašu JavaScript ekosistēmu, ko izmanto frontend pusē - tas samazina konteksta pārslēgšanas izmaksas izstrādes laikā.

Klientpuses (frontend) izstrādei tika izvēlēts React v19. React ļauj veidot dinamisku lietotāja saskarni ar atkārtoti izmantojamiem komponentiem, kas ievērojami paātrina izstrādi un atvieglo uzturēšanu. React Hooks nodrošina efektīvu stāvokļa pārvaldību bez papildu bibliotēkām.

Datu glabāšanai tika izvēlēta PostgreSQL v18.3 - uzticama relāciju datu bāzes pārvaldības sistēma ar transakciju atbalstu un jaudīgiem vaicājumu iespējām. Sequelize v6.37 ORM atvieglo datu bāzes modeļu definīciju un CRUD operācijas, abstrahējot tiešo SQL rakstīšanu.

Drošībai tika izvēlēts JSON Web Token (JWT) standarts sesiju pārvaldībai bez serverpusē glabātas stāvokļa informācijas, un bcrypt v5.1 paroles hešēšanai, kas nodrošina aizsardzību pret vārdnīcu uzbrukumiem. Izstrādes vide: Visual Studio Code v1.100. Versiju kontrolei: Git v2.54 un GitHub - izmantots izstrādes vēstures glabāšanai un projekta nodošanai vērtēšanai. Repozitorijs: <https://github.com/AleksandrsVodnevs/astronomy-platform>.

3.1. Izmantoto rīku detalizēts apraksts

Tabulā norādītas visas izstrādē izmantotās tehnoloģijas ar to versijām, raksturojumu un lomu sistēmā.

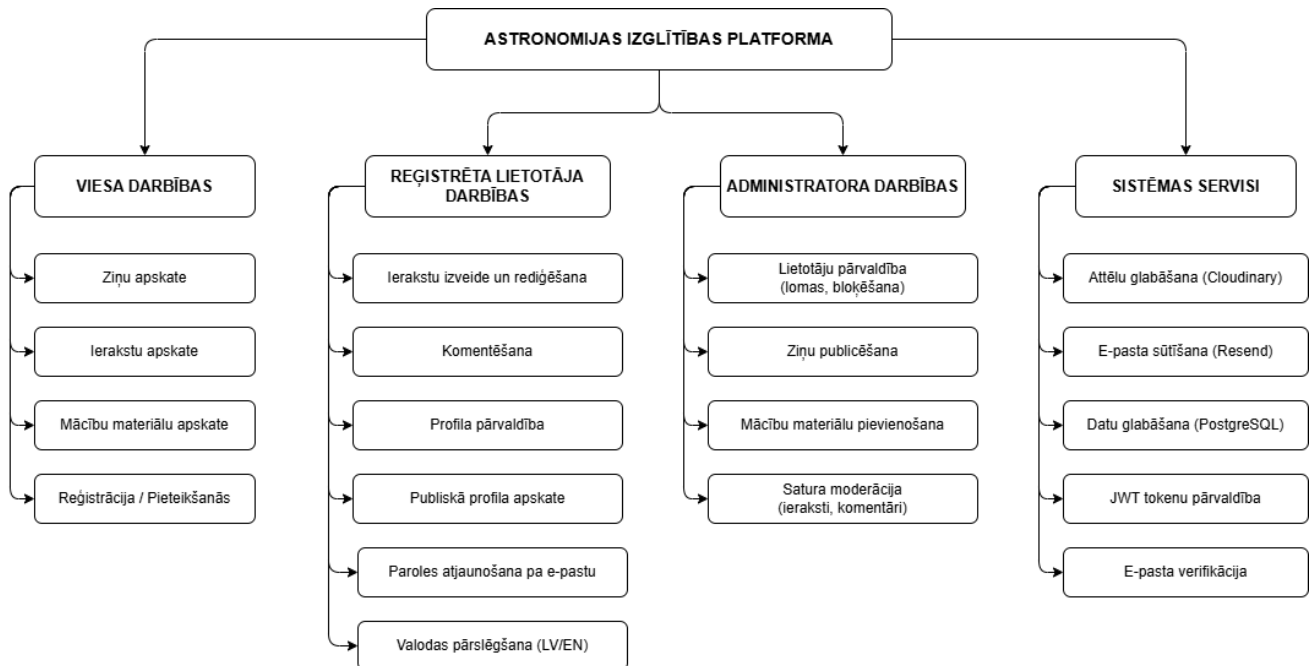
Rīks / Ietvars	Versija	Raksturojums	Izmantošana sistēmā
Node.js	v22 LTS	JavaScript izpildes vide servera pusē. Ātra, notikumu virzīta arhitektūra, piemērota API izstrādei.	Backend REST API izstrāde un servera loģikas izpilde.
Express.js	v5.1	Minimālistisks Node.js ietvars HTTP serveru un maršrutu veidošanai.	API maršrutu definīcija, starpprogrammatūras (middleware) apstrāde.
React	v19	JavaScript bibliotēka dinamisku lietotāja interfeisu veidošanai ar komponentu arhitektūru.	Visa frontend daļa: navigācija, formas, lapu komponenti.
PostgreSQL	v18.3	Jaudīga relāciju DBVS ar atbalstu sarežģītiem vaicājumiem un transakcijām.	Visu sistēmas datu (lietotāji, ieraksti, komentāri) glabāšana.
Sequelize	v6.37	ORM (objektu-relāciju kartēšanas) bibliotēka Node.js; abstrahē SQL vaicājumus.	Datu bāzes modeļu definīcija un CRUD operāciju veikšana bez tieša SQL.
JSON Web Token	v9.0	Standarts lietotāja sesiju pārvaldībai, izmantojot parakstītus tokenus.	Lietotāja autentifikācija un aizsargātu API galapunktu drošība.
bcrypt	v5.1	Bibliotēka paroles hešošanai ar sāls pievienošanu, aizsargājoties pret vārdārn. uzbrukumiem.	Lietotāju paroli droši saglabāšana datu bāzē.
Visual Studio Code	v1.100	Viegls, plašām papildinājumu iespējām bagāts koda redaktors.	Galvenā izstrādes vide (IDE) kāda frontend, tā backend kodam.
Git	v2.54	Versiju kontroles sistēma koda izmaiņu sekošanai.	Izstrādes vēstures glabāšana, zarošana, funkcijās izstrāde.
GitHub	-	Attālinātā Git repozitorija platforma ar projektu pārvaldības rīkiem.	Koda bāzes publikācija, darba vadītāja piekļuve projektam.
Cloudinary	Free	Mākoņpakalpojums attēlu glabāšanai un apstrādei. Nodrošina automātisku attēlu optimizāciju.	Lietotāju, ierakstu, ziņu un materiālu attēlu augšupielāde un glabāšana.
Resend	v4.1	E-pasta sūtīšanas pakalpojums ar REST API. Nodrošina uzticamu e-pasta piegādi no pielāgota domēna.	Paroles atjaunošanas e-pastu un e-pasta verifikācijas kodu sūtīšana.
react-markdown	v9.0	React komponents Markdown formāta teksta atveidošanai HTML.	Mācību materiālu satura formatēta attēlošana ar virsrakstiem, sarakstiem, tabulām.
Three.js	r128	JavaScript bibliotēka 3D grafiku renderēšanai pārlūkprogrammā, izmantojot WebGL.	Interaktīvās 3D Saules sistēmas vizualizācijas izveide ar planētu orbītām, tekstūrām un informācijas paneli.
NASA APOD API	-	NASA bezmaksas API ikdienas astronomijas attēla iegūšanai ar aprakstu.	Sāklumlapas APOD bloks - ikdienas astronomijas attēls ar aprakstu un saiti uz NASA.

4. PROGRAMMATŪRAS PRODUKTA MODELĒŠANA UN PROJEKTĒŠANA

4.1. Sistēmas struktūras modelis

4.1.1. Sistēmas arhitektūra

Astronomijas izglītības platforma ir veidota kā daudzlīmeņu tīmekļa sistēma, kuras struktūra un funkcionalitāte ir sadalīta četrās galvenajās sadaļās (trīs lietotāju lomu apakšsistēmās un vienā tehniskā servisu modulī). Šāds iedalījums nodrošina skaidru atbildību un funkciju sadalījumu, vienkāršo sistēmas uzturēšanu, kā arī strukturē tehnoloģisko risinājumu kopumu. Funkcionālās dekompozīcijas hierarhiskā diagramma uzskatāmi attēlo šo arhitektūru un ilustrē katra moduļa sastāvdaļas.



4.att. Funkcionālās dekompozīcijas diagramma

1. Viesa darbības

Šī apakšsistēma apkalpo neautenticētos lietotājus (viesus). Viesis var apskatīt publiskās ziņas un ierakstus, pārlūkot mācību materiālus ierobežotā režīmā, kā arī piekļūt reģistrācijas un pieteikšanās veidlapai. Šis modulis ir publiski piekļūstams un nav atkarīgs no autentifikācijas.

2. Reģistrētā lietotāja darbības

Reģistrēts lietotājs pēc autentifikācijas iegūst plašākas piekļuves tiesības. Šī apakšsistēma ietver ierakstu izveidi un rediģēšanu, komentāru pievienošanu pie citiem ierakstiem, profila pārvaldību (apraksts, interešu joma, zināšanu līmenis), kā arī paplašinātu satura piekļuvi salīdzinājumā ar viesu lomu.

3. Administratora darbības

Administrators ir lietotājs ar visaugstākajiem piekļuves līmeņiem. Šī apakšsistēma nodrošina lietotāju saraksta pārvaldību (lomešana, bloķēšana), visu sistēmas ierakstu un komentāru moderāciju, kā arī ziņu ievietošanu un rediģēšanu administrācijas panelī. Administrators piekļūst īpašam administrācijas interfeisam, kas nav pieejams pārējiem lietotājiem.

4. Sistēmas servisi

Šis ir tehnisko un aizmugursistēmas (backend) servisu modulis, kas darbojas fonā un nodrošina platformas funkcionālo un tehnoloģisko pamatbāzi, integrējot ārējos pakalpojumus un datu apstrādes rīkus

4.1.2. Sistēmas ER modelis

Datu bāzes projektēšanā datu kopu un saišu starp tām attēlošanai tika izmantota realitāšu-saišu diagramma (Entity-Relationship diagramma). ER modelis sastāv no entītijām, kas attēlo sistēmā apstrādājamās objektus, un relācijām, kas raksturo šo objektu savstarpējās attiecības.

Izstrādātās sistēmas ER diagramma (sk. pielikumā) sastāv no vairākām galvenajām entītijām, kas atspoguļo datu apriti astronomijas izglītības platformā.

Sistēmā tiek izmantotas šādas entītijas:

- Lietotājs - uzskaita visus sistēmā reģistrētos lietotājus. Entītija satur informāciju par lietotāja vārdu, uzvārdu, e-pastu, paroli, reģistrācijas datumu un konta statusu.
- Administrators - administrators ir specializēts lietotāja tips ar paplašinātām sistēmas pārvaldības tiesībām.
- Profils - satur papildu informāciju par lietotāju, piemēram, profila aprakstu, interešu jomu un zināšanu līmeni astronomijā.
- Ieraksts - uzskaita lietotāju publicētos ierakstus par astronomijas tēmām. Entītija satur virsrakstu, saturu, publicēšanas datumu un statusu.

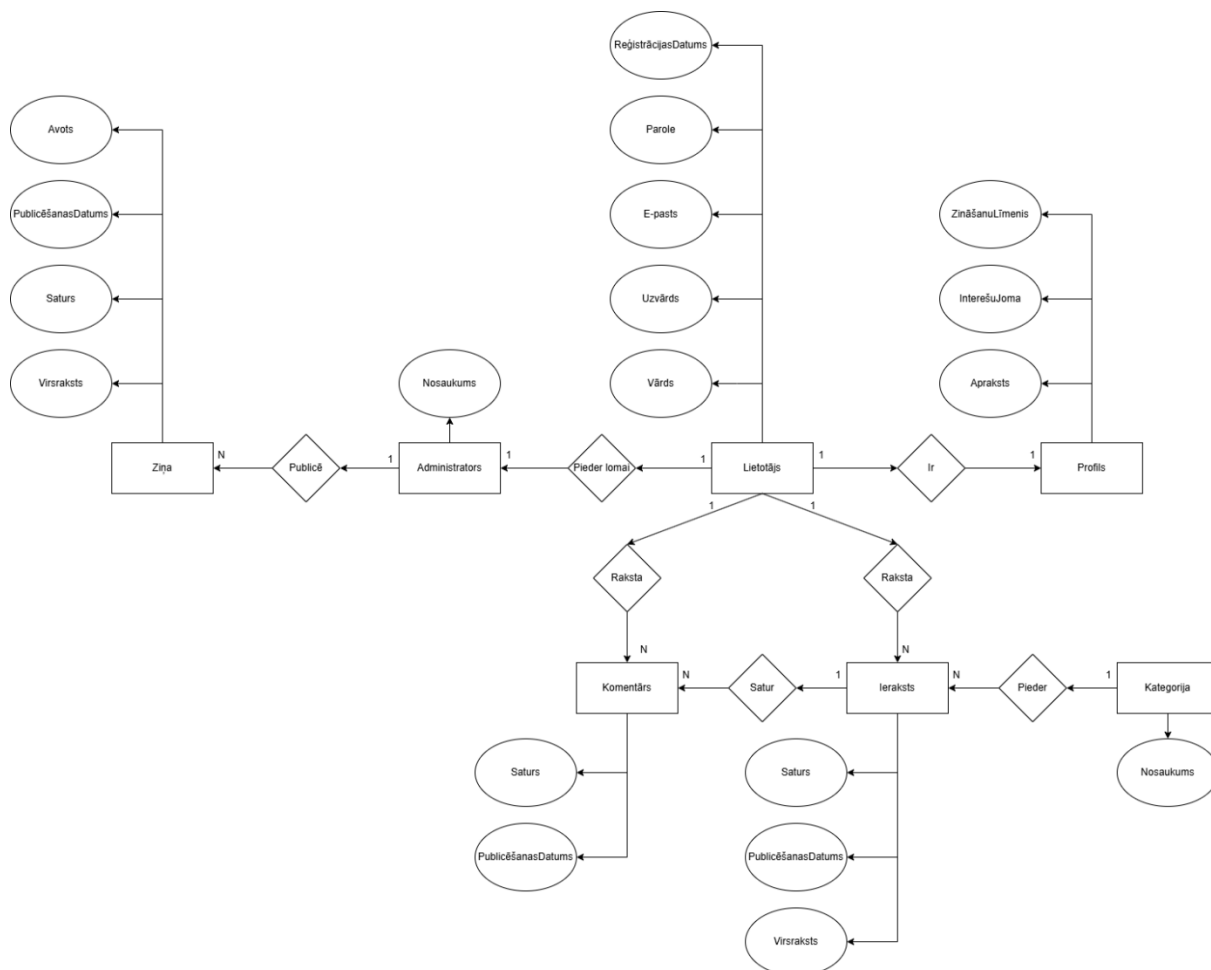
- Komentārs - uzskaita lietotāju pievienotos komentārus pie ierakstiem. Tajā tiek glabāts komentāra saturs un publicēšanas datums.
- Ziņa - satur administratora publicētos informatīvos paziņojumus vai jaunumus platformā.
- Kategorija - definē ierakstu tematisko iedalījumu (piem., Planētas, Galaktikas, Kosmoss, Astrofizika).

Relācijas starp entītijām

Datu bāzes relācijas parāda, kā entītijas ir savstarpēji saistītas un kā notiek datu aprīte sistēmā.

- Starp Lietotājs un Loma pastāv attiecība daudzi pret vienu (N:1), jo daudzi lietotāji var piederēt vienai lomai, bet katram lietotājam ir tikai viena loma.
- Starp Lietotājs un Profils pastāv attiecība viens pret vienu (1:1), jo katram lietotājam ir viens profils un katrs profils pieder tikai vienam lietotājam.
- Starp Lietotājs un Ieraksts pastāv attiecība viens pret daudziem (1:N), jo viens lietotājs var izveidot vairākus ierakstus, bet katrs ieraksts pieder tikai vienam konkrētam lietotājam.
- Starp Ieraksts un Komentārs pastāv attiecība viens pret daudziem (1:N), jo vienam ierakstam var būt vairāki komentāri, bet katrs komentārs ir piesaistīts vienam konkrētam ierakstam.
- Starp Lietotājs un Komentārs pastāv attiecība viens pret daudziem (1:N), jo lietotājs var pievienot vairākus komentārus, bet katrs komentārs pieder vienam lietotājam.
- Starp Ieraksts un Kategorija pastāv attiecība daudzi pret vienu (N:1), jo vairāki ieraksti var piederēt vienai kategorijai, bet katram ierakstam ir viena galvenā kategorija.

- Starp Administrators (kas ir Lietotāja lomas paveids) un Ziņa pastāv attiecība viens pret daudziem (1:N), jo administrators var publicēt vairākas ziņas, bet katru ziņu publicē viens administrators.



5.att. ER diagramma

4.2. Funkcionālais sistēmas modelis

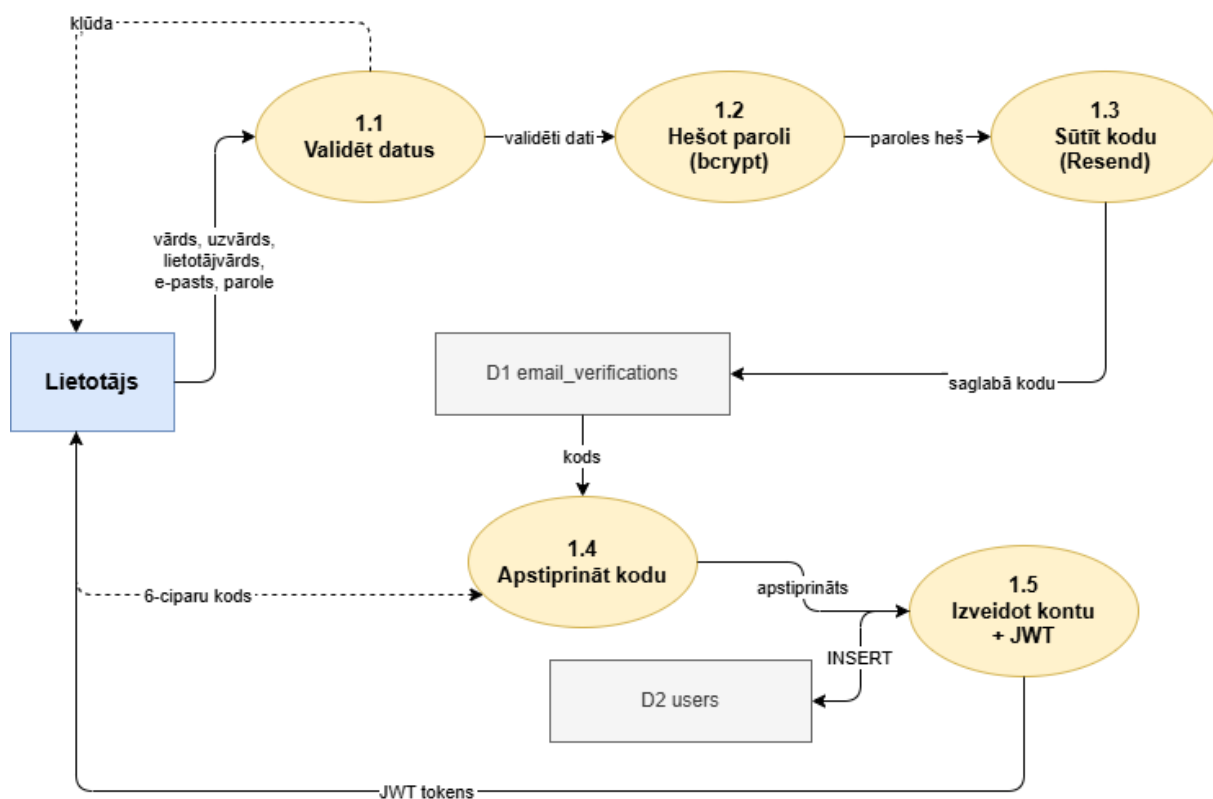
4.2.1. Datu plūsmu modelis

Datu plūsmu diagrammas attēlo, kā informācija ieviesto funkcionalitāšu ietvaros pārvietojas starp lietotāju, sistēmas procesiem un datu glabātuvi. Zemāk aprakstītas trīs pamatfunkcionalitātes, kurām izstrādātas datu plūsmu diagrammas.

1. Lietotāja reģistrācija

Lietotājs ievada reģistrācijas datus (vārds, uzvārds, e-pasts, parole). Sistēma veic ievaddatu validāciju - pārbauda e-pasta formātu un paroles garumu. Ja validācija ir veiksmīga, parole tiek hešēta un jauns lietotāja ieraksts tiek saglabāts datu bāzē. Sistēma piekšķir lietotājam lomu "Reģistrēts lietotājs" un nosūta apstiprinājumu. Ja e-pasts jau eksistē, sistēma

atgriež kļūdas paziņojumu. Procesi: (1) Datu ievade → (2) Formāta validācija → (3) Unikālītātes pārbaude DB → (4) Paroles hešēšana → (5) Ieraksta saglabāšana → (6) Atbildes sniegšana.

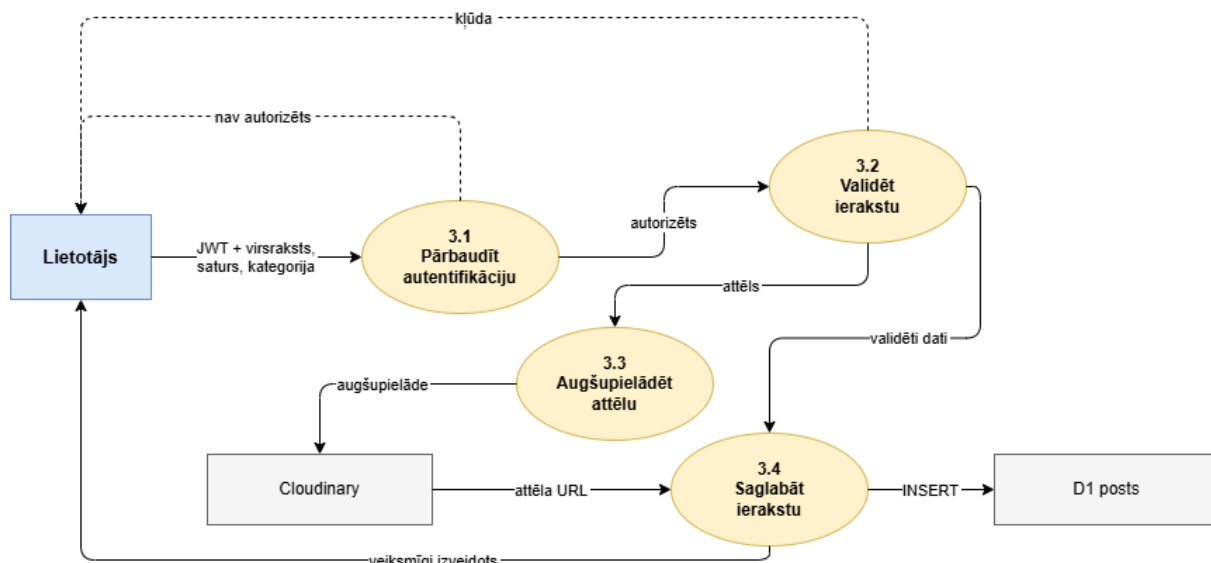


6. att. DFD diagramma: Lietotāja reģistrācija

2. Ieraksta izveide

Reģistrēts lietotājs ievada jauna ieraksta datus (virsraksts, saturs, kategorija, statuss). Sistēma pārbauda autentifikāciju - ja sesija nav spēkā, lietotājs tiek novirzīts uz pieteikšanās lapu. Derīgos datos notiek satura sanitācija (aizsardzība pret XSS), pēc tam ieraksts tiek saglabāts ar autora ID un laika zīmogu. Ja statuss ir “Publicēts”, ieraksts kļūst redzams platformā. Procesi: (1) Autentifikācijas pārbaude → (2) Datu validācija → (3) Satura sanitācija → (4) Ieraksta saglabāšana DB → (5) Statusa apstrāde → (6) Atbildes par

saglabāšanu sniegšana.

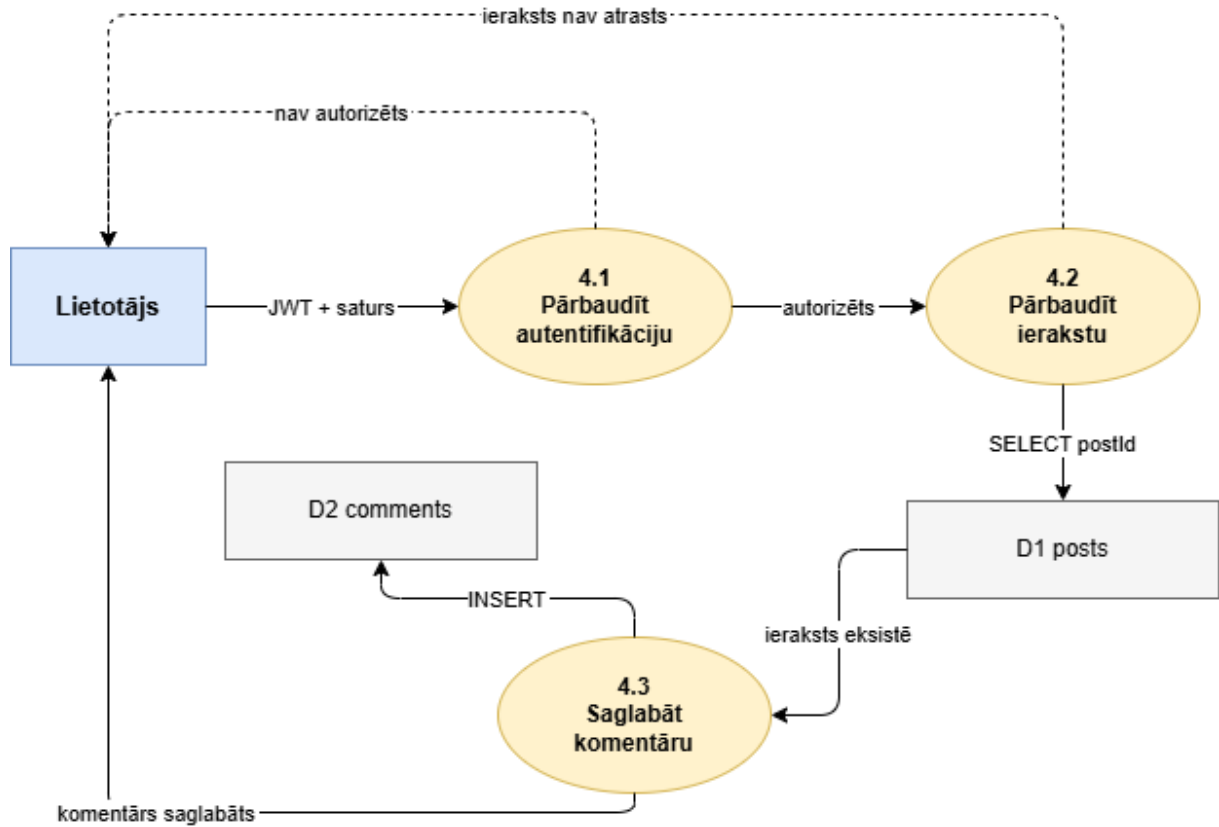


7. att. DFD diagramma: Ieraksta izveide

3. Komentāra pievienošana

Reģistrēts lietotājs atver ieraksta lapu un ievada komentāra tekstu (līdz 500 rakstzīmēm). Sistēma autentificē pieprasījumu un validē ievaddatu garumu. Veiksmīgas validācijas gadījumā komentārs tiek saglabāts datu bāzē ar autora ID, ieraksta ID un publicēšanas laika zīmogu. Pēc saglabāšanas tiek atjaunināts komentāru skaita rādītājs pie ieraksta. Procesis: (1) Pieprasījuma autentifikācija → (2) Teksta validācija → (3) Satura sanitācija → (4) Komentāra saglabāšana DB → (5) Skaita atjaunināšana → (6) Atveidojuma

atsvaidzināšana.

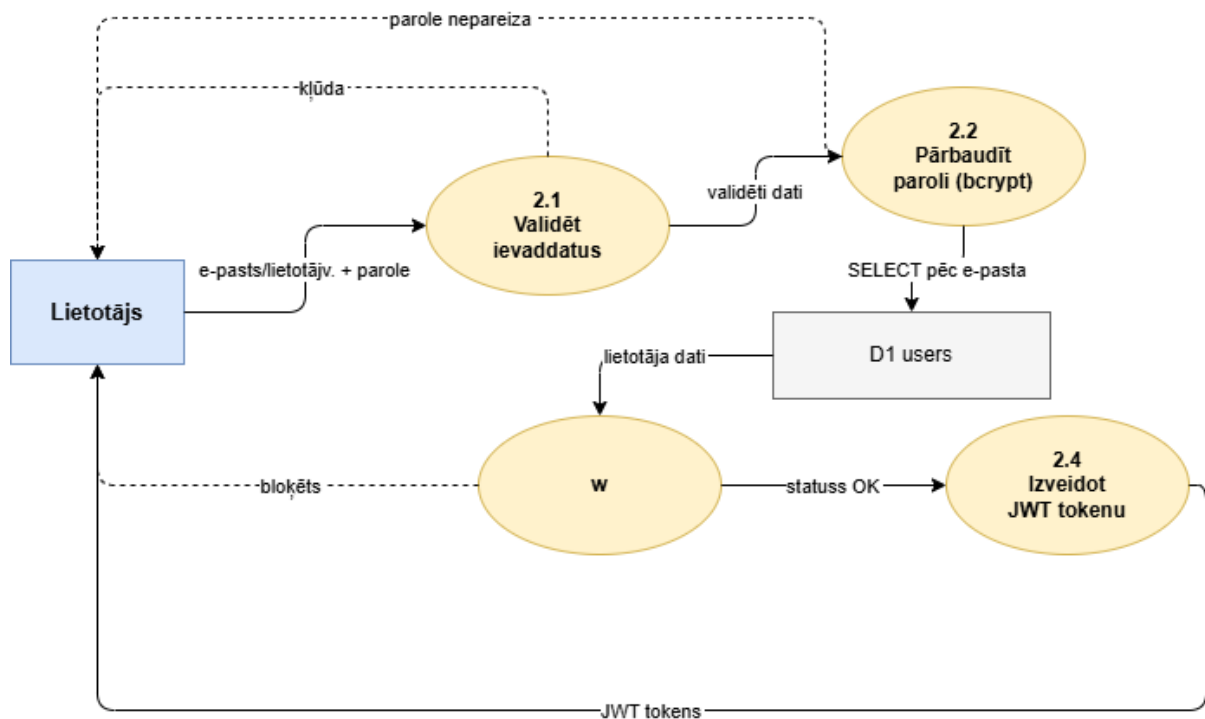


8. att. Komentāra pievienošana

4. Lietotāja pieteikšanās (autentifikācija)

Lietotājs ievada e-pastu un paroli pieteikšanās formā. Sistēma pārbauda, vai lietotājs ar šādu e-pastu eksistē datu bāzē. Ja eksistē, ievadītā parole tiek salīdzināta ar saglabāto hešu. Veiksmīgas pārbaudes gadījumā tiek izgūts lietotāja profils un ģenerēts JWT tokens. Ja kaut kas neatbilst, tiek atgriezts kļūdas paziņojums. Procesi: (1) Datu ievade → (2) Lietotāja

meklēšana DB → (3) Paroles salīdzināšana → (4) JWT ģenerēšana → (5) Sesijas izveide → (6) Atbildes sniegšana.



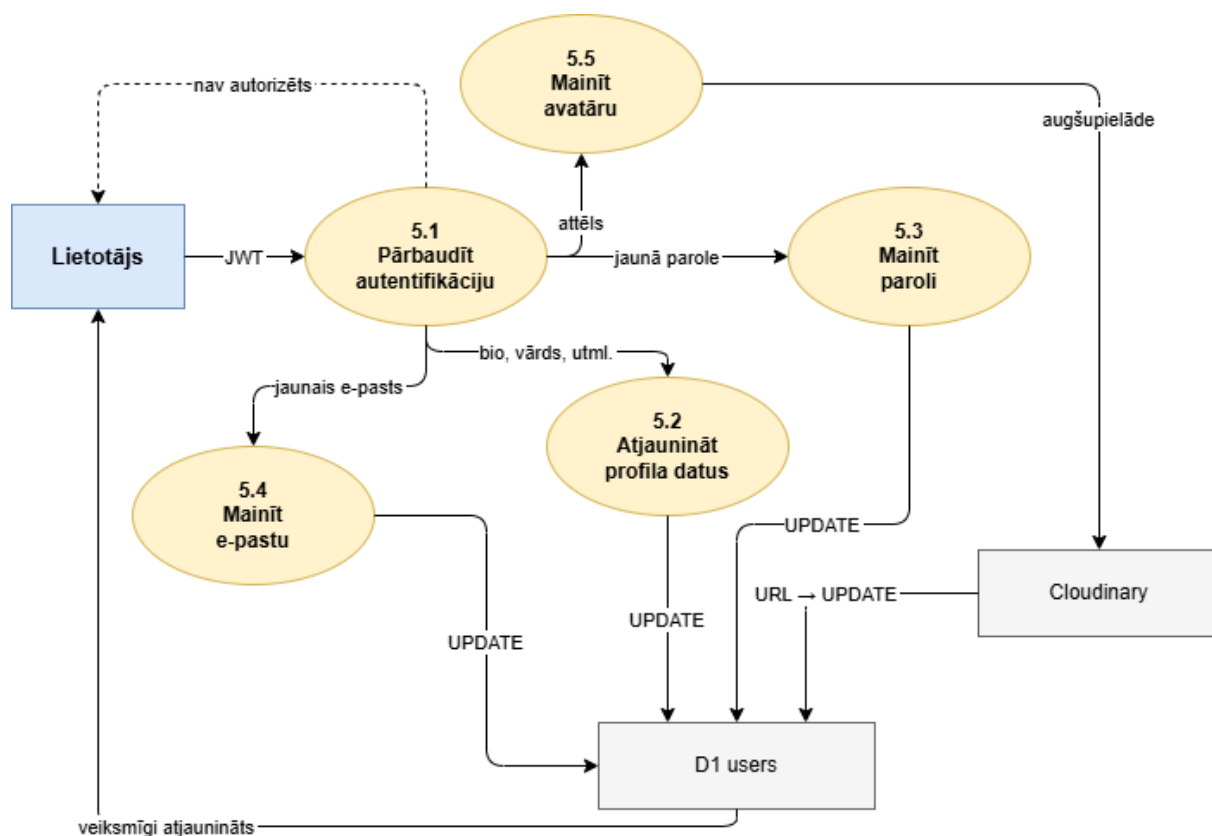
9. att. DFD diagramma: Lietotāja pieteikšanās (autentifikācija)

5. Profila rediģēšana

Reģistrēts lietotājs atver profila rediģēšanas lapu. Sistēma ielādē esošos profila datus no datu bāzes un attēlo tos formā. Lietotājs veic izmaiņas (apraksts, interešu joma, zināšanu līmenis) un apstiprina saglabāšanu. Sistēma validē ievaddatus un saglabā jaunus datus.

Procesi: (1) Autentifikācijas pārbaude → (2) Esošo datu ielāde → (3) Izmainu validācija →

(4) Profila atjaunināšana DB → (5) Apstiprinājuma sniegšana.

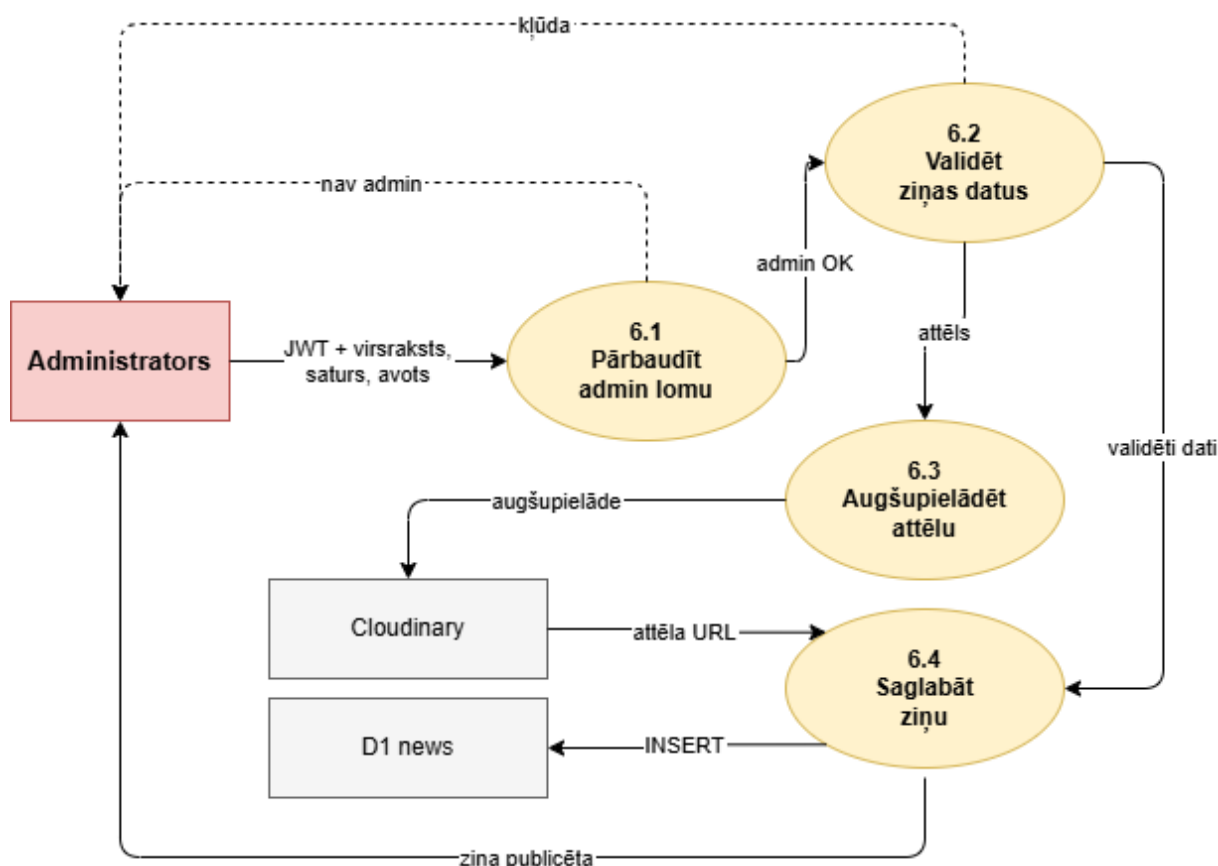


10. att. DFD diagramma: Profila rediģēšana

6. Ziņas ievietošana (administrators)

Administrators piesakās administrācijas panelī un atver ziņu izveides formu. Ievada ziņas virsrakstu, saturu un avota saiti. Sistēma validē URL formātu un lauku garumu. Pēc veiksmīgas validācijas ziņa tiek saglabāta ar laika zīmogu un administratora ID, un kļūst redzama platformā. Procesis: (1) Administratora autentifikācija → (2) Datu ievade un

validācija → (3) URL formāta pārbaude → (4) Ziņas saglabāšana DB → (5) Publicēšana saskarnē.

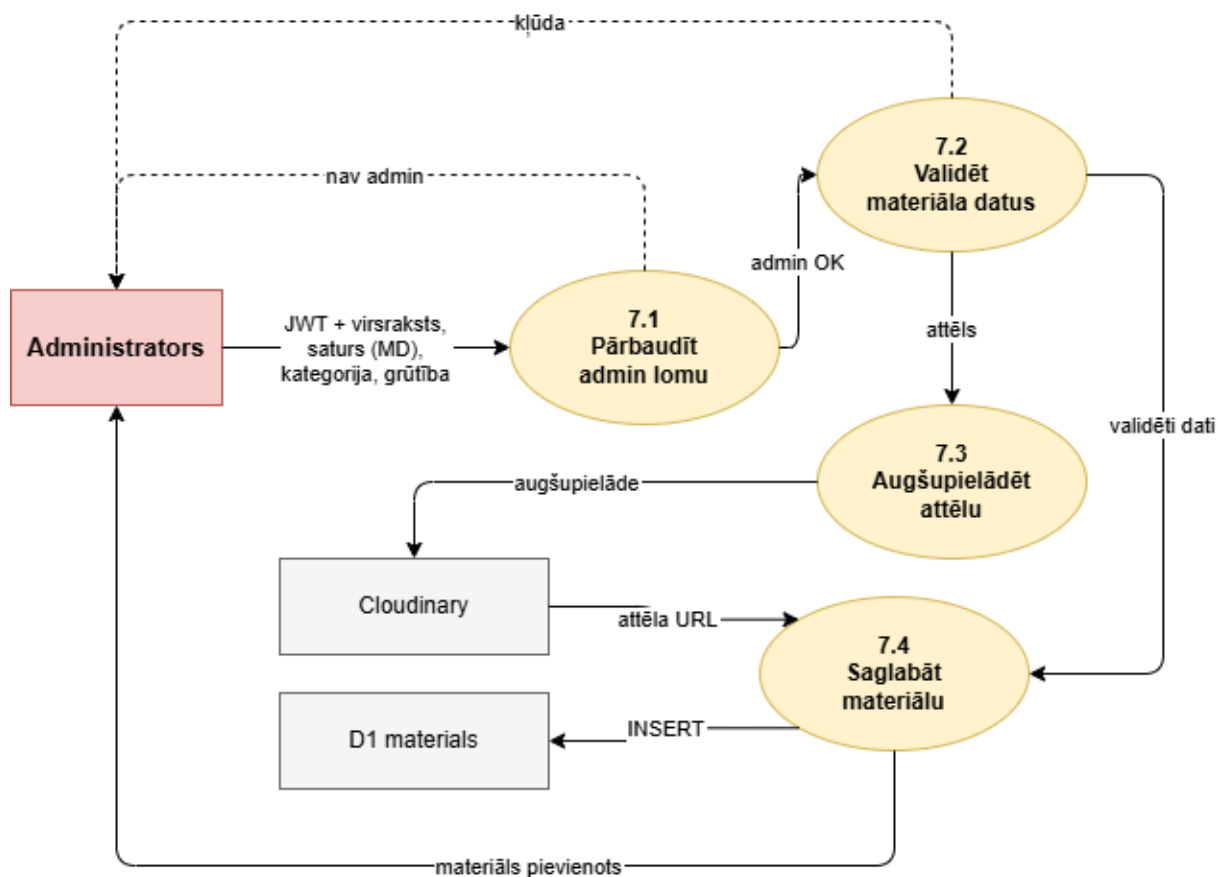


11. att. DFD diagramma: Ziņas ievietošana (administrators)

7. Mācību materiāla pievienošana (administrators)

Administrators iesniedz datus (JWT, virsrakstu, saturu (MD), kategoriju, grūtību). Sistēma pārbauda administratora lomu; ja pārbaude neizdodas, process tiek pārtraukts. Pēc sekmīgas lomas apstiprināšanas tiek veikta materiāla datu validācija (kļūdas gadījumā administrators saņem paziņojumu). Validētais attēls tiek augšupielādēts Cloudinary platformā, kas atgriež attēla URL. Noslēgumā visi validētie dati un attēla saite tiek saglabāti datubāzē (D1 materials), un administrators saņem apstiprinājumu.

Procesi: (1) Pārbaudīt admin lomu → (2) Validēt materiāla datus → (3) Augšupielādēt attēlu → (4) Saglabāt materiālu DB un atgriezt apstiprinājumu



12. att. DFD diagramma: Mācību materiāla pievienošana (administrators)

5. DATU STRUKTŪRU APRAKSTS

Astronomijas izglītības platformas datu bāze ir veidota, izmantojot PostgreSQL relāciju datu bāzes pārvaldības sistēmu — vienu no uzticamākajām un funkcijām bagātākajām atvērtā koda risinājumiem. PostgreSQL tika izvēlēts tā ACID atbilstības, ārējo atslēgu atbalsta, JSON lauku iespēju un lieliskās saderības ar Sequelize ORM bibliotēku dēļ. Datu bāze tiek mitināta Neon.tech serverless platformā, kas nodrošina automātisku mērogošanu un augstas pieejamības garantijas.

Datu bāze sastāv no septiņām savstarpēji saistītām tabulām, kas kopā veido pilnīgu datu modeli visām platformas funkcijām — lietotāju pārvaldībai, satura publicēšanai, komentēšanai, ziņu ievietošanai, izglītojošo materiālu glabāšanai un drošības mehānismiem. Tabulu saišu shēma ir atrodama pielikumā. Katras tabulas lauki ir projektēti ar minimālās datu dublēšanas principu, izmantojot ārējās atslēgas (foreign keys) savstarpējo saišu nodrošināšanai.

Datu bāze satur šādas tabulas: Users (lietotāji), Posts (forumā ieraksti), Comments (komentāri), News (ziņas), Materials (mācību materiāli), Email_verifications (e-pasta verifikācijas kodi) un Password_resets (paroles atjaunošanas tokeni).

Datu bāze satur šādas tabulas:

1. Tabula "Users" - glabā informāciju par reģistrētiem lietotājiem (vārds, uzvārds, e-pasts, paroles heš, loma, statuss, avatars, bio, reģistrācijas datums).
2. Tabula "Posts" - uzskaita lietotāju publicētos ierakstus (virsraksts, saturs, kategorija, statuss, datums).
3. Tabula "Comments" - glabā komentārus pie ierakstiem (saturs, autors, datums, ieraksta ID, attēla URL).
4. Tabula "News" - satur administratora publicētās ziņas (virsraksts, saturs, avota saite, datums).
5. Tabula "Materials" - glabā izglītojošos materiālus (virsraksts, saturs, kategorija, grūtības pakāpe, administratora ID, attēla URL, datums).
6. Tabula "Email_verifications" - saglabā e-pasta verifikācijas kodus lietotāju reģistrācijai un e-pasta maiņai (lietotāja ID, e-pasts, kods, tips, derīguma termiņš).
7. Tabula "Password_resets" - glabā paroles atjaunošanas tokenus (lietotāja ID, tokens, derīguma termiņš, izmantošanas statuss).

Tabulas "Users" struktūra

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
1	id	SERIAL	PK	Ieraksta identifikators (primārā atslēga)
2	first_name	VARCHAR	30	Lietotāja vārds
3	last_name	VARCHAR	30	Lietotāja uzvārds
role	VARCHAR	30	Lietotāja loma sistēmā. Iespējamās vērtības: guest, user, admin (NOT NULL, pēc noklusējuma: user)	-
4	email	VARCHAR	100	Unikāla e-pasta adrese (NOT NULL, UNIQUE)
5	password_hash	VARCHAR	255	Hešēta parole (bcrypt)
6	created_at	TIMESTAMP	-	Reģistrācijas datums un laiks
7	status	VARCHAR	20	Konta statuss (aktīvs / bloķēts)

Tabula "Users" ir saistīta ar tabulām "Posts" un "Comments" attiecībā viens pret daudziem (viens lietotājs var radīt vairākus ierakstus un komentārus), ar tabulu "News" attiecībā viens pret daudziem, kā arī ar tabulu "Materials" attiecībā viens pret daudziem.

Tabulas "Posts" struktūra

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
1	id	SERIAL	PK	Ieraksta identifikators (primārā atslēga)
2	title	VARCHAR	100	Ieraksta virsraksts (NOT NULL)
3	content	TEXT	3000	Ieraksta saturs
4	category_id	INT	FK	Atsauce uz tabulu "Categories"
5	author_id	INT	FK	Atsauce uz tabulu "Users"
6	status	VARCHAR	20	Statuss: melnraksts vai publicēts

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
image_url	VARCHAR	500	URL attēlam no Cloudinary (NULL - ja attēls nav pievienots)	-
7	created_at	TIMESTAMP	-	Publicēšanas datums un laiks

Tabula "Posts" ir saistīta ar "Users" attiecībā daudzi pret vienu (N:1), ar "Categories" attiecībā daudzi pret vienu (N:1) un ar "Comments" attiecībā viens pret daudziem (1:N).

5.3.tabula

Tabulas "Comments" struktūra

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
1	id	SERIAL	PK	Komentāra identifikators (primārā atslēga)
2	content	VARCHAR	500	Komentāra teksts (NOT NULL)
3	post_id	INT	FK	Atsauce uz tabulu "Posts"
4	author_id	INT	FK	Atsauce uz tabulu "Users"
image_url	VARCHAR	500	URL attēlam no Cloudinary (NULL - ja attēls nav pievienots)	-
5	created_at	TIMESTAMP	-	Pievienošanas datums un laiks

Tabula "Comments" ir saistīta ar "Posts" attiecībā daudzi pret vienu (N:1) un ar "Users" attiecībā daudzi pret vienu (N:1). Katrs komentārs pieder vienam ierakstam un vienam autoram.

5.4.tabula

Tabulas "News" struktūra

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
1	id	SERIAL	PK	Ziņas identifikators (primārā atslēga)
2	title	VARCHAR	150	Ziņas virsraksts (NOT NULL)
3	content	TEXT	-	Pilns ziņas teksts
4	source_url	VARCHAR	300	Avota URL saite (https:// formāts)
5	admin_id	INT	FK	Atsauce uz tabulu "Users" (administrators)

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
image_url	VARCHAR	500	URL vāka attēlam no Cloudinary (NULL - ja attēls nav pievienots)	-
6	created_at	TIMESTAMP	-	Publicēšanas datums un laiks

Tabula "News" ir saistīta ar "Users" attiecībā daudzi pret vienu (N:1) - katru ziņu publicē viens administrators, bet administrators var publicēt daudzas ziņas.

5.5.tabula

Tabulas "Materials" struktūra

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
1	id	SERIAL	PK	Materiāla identifikators (primārā atslēga)
2	title	VARCHAR	200	Materiāla virsraksts (NOT NULL)
3	content	TEXT	-	Materiāla saturs Markdown formātā
4	category	VARCHAR	50	Kategorija: Astronomija vai Astrofotogrāfija
5	difficulty	VARCHAR	20	Grūtības līmenis: Iesācējs, Vidējs, Pieredzējis
6	image_url	VARCHAR	500	URL vāka attēlam no Cloudinary (NULL - nav attēla)
7	admin_id	INT	FK	Atsauce uz tabulu "Users" (administrators)
8	created_at	TIMESTAMP	-	Pievienošanas datums un laiks

Tabula "Materials" ir saistīta ar "Users" attiecībā daudzi pret vienu (N:1) - katru materiālu pievieno viens administrators, bet administrators var pievienot daudzus materiālus.

5.6.tabula

Tabulas "Email_verifications" struktūra

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
1	id	SERIAL	PK	Ieraksta identifikators (primārā atslēga)
2	user_id	INT	FK	Atsauce uz tabulu "Users"
3	email	VARCHAR	100	E-pasta adrese, uz kuru nosūtīts kods
4	code	VARCHAR	10	6 ciparu verifikācijas kods (NOT NULL)
5	type	VARCHAR	30	Koda tips: register vai email_change

6	expires_at	TIMESTAMP	-	Koda derīguma termiņš (15 minūtes no izveides)
7	used	BOOLEAN	-	Vai kods jau izmantots (false - netiek; true - jau izmantots)

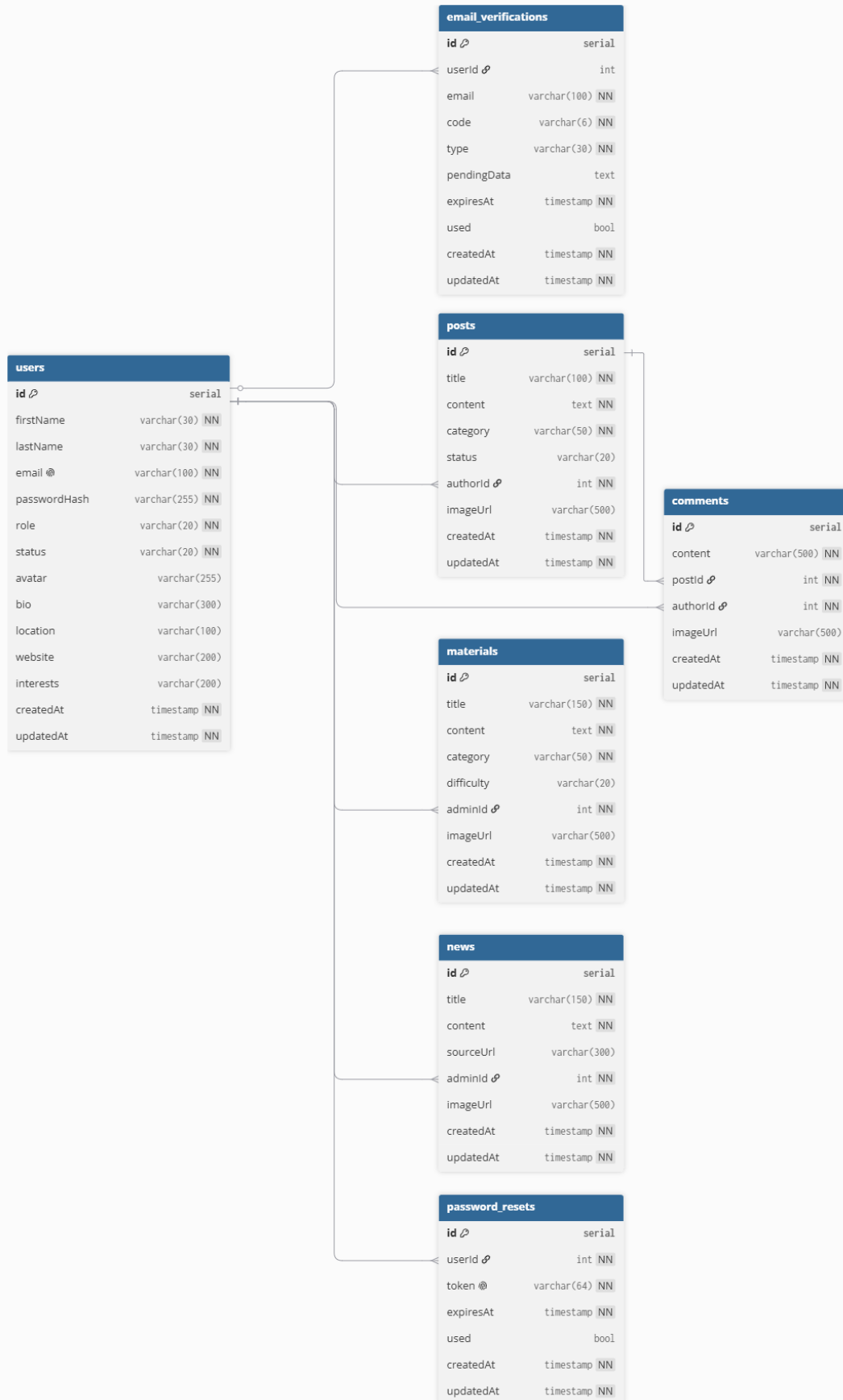
Tabula "Email_verifications" ir saistīta ar "Users" attiecībā daudzi pret vienu (N:1). Katrs verifikācijas kods pieder vienam lietotājam, bet vienam lietotājam var būt vairāki kodi (piem., reģistrācijas laikā un e-pasta maiņas laikā).

5.7.tabula

Tabulas "Password_resets" struktūra

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
1	id	SERIAL	PK	Ieraksta identifikators (primārā atslēga)
2	user_id	INT	FK	Atsauce uz tabulu "Users"
3	token	VARCHAR	255	Hešēts tokens paroles atjaunošanas saitei (NOT NULL)
4	expires_at	TIMESTAMP	-	Tokena derīguma termiņš (1 stunda no izveides)
5	used	BOOLEAN	-	Vai tokens jau izmantots
6	created_at	TIMESTAMP	-	Izveides datums un laiks

Tabula "Password_resets" ir saistīta ar "Users" attiecībā daudzi pret vienu (N:1). Katrs atjaunošanas tokens pieder vienam lietotājam. Pēc veiksmīgas paroles maiņas tokena lauks "used" tiek iestatīts uz true, padarot to nederīgu atkārtotai lietošanai.



13.att. Tabulu saišu shēma

6. LIETOTĀJA CEĻVEDIS

6.1. Sistēmas prasības aparatūrai un programmatūrai

Astronomijas izglītības platforma ir tīmekļa lietojumprogramma, kas paredzēta lietošanai pārlūkprogrammā. Sistēma atbilst šādām minimālajām prasībām:

- Operētājsistēma: Windows 10 vai jaunāka versija; macOS 10.15 vai jaunāka versija; jebkura moderna Linux versija.
- Interneta pārlūks: Google Chrome 89 vai jaunāka versija; Mozilla Firefox 86 vai jaunāka versija; Microsoft Edge 89 vai jaunāka versija; Safari 14 vai jaunāka versija.
- Interneta savienojums: nepieciešams stabils platjoslas interneta savienojums.
- Ekrāna izšķirtspēja: ieteicams vismaz 1280×720 pikseļi; platforma ir responsīva un darbojas arī mobilajās ierīcēs.

Sistēmas izstrādātāja videi (lokālajai darbināšanai) nepieciešams Node.js v22+, npm v11.6+, PostgreSQL v18.3+ un Git.

6.2. Sistēmas instalācija un palaišana

Lai instalētu un palaistu sistēmu lokālā izstrādes vidē, jāveic šādas darbības:

1. Klonēt projektu no GitHub: `git clone https://github.com/AleksandrsVodnevs/astronomy-platform.git`
2. Ieiet projekta mapē: `cd astronomy-platform`
3. Instalēt backend atkarības: `cd backend && npm install`
4. Konfigurēt vides mainīgos: izveidot failu `.env` pēc `.env.example` parauga, norādot datu bāzes savienojuma datus un `JWT_SECRET`.
5. Izveidot datu bāzi un palaist migrācijas: `npm run migrate`
6. Palaist backend serveri: `npm run dev` (serveris darbojas uz porta 5000).
7. Jaunā termināļa logā instalēt frontend atkarības: `cd ../frontend && npm install`
8. Palaist frontend: `npm run dev` (platforma pieejama pārlūkā `http://localhost:5173`).

Ja platforma ir izvietota publiskā hostingā, tā ir pieejama tieši pārlūkprogrammā, bez lokālas instalācijas.

6.3. Programmas apraksts

Atverot platformu pārlūkprogrammā, lietotājam tiek rādīta sākumlapa ar pēdējām ziņām un populārākajiem ierakstiem. Navigācijas joslā pieejamas sadaļas: Sākumlapa, Ziņas, Ieraksti, Mācību materiāli, Pieteikšanās/Reģistrācija.

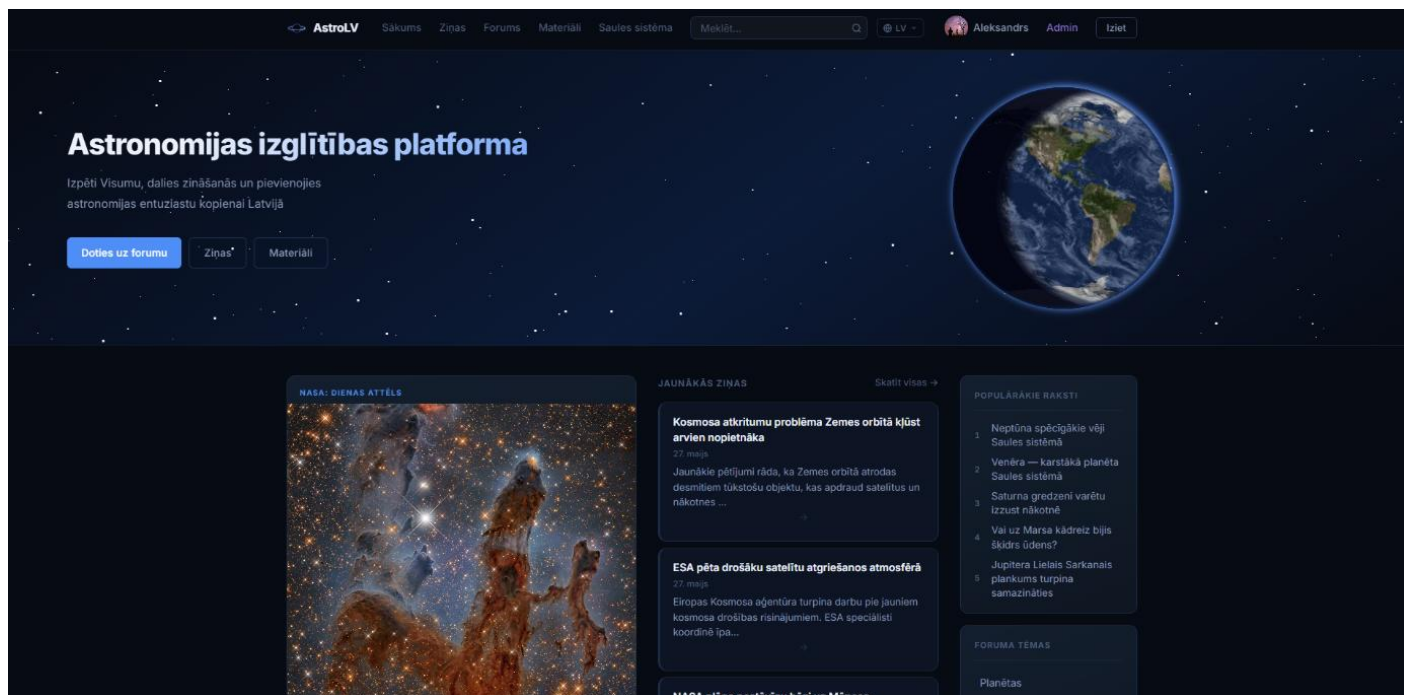
Reģistrējoties sistēmā, lietotājs aizpilda formu ar vārdu, uzvārdu, e-pasta adresi un paroli. Pēc veiksmīgas reģistrācijas sistēma automātiski piesaka lietotāju un novirza uz sākumlapu. Pieteikšanās notiek, ievadot e-pastu un paroli - pēc veiksmīgas autentifikācijas tiek izsniegts JWT tokens un lietotājs iegūst paplašinātas piekļuves tiesības.

Reģistrēts lietotājs var izveidot jaunu ierakstu, nospiežot pogu "Jauns ieraksts" - tiek atvērta forma ar laukiem virsrakstam, saturam, kategorijas izvēlei un publicēšanas statusam. Ierakstus var rediģēt un dzēst tikai to autori. Pie katra ieraksta ir pieejama komentāru sadaļa, kur reģistrēti lietotāji var pievienot komentārus.

Administrators piekļūst speciālam administrācijas panelim (saite /admin), kur var pārvaldīt lietotājus (mainīt lomas, bloķēt kontus), moderēt ierakstus un komentārus, kā arī publicēt ziņas ar avota saitēm. Lai pabeigtu darbu ar platformu, lietotājs nospiež pogu "Iziet" navigācijas joslā - sesija tiek izbeigta un JWT tokens dzēsts.

6.3.1. Sākumlapa

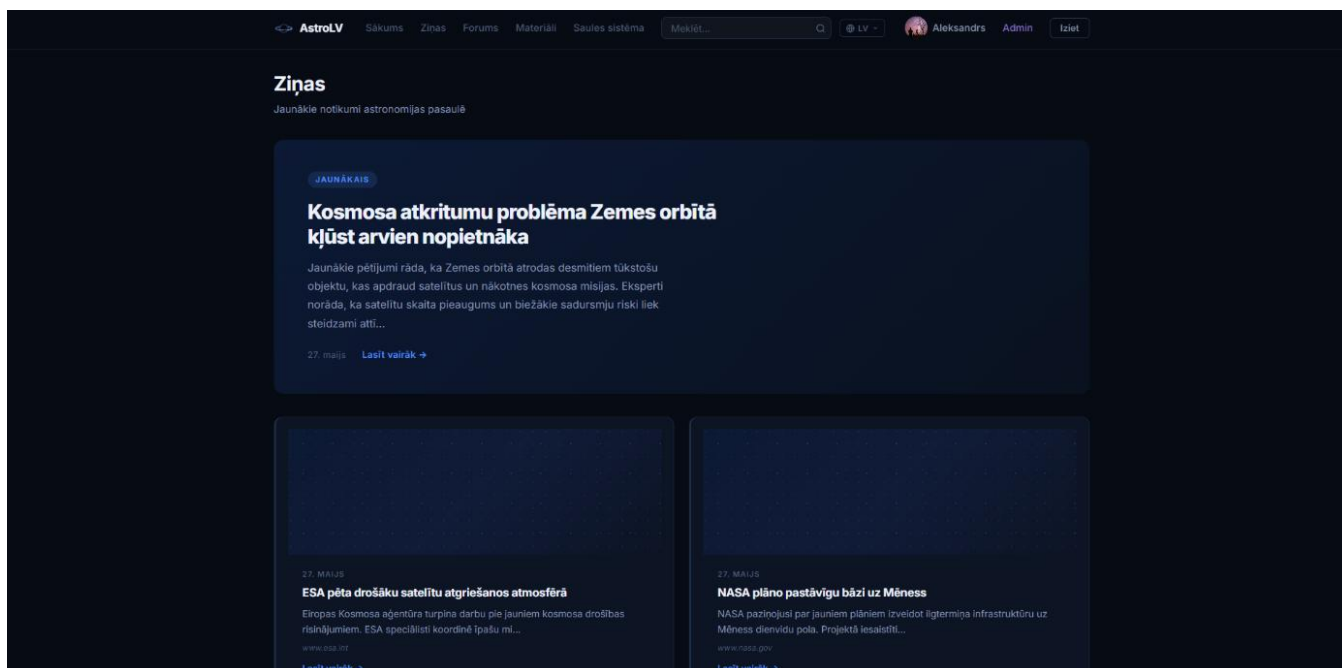
Sākumlapa ir pirmais skats, ko lietotājs redz, atverot platformu (skat. N.att.). Tā sastāv no trīs kolonnām: kreisajā kolonnā (plašākajā) atrodas NASA astronomijas attēls dienā (APOD) ar nosaukumu, aprakstu un saiti uz NASA; vidējā kolonnā - jaunāko ziņu saraksts ar kartēm, kas satur ziņas virsrakstu, publicēšanas datumu un īsu fragmentu; labajā sānjoslā - bloki "Populārākie raksti" un "Foruma tēmas". APOD dati tiek ielādēti automātiski no NASA API caur backend proxy un kešoti 24 stundas. Navigācijas joslā ir logotips "AstroLV", saites uz Sākums/Ziņas/Forums/Saules sistēma un pogas Pieteikties/Reģistrēties. Ja lietotājs ir pieteicies - redzams viņa vārds un poga "Iziet".



13.att. Sākumlapa

6.3.2. Ziņu saraksts

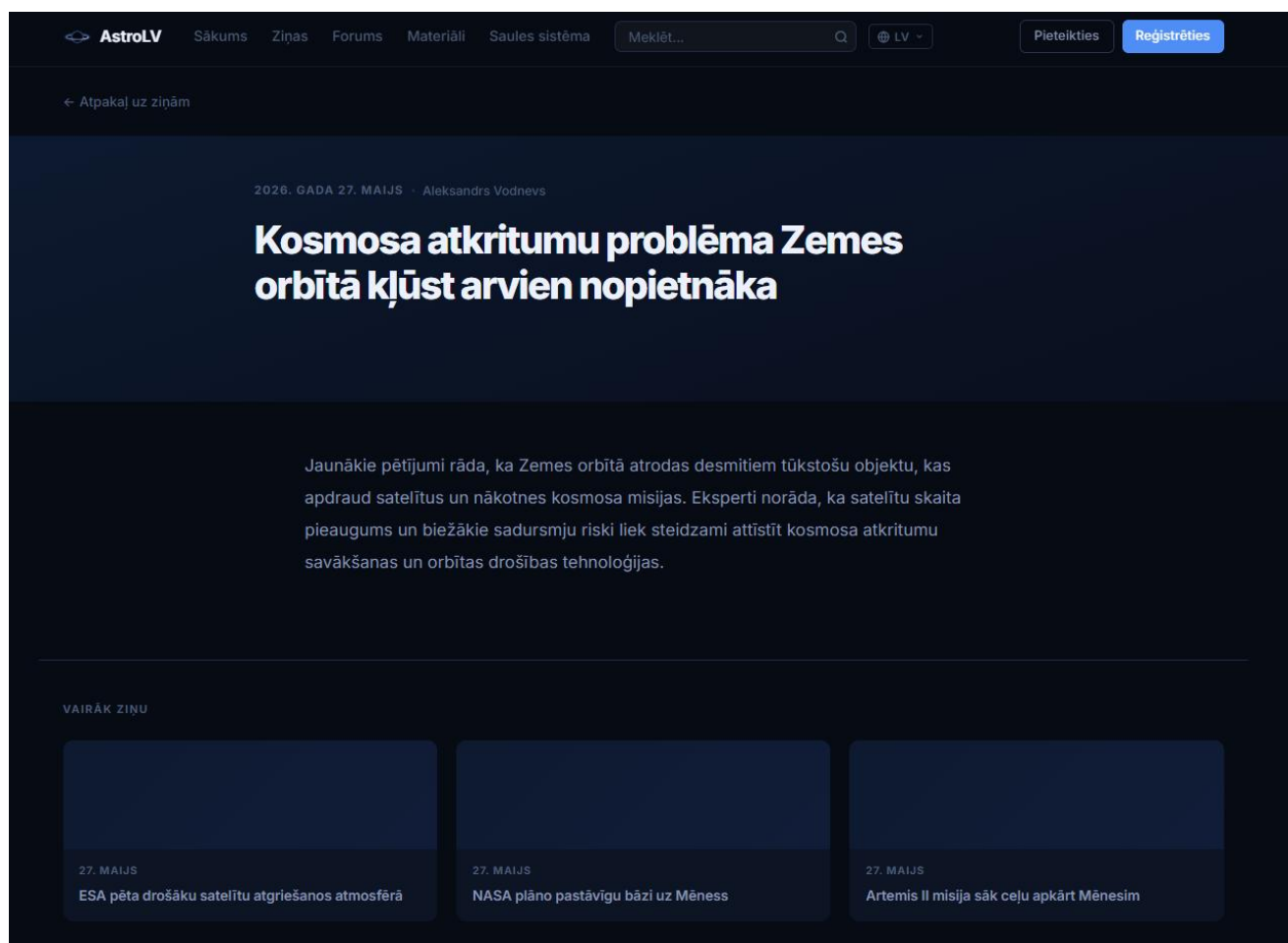
Ziņu sadaļa (skat. N.att.) ir pieejama visiem lietotājiem, noklikšķinot uz "Ziņas" navigācijā. Lapa attēlo visas publicētās ziņas režģa izkārtojumā - katra ziņa ir karte ar ikonu, publicēšanas datumu, virsrakstu, satura fragmentu un saitē "Lasīt vairāk →". Kartēm ir viļņošanās efekts, uzvelkot peli. Ziņu detalizētajā skatā (pēc noklikšķināšanas) tiek rādīts pilnais saturs, publicēšanas datums, autors un, ja norādīta, saite uz oriģinālo avotu.



14.att. Ziņu saraksts

6.3.2 Ziņas (paplašināts)

Ziņu sadaļa veic GET pieprasījumu uz /api/news endpointu un attēlo ziņas karšu izkārtojumā ar hover efektu. Katra karte satur virsrakstu, īsu ievadu, publicēšanas datumu un autora vārdu. Uzspiežot uz ziņas kartes, lietotājs tiek novirzīts uz ziņas detaļu lapu /zinas/:id, kur tiek ielādēts pilns ziņas saturs ar GET pieprasījumu pēc konkrētā ieraksta ID. Detaļu lapā tiek rādīts ziņas virsraksts, pilns teksts, publicēšanas datums un galvenais attēls, kas tiek ielādēts no Cloudinary CDN. Ziņas saturs tiek renderēts no Markdown formāta uz HTML, nodrošinot virsrakstu, sarakstu un teksta formatējuma atbalstu. Ja norādītais ID neeksistē datubāzē, lietotājs tiek novirzīts uz 404 kļūdas lapu. Ziņas publicēšana ir pieejama tikai administratoram caur admin paneli, savukārt parastiem lietotājiem ziņu detaļu lapa ir pieejama tikai lasīšanas režīmā.

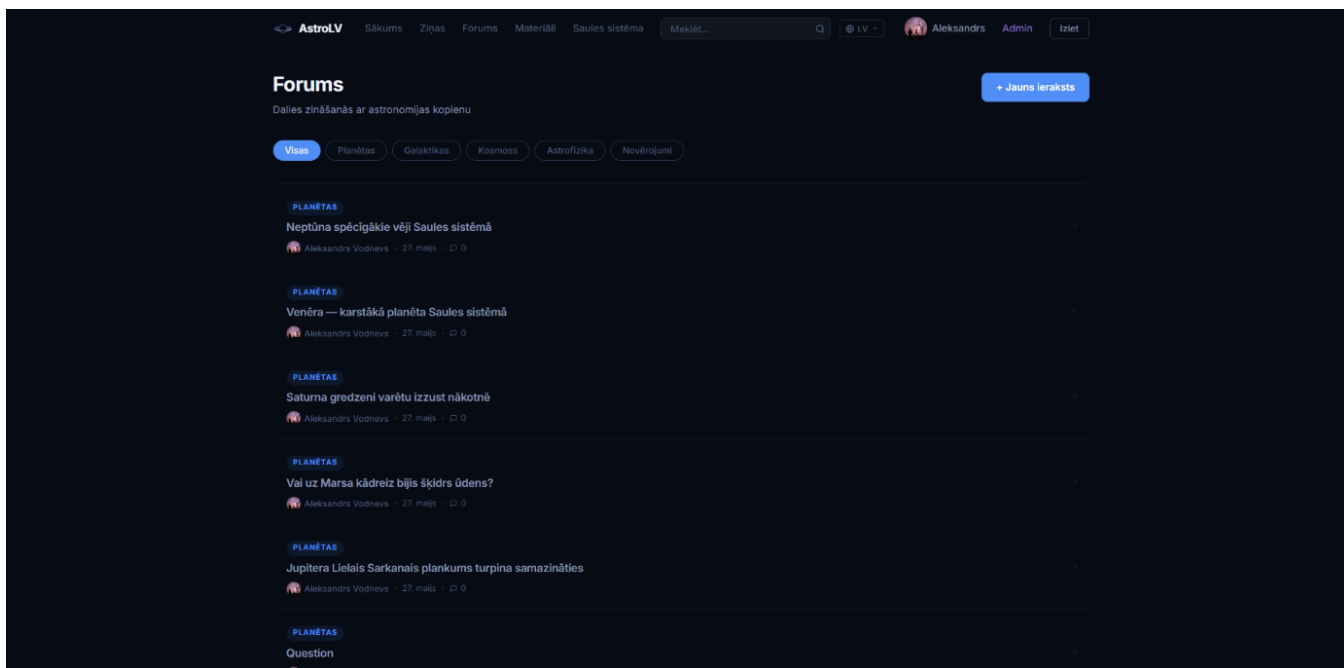


15.att. Ziņas detaļu lapa

6.3.3. Forums

Forums (skat. N.att.) ir pieejams visiem lietotājiem. Lapas augšpusē ir kategoriju filtri (Visas, Planētas, Galaktikas, Kosmoss, Astrofizika, Novērojumi), kas ļauj filtrēt ierakstus pēc tematikas. Ieraksti tiek rādīti sarakstā - katra karte satur kategorijas nozīmīti, virsrakstu,

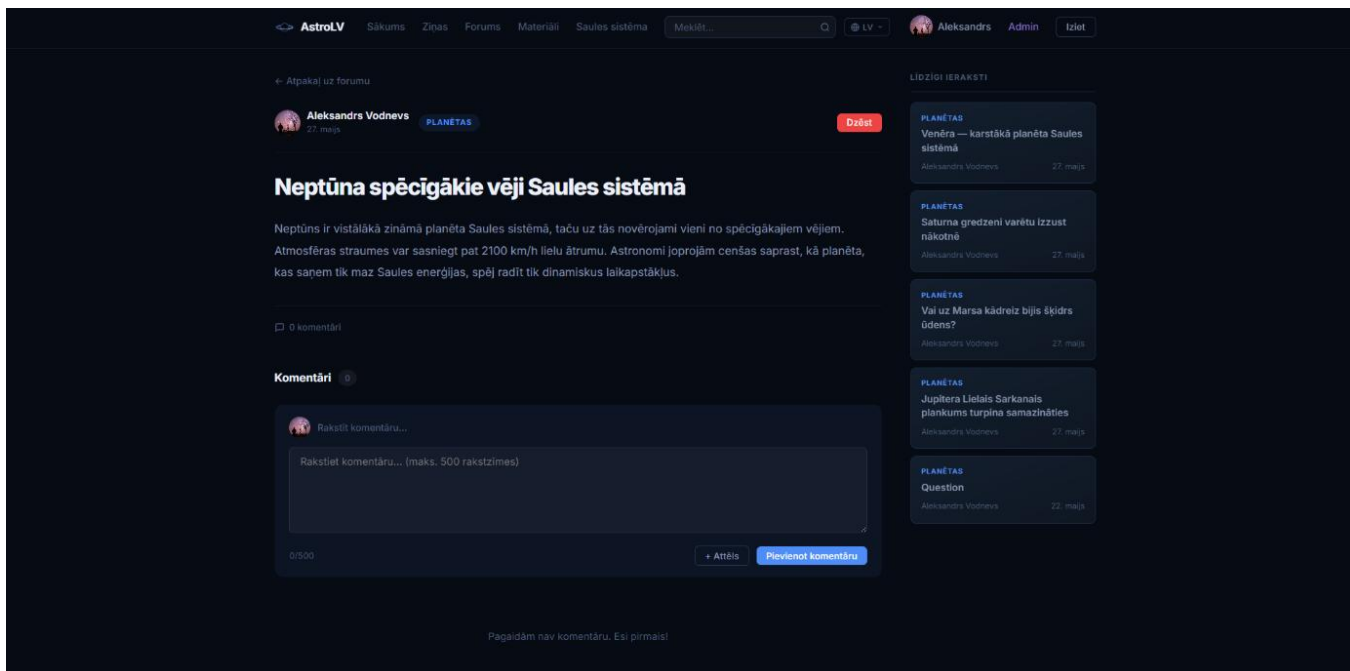
satura fragmentu, autoru un datumu. Reģistrēts lietotājs redz pogu "+ Jauns ieraksts", kas atver ieraksta izveides formu. Meklēšanas funkcija (navigācijas meklēšanas lauks) novirza uz forumu ar meklēšanas rezultātiem.



16.att. Forums

6.3.4. Ieraksta detaļu lapa

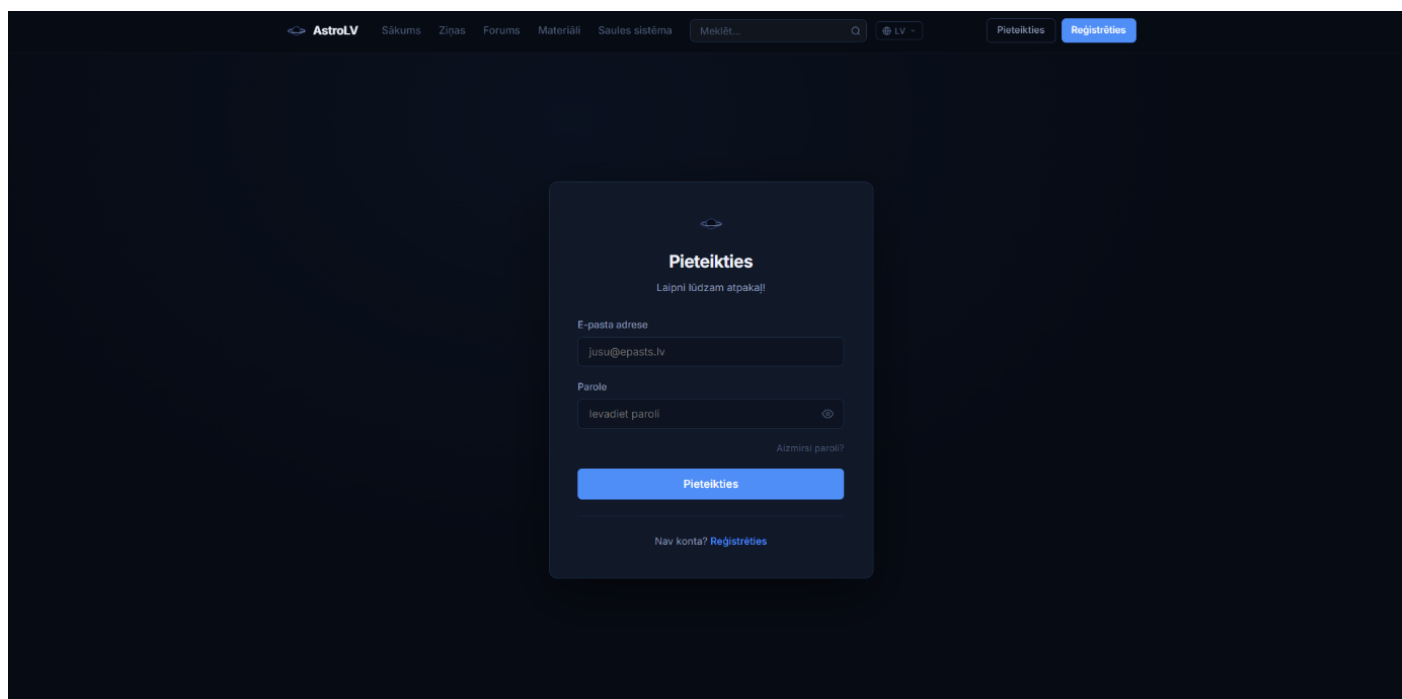
Ieraksta detaļu lapā (skat. N.att.) tiek rādīts pilnais ieraksta saturs ar kategorijas nozīmīti, autoru, datumu un tekstu. Zem ieraksta atrodas komentāru sekcija ar komentāru skaitu. Reģistrēts lietotājs redz komentāra ievades lauku ar rakstzīmju skaitītāju (N/500) un pogu "Pievienot komentāru". Neautentificētam lietotājam tiek rādīts aicinājums pieteikties. Autors un administrators var dzēst gan ierakstu, gan komentārus.



17.att. Ieraksta detaļu lapa

6.3.5. Reģistrācijas un pieteikšanās forma

Reģistrācijas lapa (skat. N.att.) satur formu ar laukiem: Vārds, Uzvārds, E-pasta adrese, Parole un Paroles apstiprinājums. Pēc formas aizpildīšanas lietotājam uz e-pastu tiek nosūtīts 6 ciparu verifikācijas kods caur Resend API. Tikai pēc koda ievadīšanas konts tiek izveidots. Visi lauki ir obligāti. Poga "Reģistrēties" aktivizē validāciju un datu nosūtīšanu. Kļūdu gadījumā tiek rādīts sarkans kļūdas paziņojums. Veiksmīgas reģistrācijas gadījumā lietotājs tiek automātiski pieteikts un novirzīts uz sākumlapu. Pieteikšanās lapā (skat. N.att.) ir e-pasta un paroles lauki. Kļūdu ziņojumi parādās tieši virs pogas.



The screenshot shows the login form on the AstroLV website. The form is titled "Pieteikties" (Login) and includes a subtitle "Laipni lūdzam atpakaļ!" (Welcome back!). It features two input fields: "E-pasta adrese" (Email address) with the placeholder "jusu@epasts.lv" and "Parole" (Password) with the placeholder "ievadiet paroli". A toggle icon for password visibility is located to the right of the password field. Below the fields is a blue "Pieteikties" (Login) button. At the bottom of the form, there is a link "Nav konta? Reģistrēties" (No account? Register).

18.att. Pieteikšanās forma

19.att. Reģistrācijas forma

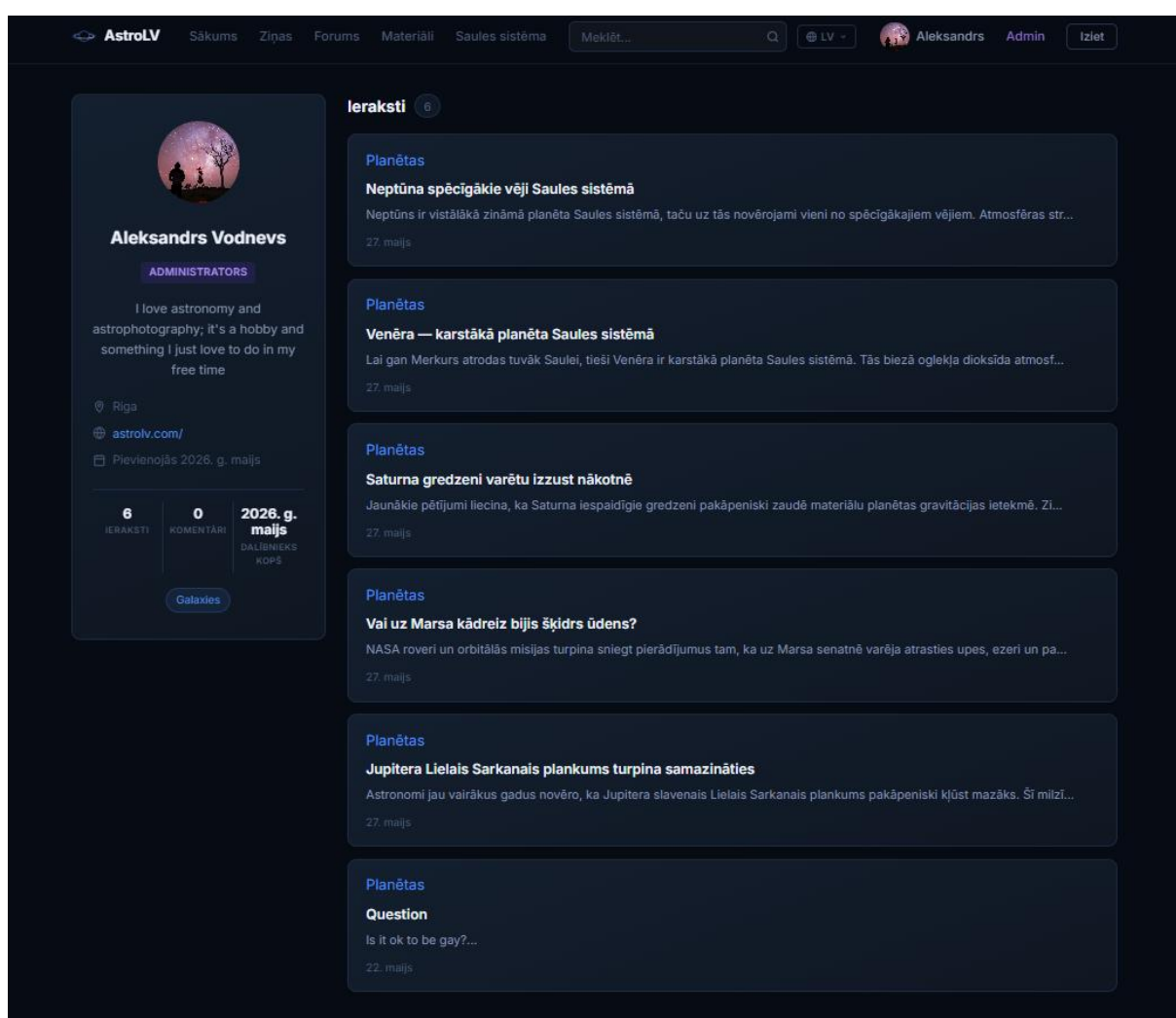
6.3.6. Profila lapa

Profila lapa (skat. N.att.) ir pieejama tikai autentificētiem lietotājiem. Lapas augšpusē ir profila karte ar lietotāja iniciāļiem (ģenerēts avatars), vārdu, e-pastu un lomas nozīmīti. Zemāk atrodas rediģēšanas forma ar laukiem Vārds un Uzvārds. E-pasta lauks ir deaktivizēts (nevar mainīt). Pēc izmaiņu saglabāšanas parādās zaļš apstiprinājuma paziņojums.

20.att. Profila lapa

6.3.11 Publiskais lietotāja profils

Uzspiežot uz lietotāja vārda pie foruma ieraksta vai komentāra, lietotājs tiek novirzīts uz publisko profila lapu /lietotajs/:id, kur tiek ielādēta konkrētā lietotāja publiskā informācija ar GET pieprasījumu pēc lietotāja ID. Lapā tiek rādīts lietotāja displeja vārds, profilbilde, biogrāfija, atrašanās vieta, vietnes saite un intereses, ja tie ir aizpildīti profilā. Zemāk tiek attēloti visi lietotāja publicētie foruma ieraksti hronoloģiskā secībā ar nosaukumu, kategoriju un publicēšanas datumu. Uzspiežot uz ieraksta, lietotājs tiek novirzīts uz attiecīgo ieraksta detaļu lapu. Ja lietotājs ar norādīto ID neeksistē, tiek attēlota 404 kļūdas lapa. Sensitīva informācija, piemēram, e-pasta adrese un parole, publiskajā profilā netiek rādīta - tā ir pieejama tikai pašam lietotājam savā profila rediģēšanas lapā.

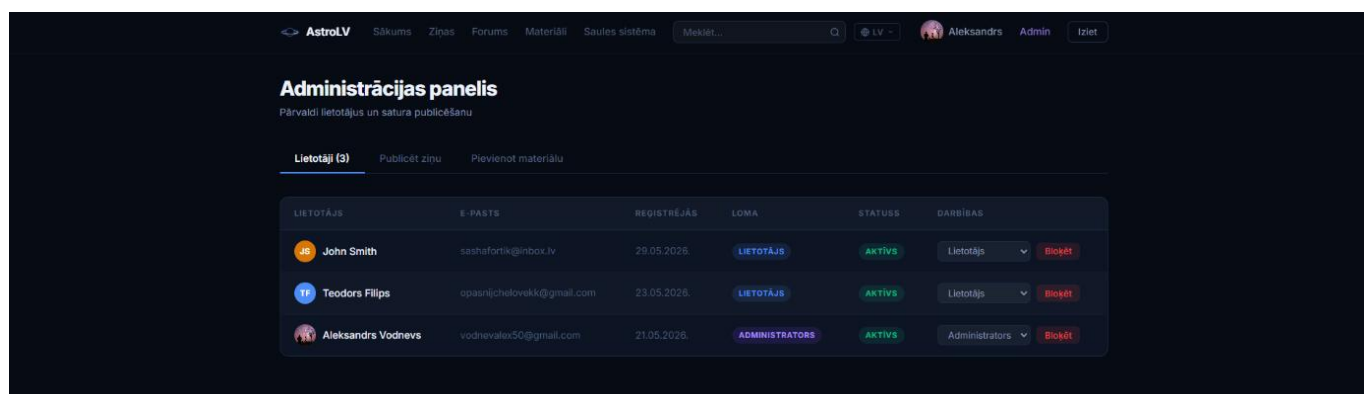


21.att. Publiskais lietotāja profils

6.3.7. Administrācijas panelis

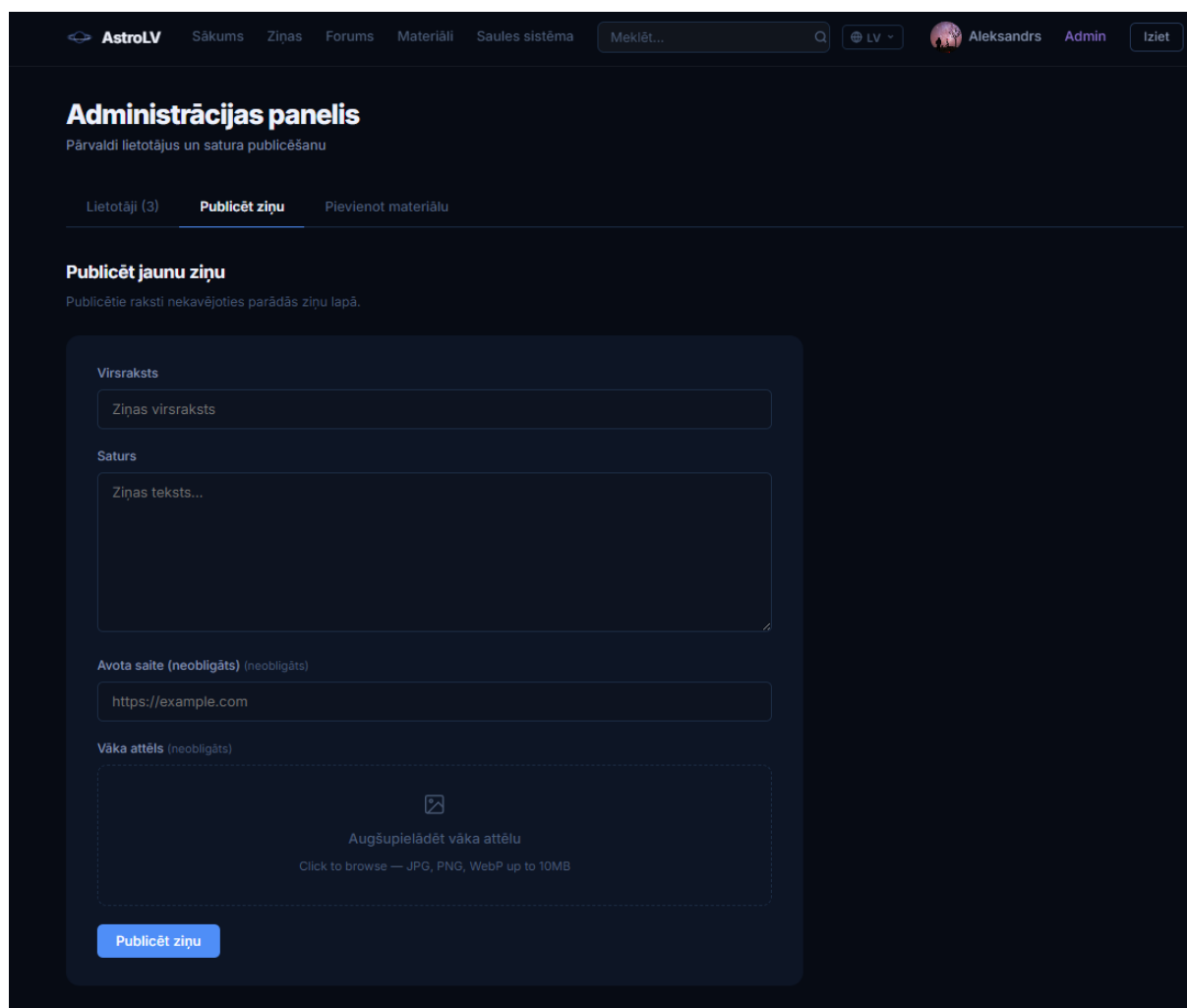
Administrācijas panelis (skat. N.att.) ir pieejams tikai lietotājiem ar lomu "admin" (/admin). Tas sastāv no trim cilnēm: "Lietotāji", "Publicēt ziņu" un "Materiāli". Lietotāju cilnē tiek rādīta tabula ar visiem reģistrētiem lietotājiem - vārds, e-pasts, reģistrācijas datums,

lomas izvēlne un statusa poga. Administrators var mainīt lietotāja lomu no nolaižamās izvēlnes un bloķēt/atbloķēt kontu ar pogu. Ziņu cilnē atrodas forma ar laukiem Virsraksts,



22.att. Administrācijas panelis

Saturs un Avota saite, kas ļauj publicēt jaunas ziņas ar attēlu augšupielādi. Materiālu cilnē administrators var pievienot jaunus mācību materiālus, norādot virsrakstu, saturu Markdown formātā, kategoriju (Astronomija vai Astrofotogrāfija) un grūtības līmeni.



22.att. Administrācijas panelis 2

Administrācijas panelis
Pārvaldi lietotājus un satura publicēšanu

Lietotāji (3) Publicēt ziņu **Pievienot materiālu**

Pievienot mācību materiālu
Izglītojošie materiāli atbalsta Markdown formatēšanu.

Virsraksts ****prekšraksts**** ***slīpraksts*** - sakrsts > citāts `kods`

Virsraksts
Materiāla virsraksts

Kategorija **Grūtības līmenis**
Astronomija **iesācējs**

Saturs
Ievads
Rakstiet savu rokasgrāmatu šeit...

Vāka attēls (neobligāts)
Augšupielādēt vāka attēlu
Click to browse — JPG, PNG, WebP up to 10MB

Pievienot materiālu

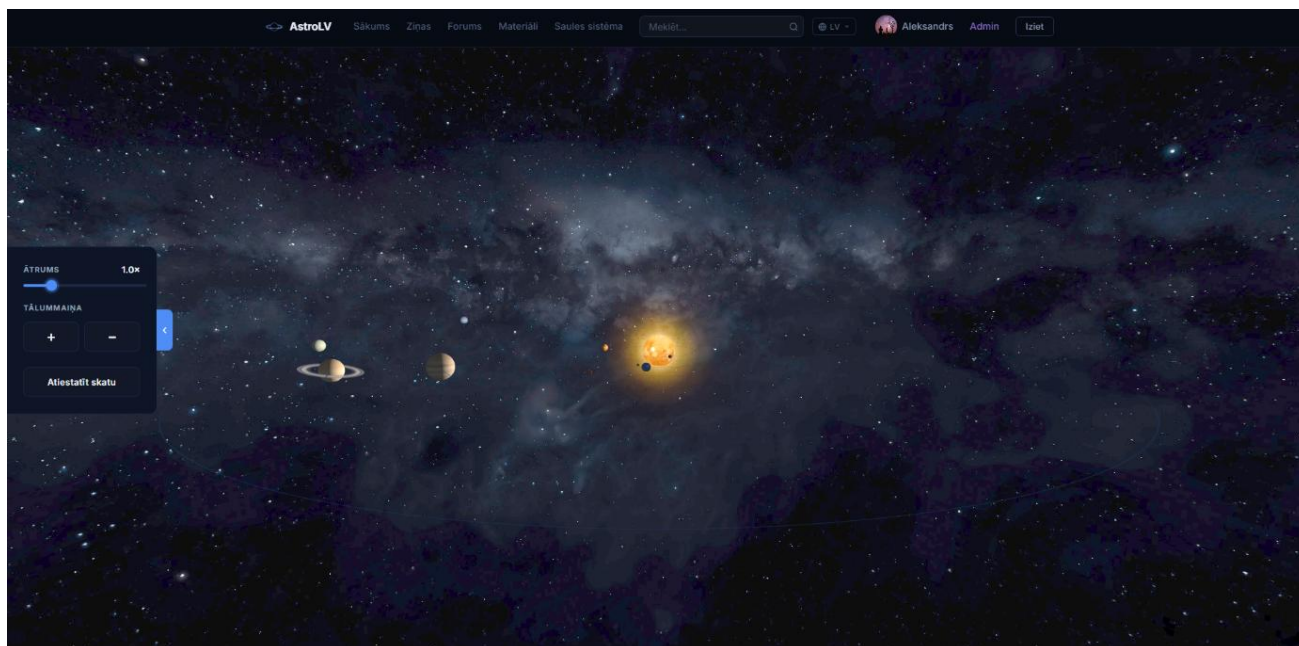
22.att. Administrācijas panelis (3)

6.3.8. Saules sistēmas vizualizācija

Saules sistēmas lapa (/saules-sistema) piedāvā interaktīvu 3D vizualizāciju, kas izstrādāta ar Three.js bibliotēku (skat. N.att.). Lapā redzamas visas 8 planētas ar Mēnesi, kas riņķo ap Sauli reālistiskos attālumos ar individuāliem ātrumiem. Katrai planētai un Mēnesim ir uzklātas NASA augstas izšķirtspējas tekstūras (8K). Saturnam ir renderēti gredzeni ar caurspīdīguma tekstūru. Fons ir Piena ceļa 8K panorāmas attēls kā pilnekrāna sfēriskais debess kupols.

Lietotājs var mijiedarboties ar vizualizāciju vairākos veidos: (1) vilkt ar peli, lai rotētu kameru ap Sauli; (2) ritināt, lai tuvinātu vai tālinātu skatu; (3) klikšķināt uz jebkuras planētas vai Mēness - kamera animēti pietuvojas un labajā pusē parādās informācijas panelis ar planētas nosaukumu, diametru, attālumu no Saules, mēnešu skaitu, orbītas periodu, virsmas temperatūru un interesantu faktu. ESC taustiņš vai atpakaļ poga atgriež kameru sākotnējā pozīcijā. Kreisajā pusē ir saliekams vadības panelis ar ātruma slīdni (0-5x), tālummaiņas

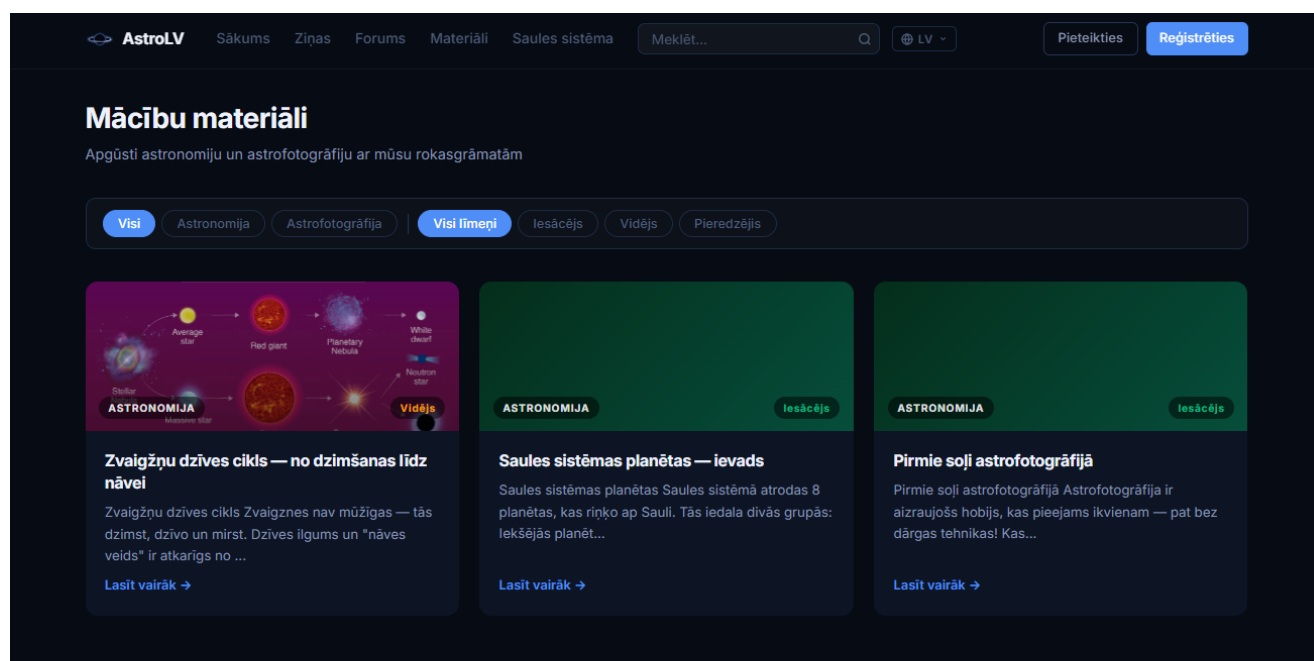
pogām un skata atiestatīšanas pogu.



23.att. Saules sistēmas vizualizācija

6.3.9 Materiāli

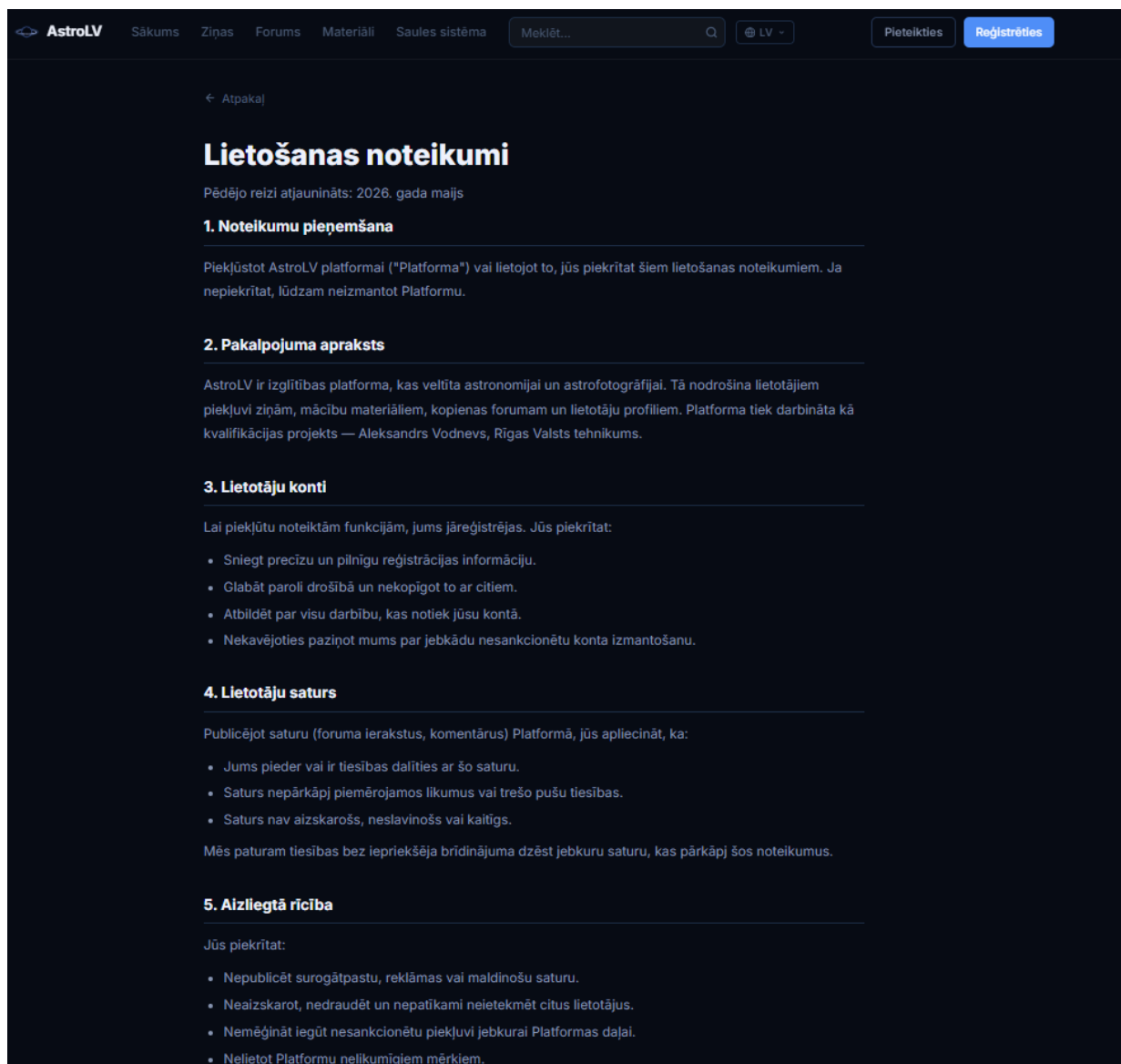
Materiālu sadaļa nodrošina lietotājiem piekļuvi izglītojošiem astronomijas materiāliem - rakstiem, ceļvežiem un resursiem. Materiāli tiek ielādēti ar GET pieprasījumu uz /api/materials endpointu un attēloti karšu izkārtojumā ar nosaukumu, aprakstu un kategoriju. Katram materiālam var būt pievienots ārējās saites URL vai lejupielādējams fails. Materiālu pārvaldība notiek caur admin paneli cilnē "Materiāli", kur administrators var pievienot, rediģēt un dzēst ierakstus. Parastiem lietotājiem sadaļa ir pieejama tikai lasīšanas režīmā.



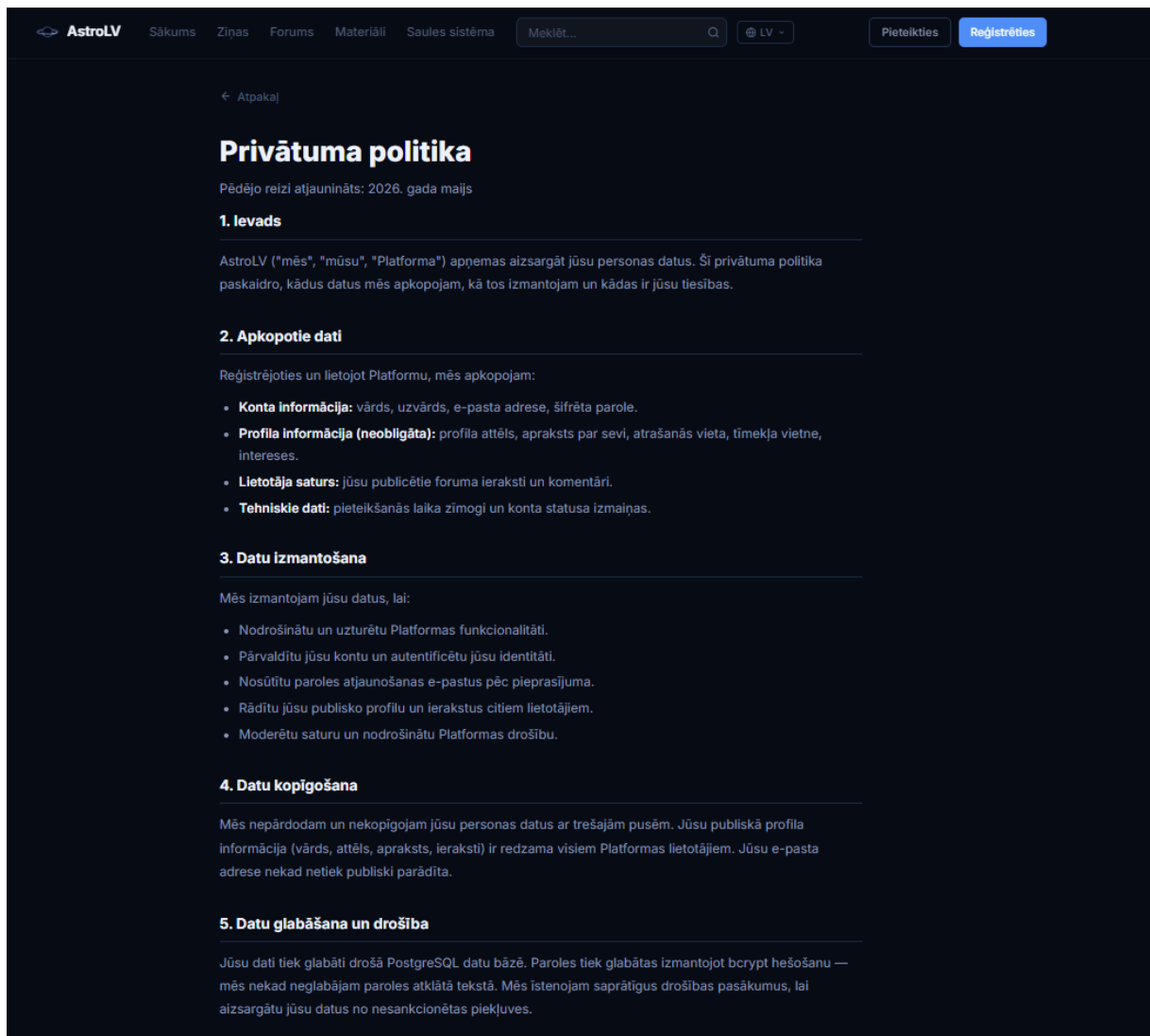
24.att. Materiālu saraksts

6.3.9 Materiāli (paplašināts)

Materiālu sadaļa veic GET pieprasījumu uz /api/materials endpointu un attēlo materiālus karšu izkārtojumā ar nosaukumu, aprakstu un kategoriju. Uzspiežot uz materiāla kartes, lietotājs tiek novirzīts uz materiāla detaļu lapu /materiali/:id, kur tiek ielādēts pilns materiāla saturs ar GET pieprasījumu pēc konkrētā ieraksta ID. Detaļu lapā tiek rādīts materiāla virsraksts, pilns apraksts, kategorija un, ja pieejams, ārējās saites URL vai lejupielādējamā faila poga. Materiāla saturs tiek renderēts no Markdown formāta uz HTML, nodrošinot virsrakstu, sarakstu un teksta formatējuma atbalstu. Ja norādītais ID neeksistē datubāzē, lietotājs tiek novirzīts uz 404 kļūdas lapu. Materiālu pievienošana, rediģēšana un dzēšana ir pieejama tikai administratoram caur admin paneli cilnē "Materiāli", savukārt parastiem lietotājiem materiālu detaļu lapa ir pieejama tikai lasīšanas režīmā.



26.att. Lietošanas noteikumi



27.att. Privātumu politika

7. PROGRAMMATŪRAS LIETOJAMĪBAS TESTĒŠANA

Testēšanas metodoloģija

Sistēmas testēšana tika veikta manuāli, izmantojot melnās kastes testēšanas metodi (black-box testing), kur testētājs pārbauda sistēmas uzvedību no gala lietotāja perspektīvas, neiedziļinoties iekšējā implementācijā. Katram funkcionālajam prasību punktam tika sastādīts testēšanas gadījums ar definētiem ievaddatiem, gaidāmo rezultātu un faktisko rezultātu.

Testēšana tika veikta trīs pārlūkprogrammās - Google Chrome, Mozilla Firefox un Microsoft Edge - lai pārlicinātos par krustpārlūka saderību. Papildus tika pārbaudīta responsivitāte mobilajās ierīcēs, izmantojot Chrome DevTools mobilās ierīces emulatoru. Visi identificētie defekti tika izlaboti pirms sistēmas publiskas izvietošanas.

Testēšanas vide

Testēšana tika veikta produkcijas vidē vietnē astro.lv.com, lai nodrošinātu, ka rezultāti atbilst reālajai sistēmas darbībai. Backend API testēšanai papildus tika izmantots Postman rīks, kas ļāva pārbaudīt endpointus neatkarīgi no frontend saskarnes un simulēt dažādus pieprasījumu scenārijus, tostarp nepareizus vai ļaunprātīgus ievaddatus.

7.1.tabula

Lietotāja loma:	Viesis	
Prasības identifikators:	FP-01	
Prasības apraksts:	Sistēma jānodrošina jauna lietotāja reģistrācija ar e-pastu un paroli. Formai jāvalidē ievadītie dati un jānosūta e-pasta verifikācijas kods.	
Ievaddati / Situācijas apraksts	Sagaidāmais rezultāts	Statuss
Korekts vārds, uzvārds, e-pasts (user@test.com), parole (8+ simboli)	Verifikācijas kods nosūtīts; pēc apstiprināšanas konts izveidots	Pareizi
Nederīgs e-pasta formāts (bez @)	Kļūdas paziņojums: Nepareizs e-pasta formāts	Pareizi
Parole - tikai 5 simboli	Kļūdas paziņojums: Parole ir pārāk īsa (min. 8 simboli)	Pareizi
E-pasts jau reģistrēts sistēmā	Kļūdas paziņojums: Lietotājs ar šādu e-pastu jau eksistē	Pareizi

7.2.tabula

Lietotāja loma:	Reģistrēts lietotājs	
Prasības identifikators:	FP-02	
Prasības apraksts:	Sistēma jānodrošina esošā lietotāja autentifikācija ar e-pastu un paroli. Veiksmīgas autentifikācijas gadījumā jāizsniedz JWT tokens.	
Ievaddati / Situācijas apraksts	Sagaidāmais rezultāts	Statuss
Korekts e-pasts un parole	Veiksmīga pieteikšanās; lietotājs novirzīts uz sākumlapu	Pareizi

Korekts e-pasts un parole	Veiksmīga pieteikšanās; JWT tokens izsniegts	Pareizi
Nepareiza parole (korekts e-pasts)	Kļūdas paziņojums: Nepareizs e-pasts vai parole	Pareizi
Bloķēts konts - korekti dati	Kļūdas paziņojums: Jūsu konts ir bloķēts	Pareizi

7.3.tabula

Lietotāja loma:	Reģistrēts lietotājs	
Prasības identifikators:	FP-03	
Prasības apraksts:	Reģistrēts lietotājs var izveidot jaunu ierakstu forumā. Sistēmai jāvalidē obligātie lauki un jānodrošina piekļuves kontrole.	
Ievaddati / Situācijas apraksts	Sagaidāmais rezultāts	Statuss
Aizpildīti visi lauki, statuss Publicēts	Ieraksts saglabāts un redzams forumā	Pareizi
Tukšs virsraksta lauks	Validācijas kļūda: Virsraksts ir obligāts lauks	Pareizi
Viesis mēģina piekļūt ieraksta izveides lapai	Novirzīšana uz pieteikšanās lapu	Pareizi

7.4.tabula

Lietotāja loma:	Reģistrēts lietotājs	
Prasības identifikators:	FP-04	
Prasības apraksts:	Reģistrēts lietotājs var pievienot komentāru pie publicēta ieraksta. Komentāra garumam jābūt no 1 līdz 500 rakstzīmēm.	
Ievaddati / Situācijas apraksts	Sagaidāmais rezultāts	Statuss
Korekts teksts (50 simboli)	Komentārs pievienots un redzams ierakstā	Pareizi
Tukšs komentāra lauks	Validācijas kļūda: Komentārs nevar būt tukšs	Pareizi
Teksts, kas pārsniedz 500 rakstzīmes	Kļūdas paziņojums: Pārsniegts simbolu limits	Pareizi
Neautenticēts lietotājs mēģina komentēt	HTTP 401; aicinājums pieteikties	Pareizi

7.5.tabula

Lietotāja loma:	Administrators	
Prasības identifikators:	FP-05	
Prasības apraksts:	Administrators var pārvaldīt sistēmas lietotājus - mainīt lomas un bloķēt kontus. Parastiem lietotājiem nav piekļuves administrācijas panelim.	
Ievaddati / Situācijas apraksts	Sagaidāmais rezultāts	Statuss
Administrators bloķē lietotāju	Konts bloķēts; lietotājs nevar pieteikties	Pareizi
Administrators maina lietotāja lomu uz Administrators	Loma sekmīgi mainīta	Pareizi
Parasts lietotājs mēģina piekļūt /admin maršrutam	HTTP 403 Forbidden; piekļuve liegta	Pareizi

NOBEIGUMS

Kvalifikācijas darba ietvaros tika veiksmīgi izstrādāta un izvietota astronomijas izglītības platforma AstroLV, kas pieejama publiski vietnē astrolv.com.

Kvalifikācijas darba rezultātā ir izstrādāta pilnvērtīga tīmekļa lietojumprogramma - Astronomijas izglītības platforma. Sistēma ir realizēta, izmantojot Node.js v22 un Express.js v5.1 serverpusē, React v19 klientpusē un PostgreSQL v18.3 datu glabāšanai. Platforma nodrošina lietotāju reģistrāciju ar e-pasta verifikāciju, autentifikāciju ar JWT, ziņu un foruma ierakstu publicēšanu, mācību materiālus ar Markdown atbalstu, interaktīvu 3D Saules sistēmas vizualizāciju ar Three.js un NASA ikdienas astronomijas attēlu sākumlapā. Visi izvirzītie uzdevumi ir izpildīti atbilstoši kvalifikācijas darba prasībām.

Platforma ir izvietota publiskā hostingā (astrolv.com) un ir pieejama visiem lietotājiem. Kods atrodas GitHub repozitorijā un ir sagatavots turpmākai attīstībai. Nākotnē sistēmu varētu papildināt ar astronomijas viktorīnu ar rezultātu tabulu, lietotāju paziņojumu sistēmu, papildu valodu atbalstu un paplašinātu 3D vizualizāciju ar komētām un asteroīdiem. Platforma varētu tikt izmantota kā mācību rīks astronomijas izglītībā skolās un universitātēs.

INFORMĀCIJAS AVOTI

1. Bērziņš U. Tīmekļa tehnoloģijas un to pielietojums. - Rīga: RTU Izdevniecība, 2020. - 215 lpp.
2. Zaiceva L. Programmatūras izstrādes tehnoloģija. - Rīga: RTU, 2002. - 204 lpp.
3. Node.js dokumentācija. - <https://nodejs.org/en/docs/> - (Resurss apskatīts 15.03.2026.).
4. React dokumentācija. - <https://react.dev/> - (Resurss apskatīts 15.03.2026.).
5. PostgreSQL dokumentācija. - <https://www.postgresql.org/docs/> - (Resurss apskatīts 20.03.2026.).
6. Express.js. Fast, unopinionated, minimalist web framework for Node.js. - <https://expressjs.com/> - (Resurss apskatīts 10.02.2026.).
7. Sequelize ORM dokumentācija. - <https://sequelize.org/docs/v6/> - (Resurss apskatīts 12.02.2026.).
8. JSON Web Tokens. Introduction to JSON Web Tokens. - <https://jwt.io/introduction> - (Resurss apskatīts 05.02.2026.).
9. Fowler M. Patterns of Enterprise Application Architecture. - Boston: Addison-Wesley, 2002. - 533 p.
10. Flanagan D. JavaScript: The Definitive Guide. 7th edition. - Sebastopol: O'Reilly Media, 2020. - 706 p.
11. Elmasri R., Navathe S. Fundamentals of Database Systems. 7th edition. - Hoboken: Pearson, 2016. - 1272 p.
12. Node.js dokumentācija. - <https://nodejs.org/en/docs/> - (Resurss apskatīts 15.03.2026.).
13. React dokumentācija. - <https://react.dev/> - (Resurss apskatīts 15.03.2026.).
14. PostgreSQL dokumentācija. - <https://www.postgresql.org/docs/> - (Resurss apskatīts 20.03.2026.).
15. Express.js. Fast, unopinionated, minimalist web framework for Node.js. - <https://expressjs.com/> - (Resurss apskatīts 10.02.2026.).
16. Sequelize ORM dokumentācija. - <https://sequelize.org/docs/v6/> - (Resurss apskatīts 12.02.2026.).
17. JSON Web Tokens. Introduction to JSON Web Tokens. - <https://jwt.io/introduction> - (Resurss apskatīts 05.02.2026.).
18. Three.js dokumentācija. - <https://threejs.org/docs/> - (Resurss apskatīts 01.04.2026.).

19. Cloudinary dokumentācija. - <https://cloudinary.com/documentation> - (Resurss apskatīts 10.03.2026.).
20. Resend API dokumentācija. - <https://resend.com/docs> - (Resurss apskatīts 10.03.2026.).
21. NASA Astronomy Picture of the Day API. - <https://api.nasa.gov> - (Resurss apskatīts 20.02.2026.).
22. Eiropas Parlaments un Padome. Regula (ES) 2016/679 (Vispārīgā datu aizsardzības regula). - <https://eur-lex.europa.eu/legal-content/LV/TXT/?uri=CELEX:32016R0679> - (Resurss apskatīts 05.05.2026.).

PIELIKUMI

Programmas kods

backend/package.json

JSON

```
{  
  
  "name": "astronomy-platform-backend",  
  
  "version": "1.0.0",  
  
  "description": "Backend for astronomy platform",  
  
  "main": "src/index.js",  
  
  "scripts": {  
  
    "start": "node src/index.js",  
  
    "dev": "nodemon src/index.js"  
  },  
  
  "dependencies": {  
  
    "bcryptjs": "^2.4.3",  
  
    "cors": "^2.8.5",  
  
    "dotenv": "^16.4.5",  
  
    "express": "^4.19.2",  
  
    "jsonwebtoken": "^9.0.2",  
  
    "multer": "^1.4.5-lts.1",  
  
    "nodemailer": "^6.9.13",  
  
    "pg": "^8.11.5",  
  
    "pg-hstore": "^2.3.4",  
  
    "sequelize": "^6.37.3"
```

```

    },
    "devDependencies": {
      "nodemon": "^3.1.0"
    }
  }
}

```

backend/src/config/database.js

JavaScript

```

const { Sequelize } = require('sequelize');
require('dotenv').config();

```

```

const sequelize = process.env.DATABASE_URL ? new Sequelize(process.env.DATABASE_URL, {
  dialect: 'postgres',
  dialectOptions: {
    ssl: {
      require: true,
      rejectUnauthorized: false
    }
  },
  logging: false,
}) : new Sequelize(
  process.env.DB_NAME,
  process.env.DB_USER,
  process.env.DB_PASSWORD,
  {
    host: process.env.DB_HOST,
    port: process.env.DB_PORT || 5432,
    dialect: 'postgres',

```

```
    logging: false,  
  }  
);
```

```
module.exports = sequelize;
```

backend/src/config/emailService.js

JavaScript

```
const nodemailer = require('nodemailer');
```

```
require('dotenv').config();
```

```
const transporter = nodemailer.createTransport({  
  host: process.env.EMAIL_HOST,  
  port: process.env.EMAIL_PORT || 587,  
  secure: process.env.EMAIL_SECURE === 'true', // true for 465, false for other ports  
  auth: {  
    user: process.env.EMAIL_USER,  
    pass: process.env.EMAIL_PASS,  
  },  
});
```

```
const sendVerificationCode = async (email, code, type = 'register') => {  
  const subject = type === 'register' ? 'AstroLV — Apstiprinājuma kods' : 'AstroLV — E-pasta maiņas  
apstiprinājums';
```

```
  const text = `Sveiki!\n\nJūsu drošības kods ir: ${code}\nTas būs derīgs 15 minūtes.`;
```

```
  await transporter.sendMail({  
    from: '"AstroLV" <${process.env.EMAIL_USER}>',
```

```

    to: email,

    subject,

    text,

  });

};

const sendPasswordResetEmail = async (email, token) => {

  const resetUrl = `${process.env.FRONTEND_URL || 'http://localhost:5173'}/atjaunot-paroli?token=${token}`;

  const text = `Sveiki!\n\nEsat pieprasījis paroles atjaunošanu. Lūdzu, izmantojiet šo saiti:\n${resetUrl}\nTā būs derīga 1 stundu.`;

  await transporter.sendMail({

    from: "AstroLV" <${process.env.EMAIL_USER}>,

    to: email,

    subject: 'AstroLV — Paroles atjaunošana',

    text,

  });

};

module.exports = { sendVerificationCode, sendPasswordResetEmail };

```

backend/src/index.js

JavaScript

```

const express = require('express');

const cors = require('cors');

const path = require('path');

require('dotenv').config();

```

```

const sequelize = require('./config/database');

// Models mapping for association generation

const User = require('./models/User');

const Post = require('./models/Post');

const Comment = require('./models/Comment');

const News = require('./models/News');

const Material = require('./models/Material');

const PasswordReset = require('./models/PasswordReset');

const EmailVerification = require('./models/EmailVerification');


const app = express();


// Middlewares

app.use(cors());

app.use(express.json());

app.use('/uploads', express.static(path.join(__dirname, '../uploads')));


// Data Associations setup

User.hasMany(Post, { foreignKey: 'authorId', as: 'posts', onDelete: 'CASCADE' });

Post.belongsTo(User, { foreignKey: 'authorId', as: 'author' });


Post.hasMany(Comment, { foreignKey: 'postId', as: 'comments', onDelete: 'CASCADE' });

Comment.belongsTo(Post, { foreignKey: 'postId', as: 'post' });


User.hasMany(Comment, { foreignKey: 'authorId', as: 'comments', onDelete: 'CASCADE' });

Comment.belongsTo(User, { foreignKey: 'authorId', as: 'author' });

```

```

User.hasMany(News, { foreignKey: 'adminId', as: 'news', onDelete: 'CASCADE' });

News.belongsTo(User, { foreignKey: 'adminId', as: 'admin' });


User.hasMany(Material, { foreignKey: 'adminId', as: 'materials', onDelete: 'CASCADE' });

Material.belongsTo(User, { foreignKey: 'adminId', as: 'admin' });


// Application Routes binding

app.use('/api/auth', require('./routes/auth'));

app.use('/api/users', require('./routes/users'));

app.use('/api/posts', require('./routes/posts'));

app.use('/api/comments', require('./routes/comments'));

app.use('/api/news', require('./routes/news'));

app.use('/api/materials', require('./routes/materials'));

app.use('/api/apod', require('./routes/apod'));


// Connection mapping to Database

const PORT = process.env.PORT || 5000;

sequelize.sync({ alter: true })

  .then(() => {

    console.log('Database synced successfully.');


app.listen(PORT, () => console.log(`Server running on port ${PORT}`));



}}



.catch(err => console.error('Database connection failed:', err));


```

backend/src/middleware/auth.js

JavaScript

```

const jwt = require('jsonwebtoken');

require('dotenv').config();

const authenticate = (req, res, next) => {

  const authHeader = req.headers.authorization;

  if (!authHeader || !authHeader.startsWith('Bearer ')) {

    return res.status(401).json({ message: 'Nav autorizācijas tokena. Piekļuve liegta.' });

  }

  const token = authHeader.split(' ')[1];

  try {

    const decoded = jwt.verify(token, process.env.JWT_SECRET);

    req.user = decoded; // Contains id and role

    next();

  } catch (err) {

    return res.status(401).json({ message: 'Nederīgs vai ekspirēts tokens.' });

  }

};

const isAdmin = (req, res, next) => {

  if (!req.user || req.user.role !== 'admin') {

    return res.status(403).json({ message: 'Tikai administratoriem ir piekļuve šim resursam.' });

  }

  next();

};

module.exports = { authenticate, isAdmin };

```

backend/src/middleware/upload.js

JavaScript

```
const multer = require('multer');

const storage = multer.memoryStorage();

const fileFilter = (req, file, cb) => {

  const allowed = ['image/jpeg', 'image/png', 'image/gif', 'image/webp'];

  if (allowed.includes(file.mimetype)) {

    cb(null, true);

  } else {

    cb(new Error('Atļauti tikai attēlu faili (jpeg, png, gif, webp)'), false);

  }

};

const upload = multer({

  storage,

  fileFilter,

  limits: { fileSize: 5 * 1024 * 1024 }, // 5MB limit

});
```

```
module.exports = upload;
```

backend/src/models/User.js

JavaScript

```
const { DataTypes } = require('sequelize');

const sequelize = require('../config/database');
```



```

const User = sequelize.define('User', {
  id: { type: DataTypes.INTEGER, autoIncrement: true, primaryKey: true },
  firstName: { type: DataTypes.STRING(30), allowNull: false },
  lastName: { type: DataTypes.STRING(30), allowNull: false },
  email: { type: DataTypes.STRING(100), allowNull: false, unique: true, validate: { isEmail: true } },
  passwordHash: { type: DataTypes.STRING(255), allowNull: false },
  role: { type: DataTypes.ENUM('guest', 'user', 'admin'), defaultValue: 'user' },
  status: { type: DataTypes.ENUM('active', 'blocked'), defaultValue: 'active' },
  avatar: { type: DataTypes.STRING(255), allowNull: true },
  bio: { type: DataTypes.STRING(300), allowNull: true },
  location: { type: DataTypes.STRING(100), allowNull: true },
  website: { type: DataTypes.STRING(200), allowNull: true },
  interests: { type: DataTypes.STRING(200), allowNull: true },
}, { tableName: 'users', timestamps: true });

```

```

module.exports = User;

```

backend/src/models/Post.js

JavaScript

```

const { DataTypes } = require('sequelize');

const sequelize = require('../config/database');

const Post = sequelize.define('Post', {
  id: { type: DataTypes.INTEGER, autoIncrement: true, primaryKey: true },
  title: { type: DataTypes.STRING(100), allowNull: false },
  content: { type: DataTypes.TEXT, allowNull: false },
  category: { type: DataTypes.ENUM('Planētas', 'Galaktikas', 'Kosmos', 'Astrofizika', 'Novērojumi'),
    allowNull: false },

```

```

status: { type: DataTypes.ENUM('draft', 'published'), defaultValue: 'published' },
imageUrl: { type: DataTypes.STRING(500), allowNull: true },
authorId: { type: DataTypes.INTEGER, allowNull: false },
}, { tableName: 'posts', timestamps: true });

```

```

module.exports = Post;

```

backend/src/models/Comment.js

JavaScript

```

const { DataTypes } = require('sequelize');

const sequelize = require('../config/database');

const Comment = sequelize.define('Comment', {

  id: { type: DataTypes.INTEGER, autoIncrement: true, primaryKey: true },

  content: { type: DataTypes.STRING(500), allowNull: false },

  imageUrl: { type: DataTypes.STRING(500), allowNull: true },

  postId: { type: DataTypes.INTEGER, allowNull: false },

  authorId: { type: DataTypes.INTEGER, allowNull: false },

}, { tableName: 'comments', timestamps: true });

```

```

module.exports = Comment;

```

backend/src/models/News.js

JavaScript

```

const { DataTypes } = require('sequelize');

const sequelize = require('../config/database');

const News = sequelize.define('News', {

  id: { type: DataTypes.INTEGER, autoIncrement: true, primaryKey: true },

```

```

title: { type: DataTypes.STRING(150), allowNull: false },
content: { type: DataTypes.TEXT, allowNull: false },
imageUrl: { type: DataTypes.STRING(500), allowNull: true },
sourceUrl: { type: DataTypes.STRING(300), allowNull: true },
adminId: { type: DataTypes.INTEGER, allowNull: false },
}, { tableName: 'news', timestamps: true });

```

```
module.exports = News;
```

backend/src/models/Material.js

JavaScript

```

const { DataTypes } = require('sequelize');

const sequelize = require('../config/database');

const Material = sequelize.define('Material', {
  id: { type: DataTypes.INTEGER, autoIncrement: true, primaryKey: true },
  title: { type: DataTypes.STRING(150), allowNull: false },
  content: { type: DataTypes.TEXT, allowNull: false },
  category: { type: DataTypes.ENUM('Pamati', 'Sistēmas', 'Tehnika', 'Astrofotogrāfija'), allowNull: false },
  adminId: { type: DataTypes.INTEGER, allowNull: false },
}, { tableName: 'materials', timestamps: true });

```

```
module.exports = Material;
```

backend/src/models/EmailVerification.js

JavaScript

```

const { DataTypes } = require('sequelize');

const sequelize = require('../config/database');

```

```

const EmailVerification = sequelize.define('EmailVerification', {
  id: { type: DataTypes.INTEGER, autoIncrement: true, primaryKey: true },
  userId: { type: DataTypes.INTEGER, allowNull: true },
  email: { type: DataTypes.STRING(100), allowNull: false },
  code: { type: DataTypes.STRING(6), allowNull: false },
  type: { type: DataTypes.ENUM('register', 'change_email'), allowNull: false },
  pendingData: { type: DataTypes.TEXT, allowNull: true }, // Encoded JSON for contextual
adjustments
  used: { type: DataTypes.BOOLEAN, defaultValue: false },
  expiresAt: { type: DataTypes.DATE, allowNull: false },
}, { tableName: 'email_verifications', timestamps: true });

```

```

module.exports = EmailVerification;

```

backend/src/models/PasswordReset.js

JavaScript

```

const { DataTypes } = require('sequelize');

const sequelize = require('../config/database');

const PasswordReset = sequelize.define('PasswordReset', {
  id: { type: DataTypes.INTEGER, autoIncrement: true, primaryKey: true },
  userId: { type: DataTypes.INTEGER, allowNull: false },
  token: { type: DataTypes.STRING(64), allowNull: false },
  used: { type: DataTypes.BOOLEAN, defaultValue: false },
  expiresAt: { type: DataTypes.DATE, allowNull: false },
}, { tableName: 'password_resets', timestamps: true });

```

```
module.exports = PasswordReset;
```

backend/src/routes/auth.js

JavaScript

```
const express = require('express');

const router = express.Router();

const bcrypt = require('bcryptjs');

const jwt = require('jsonwebtoken');

const crypto = require('crypto');

const User = require('../models/User');

const PasswordReset = require('../models/PasswordReset');

const EmailVerification = require('../models/EmailVerification');

const { sendPasswordResetEmail, sendVerificationCode } = require('../config/emailService');

const generateCode = () => Math.floor(100000 + Math.random() * 900000).toString();

router.post('/register', async (req, res) => {

  try {

    const { firstName, lastName, email, password } = req.body;

    if (!firstName || !lastName || !email || !password) return res.status(400).json({ message: 'Visi lauki ir obligāti' });

    if (password.length < 8) return res.status(400).json({ message: 'Parole ir pārāk īsa (min. 8 simboli)' });

    const existing = await User.findOne({ where: { email } });

    if (existing) return res.status(400).json({ message: 'Lietotājs ar šādu e-pastu jau eksistē' });

    const passwordHash = await bcrypt.hash(password, 10);
```

```

const code = generateCode();

const expiresAt = new Date(Date.now() + 15 * 60 * 1000);

await EmailVerification.destroy({ where: { email, type: 'register', used: false } });

await EmailVerification.create({
  userId: null, email, code, type: 'register',
  pendingData: JSON.stringify({ firstName, lastName, email, passwordHash }), expiresAt
});

await sendVerificationCode(email, code, 'register');

res.json({ message: 'Kods nosūtīts', email });
} catch (err) {
  res.status(500).json({ message: 'Servera kļūda' });
}
});

router.post('/verify-registration', async (req, res) => {
  try {
    const { email, code } = req.body;

    if (!email || !code) return res.status(400).json({ message: 'E-pasts un kods ir obligāti' });

    const verification = await EmailVerification.findOne({ where: { email, code, type: 'register', used:
false } });

    if (!verification || verification.expiresAt < new Date()) {
      return res.status(400).json({ message: 'Nederīgs vai ekspirēts apstiprinājuma kods' });
    }
  }

```

```

verification.used = true;

await verification.save();

const { firstName, lastName, passwordHash } = JSON.parse(verification.pendingData);

const user = await User.create({ firstName, lastName, email, passwordHash });

const token = jwt.sign({ id: user.id, role: user.role }, process.env.JWT_SECRET, { expiresIn: '7d'
});

res.status(201).json({ token, user: { id: user.id, firstName: user.firstName, lastName: user.lastName,
email: user.email, role: user.role } });

} catch (err) {

res.status(500).json({ message: 'Servera kļūda' });

}

});

router.post('/login', async (req, res) => {

try {

const { email, password } = req.body;

if (!email || !password) return res.status(400).json({ message: 'E-pasts un parole ir obligāti' });

const user = await User.findOne({ where: { email } });

if (!user || !(await bcrypt.compare(password, user.passwordHash))) {

return res.status(400).json({ message: 'Nepareizs e-pasts vai parole' });

}

if (user.status === 'blocked') return res.status(403).json({ message: 'Jūsu profils ir bloķēts.' });

const token = jwt.sign({ id: user.id, role: user.role }, process.env.JWT_SECRET, { expiresIn: '7d'
});

```

```

    res.json({ token, user: { id: user.id, firstName: user.firstName, lastName: user.lastName, email:
user.email, role: user.role, avatar: user.avatar } });

    } catch (err) {

    res.status(500).json({ message: 'Servera kļūda' });

    }

  });

```

```

router.post('/forgot-password', async (req, res) => {

  try {

    const { email } = req.body;

    const user = await User.findOne({ where: { email } });

    if (!user) return res.json({ message: 'Ja e-pasts eksistē, instrukcija tika nosūtīta.' });

    const token = crypto.randomBytes(32).toString('hex');

    const expiresAt = new Date(Date.now() + 60 * 60 * 1000); // 1 stunda

    await PasswordReset.create({ userId: user.id, token, expiresAt });

    await sendPasswordResetEmail(user.email, token);

    res.json({ message: 'Ja e-pasts eksistē, instrukcija tika nosūtīta.' });

    } catch (err) {

    res.status(500).json({ message: 'Servera kļūda' });

    }

  });

```

```

router.post('/reset-password', async (req, res) => {

  try {

```



```

const { token, password } = req.body;

if (!token || !password) return res.status(400).json({ message: 'Dati ir obligāti' });

const reset = await PasswordReset.findOne({ where: { token, used: false } });

if (!reset || reset.expiresAt < new Date()) return res.status(400).json({ message: 'Saite ir nederīga vai
ekspirēta' });

reset.used = true;

await reset.save();

const user = await User.findByPk(reset.userId);

user.passwordHash = await bcrypt.hash(password, 10);

await user.save();

res.json({ message: 'Parole veiksmīgi nomainīta' });
} catch (err) {

res.status(500).json({ message: 'Servera kļūda' });

}

});

module.exports = router;

```

backend/src/routes/users.js

JavaScript

```

const express = require('express');

const router = express.Router();

const fs = require('fs');

const path = require('path');

```

```

const bcrypt = require('bcryptjs');

const User = require('../models/User');

const Post = require('../models/Post');

const Comment = require('../models/Comment');

const EmailVerification = require('../models/EmailVerification');

const { authenticate, isAdmin } = require('../middleware/auth');

const upload = require('../middleware/upload');

const { sendVerificationCode } = require('../config/emailService');


const generateCode = () => Math.floor(100000 + Math.random() * 900000).toString();

const PUBLIC_ATTRS = ['id', 'firstName', 'lastName', 'avatar', 'bio', 'location', 'website', 'interests',
'role', 'createdAt'];

const PRIVATE_ATTRS = [...PUBLIC_ATTRS, 'email', 'status'];


router.get('/me', authenticate, async (req, res) => {

  try {

    const user = await User.findByPk(req.user.id, { attributes: PRIVATE_ATTRS });

    if (!user) return res.status(404).json({ message: 'Lietotājs nav atrasts' });

    res.json(user);

  } catch (err) {

    res.status(500).json({ message: 'Servera kļūda' });

  }

});


router.put('/me', authenticate, async (req, res) => {

  try {

    const { firstName, lastName, bio, location, website, interests } = req.body;

```

```

const user = await User.findByPk(req.user.id);

user.firstName = firstName || user.firstName;

user.lastName = lastName || user.lastName;

user.bio = bio !== undefined ? bio : user.bio;

user.location = location !== undefined ? location : user.location;

user.website = website !== undefined ? website : user.website;

user.interests = interests !== undefined ? interests : user.interests;

await user.save();

res.json(user);
} catch (err) {
  res.status(500).json({ message: 'Servera kļūda' });
}
});

router.post('/me/avatar', authenticate, upload.single('avatar'), async (req, res) => {
  try {
    if (!req.file) return res.status(400).json({ message: 'Nav iesniegts attēls' });

    const user = await User.findByPk(req.user.id);

    const dir = path.join(__dirname, '../uploads/avatars');

    if (!fs.existsSync(dir)) fs.mkdirSync(dir, { recursive: true });

    const filename = `avatar_${user.id}_${Date.now()}${path.extname(req.file.originalname) || '.jpg'}`;

    const fullPath = path.join(dir, filename);

```

```

fs.writeFileSync(fullPath, req.file.buffer);

if (user.avatar && user.avatar.startsWith('/uploads/avatars/')) {
  const oldPath = path.join(__dirname, '..', user.avatar);
  if (fs.existsSync(oldPath)) fs.unlinkSync(oldPath);
}

user.avatar = `/uploads/avatars/${filename}`;
await user.save();
res.json({ avatar: user.avatar });
} catch (err) {
  res.status(500).json({ message: 'Attēla saglabāšanas kļūda' });
}
});

router.get('/:id', async (req, res) => {
  try {
    const user = await User.findByPk(req.params.id, { attributes: PUBLIC_ATTRS });
    if (!user) return res.status(404).json({ message: 'Lietotājs nav atrasts' });

    const postCount = await Post.count({ where: { authorId: user.id } });
    res.json({ user, postCount });
  } catch (err) {
    res.status(500).json({ message: 'Servera kļūda' });
  }
});

```

```
router.get('/:id/comments/count', async (req, res) => {
```

```
  try {
```

```
    const count = await Comment.count({ where: { authorId: req.params.id } });
```

```
    res.json({ count });
```

```
  } catch (err) {
```

```
    res.status(500).json({ message: 'Servera kļūda' });
```

```
  }
```

```
});
```

```
// Admin Panel route dependencies
```

```
router.get('/', authenticate, isAdmin, async (req, res) => {
```

```
  try {
```

```
    const users = await User.findAll({ attributes: PRIVATE_ATTRS, order: [['createdAt', 'DESC']] });
```

```
    res.json(users);
```

```
  } catch (err) {
```

```
    res.status(500).json({ message: 'Servera kļūda' });
```

```
  }
```

```
});
```

```
router.put('/:id/role', authenticate, isAdmin, async (req, res) => {
```

```
  try {
```

```
    const { role } = req.body;
```

```
    if (!['user', 'admin'].includes(role)) return res.status(400).json({ message: 'Nederīga loma' });
```

```
    const user = await User.findByPk(req.params.id);
```

```
    user.role = role;
```

```
    await user.save();
```

```
    res.json(user);
```

```

    } catch (err) {

      res.status(500).json({ message: 'Servera kļūda' });

    }

  });

```

```

router.put('/:id/status', authenticate, isAdmin, async (req, res) => {

  try {

    const { status } = req.body;

    if (!['active', 'blocked'].includes(status)) return res.status(400).json({ message: 'Nederīgs statuss' });

    const user = await User.findById(req.params.id);

    user.status = status;

    await user.save();

    res.json(user);

  } catch (err) {

    res.status(500).json({ message: 'Servera kļūda' });

  }

});

```

```

module.exports = router;

```

backend/src/routes/posts.js

JavaScript

```

const express = require('express');

const router = express.Router();

const Post = require('../models/Post');

const User = require('../models/User');

const Comment = require('../models/Comment');

const { authenticate } = require('../middleware/auth');

```

```

router.get('/', async (req, res) => {

  try {

    const posts = await Post.findAll({

      where: { status: 'published' },

      include: [{ model: User, as: 'author', attributes: ['id', 'firstName', 'lastName', 'avatar'] }],

      order: [['createdAt', 'DESC']],

    });

    res.json(posts);

  } catch (err) {

    res.status(500).json({ message: 'Servera kļūda' });

  }

});

router.get('/:id', async (req, res) => {

  try {

    const post = await Post.findByPk(req.params.id, {

      include: [{ model: User, as: 'author', attributes: ['id', 'firstName', 'lastName', 'avatar', 'role'] }],

    });

    if (!post) return res.status(404).json({ message: 'Raksts netika atrasts' });

    res.json(post);

  } catch (err) {

    res.status(500).json({ message: 'Servera kļūda' });

  }

});

router.post('/', authenticate, async (req, res) => {

```

```

try {

  const { title, content, category, imageUrl, status } = req.body;

  if (!title || !content || !category) return res.status(400).json({ message: 'Obligātie lauki nav aizpildīti'
});

  const post = await Post.create({ title, content, category, imageUrl, status: status || 'published',
authorId: req.user.id });

  res.status(201).json(post);

} catch (err) {

  res.status(500).json({ message: 'Servera kļūda' });

}

});

router.delete('/:id', authenticate, async (req, res) => {

  try {

    const post = await Post.findByPk(req.params.id);

    if (!post) return res.status(404).json({ message: 'Raksts nav atrasts' });

    if (post.authorId !== req.user.id && req.user.role !== 'admin') {

      return res.status(403).json({ message: 'Darbība liegta' });

    }

    await post.destroy();

    res.json({ message: 'Raksts izdzēsts' });

  } catch (err) {

    res.status(500).json({ message: 'Servera kļūda' });

  }

});

module.exports = router;

```


backend/src/routes/comments.js

JavaScript

```
const express = require('express');

const router = express.Router();

const Comment = require('../models/Comment');

const User = require('../models/User');

const { authenticate } = require('../middleware/auth');

router.get('/post/:postId', async (req, res) => {

  try {

    const comments = await Comment.findAll({

      where: { postId: req.params.postId },

      include: [{ model: User, as: 'author', attributes: ['id', 'firstName', 'lastName', 'avatar', 'role'] }],

      order: [['createdAt', 'ASC']],

    });

    res.json(comments);

  } catch (err) {

    res.status(500).json({ message: 'Servera kļūda' });

  }

});

router.post('/', authenticate, async (req, res) => {

  try {

    const { content, postId, imageUrl } = req.body;

    if (!content || !postId) return res.status(400).json({ message: 'Saturis un PostId ir obligāti' });

    const comment = await Comment.create({ content, postId, imageUrl, authorId: req.user.id });
```

```

const fullComment = await Comment.findByPk(comment.id, {
  include: [{ model: User, as: 'author', attributes: ['id', 'firstName', 'lastName', 'avatar', 'role'] }],
});

res.status(201).json(fullComment);

} catch (err) {
  res.status(500).json({ message: 'Servera kļūda' });
}
});

router.delete('/:id', authenticate, async (req, res) => {
  try {
    const comment = await Comment.findByPk(req.params.id);

    if (!comment) return res.status(404).json({ message: 'Komentārs nav atrasts' });

    if (comment.authorId !== req.user.id && req.user.role !== 'admin') {
      return res.status(403).json({ message: 'Piekļuve liegta' });
    }

    await comment.destroy();

    res.json({ message: 'Komentārs dzēsts' });
  } catch (err) {
    res.status(500).json({ message: 'Servera kļūda' });
  }
});

module.exports = router;

```

backend/src/routes/news.js

JavaScript

```

const express = require('express');

const router = express.Router();

const News = require('../models/News');

const User = require('../models/User');

const { authenticate, isAdmin } = require('../middleware/auth');

router.get('/', async (req, res) => {

  try {

    const news = await News.findAll({

      include: [{ model: User, as: 'admin', attributes: ['firstName', 'lastName'] }],

      order: [['createdAt', 'DESC']],

    });

    res.json(news);

  } catch (err) {

    res.status(500).json({ message: 'Servera kļūda' });

  }

});

router.get('/:id', async (req, res) => {

  try {

    const item = await News.findByPk(req.params.id, {

      include: [{ model: User, as: 'admin', attributes: ['firstName', 'lastName'] }],

    });

    if (!item) return res.status(404).json({ message: 'Ziņa nav atrasta' });

    res.json(item);

  } catch (err) {

    res.status(500).json({ message: 'Servera kļūda' });

  }

});

```

```
}  
});
```

```
router.post('/', authenticate, isAdmin, async (req, res) => {  
  try {  
    const { title, content, imageUrl, sourceUrl } = req.body;  
    if (!title || !content) return res.status(400).json({ message: 'Virsraksts un saturs ir obligāti' });  
  
    const item = await News.create({ title, content, imageUrl, sourceUrl, adminId: req.user.id });  
    res.status(201).json(item);  
  } catch (err) {  
    res.status(500).json({ message: 'Servera kļūda' });  
  }  
});
```

```
module.exports = router;
```

backend/src/routes/materials.js

JavaScript

```
const express = require('express');  
const router = express.Router();  
const Material = require('../models/Material');  
const { authenticate, isAdmin } = require('../middleware/auth');  
  
router.get('/', async (req, res) => {  
  try {  
    const { category } = req.query;  
  
    const filter = category ? { where: { category } } : {};
```

```

const list = await Material.findAll({ ...filter, order: [['createdAt', 'DESC']] });

res.json(list);

} catch (err) {

  res.status(500).json({ message: 'Servera kļūda' });

}

});

router.get('/:id', async (req, res) => {

  try {

    const material = await Material.findByPk(req.params.id);

    if (!material) return res.status(404).json({ message: 'Materiāls nav atrasts' });

    res.json(material);

  } catch (err) {

    res.status(500).json({ message: 'Servera kļūda' });

  }

});

router.post('/', authenticate, isAdmin, async (req, res) => {

  try {

    const { title, content, category } = req.body;

    if (!title || !content || !category) return res.status(400).json({ message: 'Visi lauki obligāti' });

    const material = await Material.create({ title, content, category, adminId: req.user.id });

    res.status(201).json(material);

  } catch (err) {

    res.status(500).json({ message: 'Servera kļūda' });

  }

}

```

```
});
```

```
module.exports = router;
```

backend/src/routes/apod.js

JavaScript

```
const express = require('express');
```

```
const router = express.Router();
```

```
const axios = require('axios');
```

```
let cache = { data: null, fetchTime: 0 };
```

```
router.get('/', async (req, res) => {
```

```
  try {
```

```
    const now = Date.now();
```

```
    // Cache for 1 hour to prevent flooding limits
```

```
    if (cache.data && now - cache.fetchTime < 60 * 60 * 1000) {
```

```
      return res.json(cache.data);
```

```
    }
```

```
    const apiKey = process.env.NASA_API_KEY || 'DEMO_KEY';
```

```
    const response = await axios.get(`https://api.nasa.gov/planetary/apod?api_key=${apiKey}`);
```

```
    cache.data = response.data;
```

```
    cache.fetchTime = now;
```

```
    res.json(response.data);
```

```
  } catch (err) {
```

```
// Graceful fallback profile on dependency outage

res.json({

  title: 'Mūsu Visums',

  explanation: 'Neizdevās ielādēt datus no NASA APD API. Lūdzu, mēģiniet vēlāk.',

  url: 'https://images.unsplash.com/photo-1506318137071-a8e063b4bec0?q=80&w=1000',

  media_type: 'image'

});

}

});

module.exports = router;
```

frontend/src/main.jsx

JavaScript

```
import React from 'react'

import ReactDOM from 'react-dom/client'

import App from './App.jsx'

import './index.css'

ReactDOM.createRoot(document.getElementById('root')).render(

  <React.StrictMode>

    <App />

  </React.StrictMode>,

)
```

frontend/src/context/AuthContext.jsx

JavaScript

```

import { createContext, useContext, useState, useEffect } from 'react';

import API from '../services/api';

const AuthContext = createContext(null);

export const AuthProvider = ({ children }) => {

  const [user, setUser] = useState(null);

  const [loading, setLoading] = useState(true);

  useEffect(() => {

    const fetchMe = async () => {

      const token = localStorage.getItem('token');

      if (!token) {

        setLoading(false);

        return;

      }

      try {

        const res = await API.get('/users/me');

        setUser(res.data);

      } catch (err) {

        localStorage.removeItem('token');

      } finally {

        setLoading(false);

      }

    };

    fetchMe();

  }, []);

```



```
const loginUser = (token, userData) => {

  localStorage.setItem('token', token);

  setUser(userData);

};
```

```
const logoutUser = () => {

  localStorage.removeItem('token');

  setUser(null);

};
```

```
return (

  <AuthContext.Provider value={{ user, setUser, loginUser, logoutUser, loading }}>

    {children}

  </AuthContext.Provider>

);

};
```

```
export const useAuth = () => useContext(AuthContext);
```

frontend/src/context/LanguageContext.jsx

JavaScript

```
import { createContext, useContext, useState, useEffect } from 'react';
```

```
const LanguageContext = createContext(null);
```

```
const translations = {
```

```
  lv: {
```

```

    home: 'Sākums', news: 'Ziņas', forum: 'Forums', materials: 'Materiāli',
    login: 'Pieteikties', register: 'Reģistrēties', logout: 'Iziet',
    admin: 'Admin', profileTitle: 'Mans profils', searchPlaceholder: 'Meklēt...',
    save: 'Saglabāt', cancel: 'Atcelt', delete: 'Dzēst', loading: 'Ielādē...'
  },
  en: {
    home: 'Home', news: 'News', forum: 'Forum', materials: 'Materials',
    login: 'Log In', register: 'Register', logout: 'Log Out',
    admin: 'Admin', profileTitle: 'My Profile', searchPlaceholder: 'Search...',
    save: 'Save', cancel: 'Cancel', delete: 'Delete', loading: 'Loading...'
  }
};

```

```

export const LanguageProvider = ({ children }) => {
  const [lang, setLang] = useState(localStorage.getItem('lang') || 'lv');

  useEffect(() => {
    localStorage.setItem('lang', lang);
  }, [lang]);

  const t = (key) => translations[lang]?.[key] || key;

  return (
    <LanguageContext.Provider value={{ lang, setLang, t }}>
      {children}
    </LanguageContext.Provider>
  );
}

```

```
};
```

```
export const useLang = () => useContext(LanguageContext);
```

frontend/src/utls/timeAgo.js

JavaScript

```
export const timeAgo = (dateInput, lang = 'lv') => {  
  
  if (!dateInput) return "";  
  
  const date = new Date(dateInput);  
  
  const now = new Date();  
  
  const diffMs = now - date;  
  
  const diffSec = Math.floor(diffMs / 1000);  
  
  const diffMin = Math.floor(diffSec / 60);  
  
  const diffHour = Math.floor(diffMin / 60);  
  
  const diffDay = Math.floor(diffHour / 24);  
  
  
  const lv = {  
  
    justNow: 'Tikko',  
  
    minAgo: (n) => n === 1 ? '1 min. atpakaļ' : `${n} min. atpakaļ`,  
  
    hourAgo: (n) => n === 1 ? '1 st. atpakaļ' : `${n} st. atpakaļ`,  
  
    yesterday: 'Vakar',  
  
    at: 'plkst.',  
  
  };  
  
  
  const en = {  
  
    justNow: 'Just now',  
  
    minAgo: (n) => n === 1 ? '1 min ago' : `${n} min ago`,  
  
    hourAgo: (n) => n === 1 ? '1 hour ago' : `${n} hours ago`,  
  
  };  
  
}
```

```

    yesterday: 'Yesterday',

    at: 'at',

  };

  const t = lang === 'en' ? en : lv;

  const locale = lang === 'en' ? 'en-GB' : 'lv-LV';

  if (diffSec < 60) return t.justNow;

  if (diffMin < 60) return t.minAgo(diffMin);

  if (diffHour < 24) return t.hourAgo(diffHour);

  if (diffDay === 1) {

    const timeStr = date.toLocaleTimeString(locale, { hour: '2-digit', minute: '2-digit' });

    return `${t.yesterday} ${t.at} ${timeStr}`;

  }

  return date.toLocaleDateString(locale, { year: 'numeric', month: 'short', day: 'numeric' });

};

```

frontend/src/utls/stripMarkdown.js

JavaScript

```

export function stripMarkdown(text) {

  if (!text) return "";

  return text

    .replace(/#+\s+/g, "") // Remove Headers

    .replace(/\[([^\]]+)\]\([^\)]+\)/g, '$1') // Remove links links matching standard markdown format

    .replace(/\[* _ ~\]/g, "") // Remove simple tags

    .replace(/\n+/g, ' '); // Linebreaks normalization

```

```
}
```

frontend/src/Utils/categoryUtils.js

JavaScript

```
export const CATEGORY_TRANSLATIONS = {  
  
  'Planētas': 'Planets',  
  
  'Galaktikas': 'Galaxies',  
  
  'Kosmoss': 'Deep Space',  
  
  'Astrofizika': 'Astrophysics',  
  
  'Novērojumi': 'Observations'  
  
};
```

frontend/src/App.jsx

JavaScript

```
import { BrowserRouter as Router, Routes, Route } from 'react-router-dom';  
  
import { AuthProvider } from './context/AuthContext';  
  
import { LanguageProvider } from './context/LanguageContext';  
  
  
import Navbar from './components/Navbar';  
  
import Footer from './components/Footer';  
  
import StarsBackground from './components/StarsBackground';  
  
  
import Home from './pages/Home';  
  
import News from './pages/News';  
  
import NewsDetail from './pages/NewsDetail';  
  
import Forum from './pages/Forum';  
  
import PostDetail from './pages/PostDetail';  
  
import CreatePost from './pages/CreatePost';
```

```

import Materials from './pages/Materials';

import MaterialDetail from './pages/MaterialDetail';

import SolarSystem from './pages/SolarSystem';

import Profile from './pages/Profile';

import PublicProfile from './pages/PublicProfile';

import Admin from './pages/Admin';

import Login from './pages/Login';

import Register from './pages/Register';

import ForgotPassword from './pages/ForgotPassword';

import ResetPassword from './pages/ResetPassword';

import NotFound from './pages/NotFound';


import './App.css';


function App() {

  return (

    <LanguageProvider>

      <AuthProvider>

        <Router>

          <div className="app-container">

            <StarsBackground />

            <Navbar />

            <main className="main-content">

              <Routes>

                <Route path="/" element={<Home />} />

                <Route path="/zinas" element={<News />} />

                <Route path="/zinas/:id" element={<NewsDetail />} />

```

```

    <Route path="/forums" element={<Forum />} />

    <Route path="/forums/raksts/:id" element={<PostDetail />} />

    <Route path="/forums/izveidot" element={<CreatePost />} />

    <Route path="/materiali" element={<Materials />} />

    <Route path="/materiali/:id" element={<MaterialDetail />} />

    <Route path="/saules-sistema" element={<SolarSystem />} />

    <Route path="/profilis" element={<Profile />} />

    <Route path="/lietotajs/:id" element={<PublicProfile />} />

    <Route path="/admin" element={<Admin />} />

    <Route path="/pieteikties" element={<Login />} />

    <Route path="/registreties" element={<Register />} />

    <Route path="/aizmirsu-paroli" element={<ForgotPassword />} />

    <Route path="/atjaunot-paroli" element={<ResetPassword />} />

    <Route path="*" element={<NotFound />} />

  </Routes>

</main>

<Footer />

</div>

</Router>

</AuthProvider>

</LanguageProvider>

);

}

```

```
export default App;
```

frontend/src/components/Navbar.jsx

JavaScript

```

import { useState } from 'react';

import { Link, useNavigate } from 'react-router-dom';

import { useAuth } from '../context/AuthContext';

import { useLang } from '../context/LanguageContext';

import BlackHoleLogo from './BlackHoleLogo';

import Avatar from './Avatar';

import './Navbar.css';

const Navbar = () => {

  const { user, logoutUser } = useAuth();

  const { lang, setLang, t } = useLang();

  const [menuOpen, setMenuOpen] = useState(false);

  const navigate = useNavigate();

  const handleLogout = () => {

    logoutUser();

    navigate('/');

  };

  return (

    <nav className="navbar">

      <div className="navbar-container">

        <Link to="/" className="navbar-brand">

          <BlackHoleLogo size={28} />

          <span className="navbar-title">AstroLV</span>

        </Link>

```



```

<div className="navbar-links">

  <Link to="/" className="nav-link">{t('home')}</Link>

  <Link to="/zinas" className="nav-link">{t('news')}</Link>

  <Link to="/forums" className="nav-link">{t('forum')}</Link>

  <Link to="/materiali" className="nav-link">{t('materials')}</Link>

  <Link to="/saules-sistema" className="nav-link">Saules Sistēma</Link>

  {user?.role === 'admin' && <Link to="/admin" className="nav-link admin-
link">{t('admin')}</Link>}

</div>


<div className="navbar-auth">

  <button className="lang-toggle" onClick={() => setLang(lang === 'lv' ? 'en' : 'lv')}>

    {lang.toUpperCase()}

  </button>

  {user ? (

    <div className="navbar-user">

      <Link to="/profilis" className="navbar-user-link">

        <Avatar user={user} size={32} />

        <span className="navbar-username">{user.firstName}</span>

      </Link>

      <button onClick={handleLogout} className="btn btn-secondary btn-
sm">{t('logout')}</button>

    </div>

  ) : (

    <div className="navbar-auth-buttons">

      <Link to="/pieteikties" className="btn btn-secondary btn-sm">{t('login')}</Link>

      <Link to="/registreties" className="btn btn-primary btn-sm">{t('register')}</Link>

    </div>

```

```

    );

    </div>

  </div>

</nav>

);

};

```

```
export default Navbar;
```

frontend/src/components/StarsBackground.jsx

JavaScript

```

import { useEffect, useRef } from 'react';

const StarsBackground = () => {

  const canvasRef = useRef(null);

  useEffect(() => {

    const canvas = canvasRef.current;

    const ctx = canvas.getContext('2d');

    let animationFrameId;

    const resizeCanvas = () => {

      canvas.width = window.innerWidth;

      canvas.height = window.innerHeight;

    };

    window.addEventListener('resize', resizeCanvas);

    resizeCanvas();

```

```

const stars = Array.from({ length: 150 }, () => ({
  x: Math.random() * canvas.width,
  y: Math.random() * canvas.height,
  size: Math.random() * 1.5,
  alpha: Math.random(),
  speed: 0.005 + Math.random() * 0.01
}));

const draw = () => {
  ctx.clearRect(0, 0, canvas.width, canvas.height);
  ctx.fillStyle = '#030712';
  ctx.fillRect(0, 0, canvas.width, canvas.height);

  stars.forEach(star => {
    star.alpha += star.speed;
    if (star.alpha > 1 || star.alpha < 0) star.speed = -star.speed;

    ctx.fillStyle = `rgba(255, 255, 255, ${Math.abs(star.alpha)})`;
    ctx.beginPath();
    ctx.arc(star.x, star.y, star.size, 0, Math.PI * 2);
    ctx.fill();
  });

  animationFrameId = requestAnimationFrame(draw);
};

draw();

```

```

return () => {

  window.removeEventListener('resize', resizeCanvas);

  cancelAnimationFrame(animationFrameId);

};

}, []);

return <canvas ref={canvasRef} style={{ position: 'fixed', top: 0, left: 0, zIndex: -1, pointerEvents:
'none' }} />;

};

export default StarsBackground;

```

frontend/src/pages/Home.jsx

JavaScript

```

import { useEffect, useState } from 'react';

import { getApod } from '../services/api';

import EarthPlanet from '../components/EarthPlanet';

import './Home.css';

const Home = () => {

  const [apod, setApod] = useState(null);

  useEffect(() => {

    getApod().then(res => setApod(res.data)).catch(err => console.error(err));

  }, []);

  return (

```

```

<div className="home-container">

  <header className="hero-section">

    <h1>Atklāj Visuma Noslēpumus</h1>

    <p>Latvijas astronomijas entuziastu kopiena un mācību resursi.</p>

  </header>


  <div className="home-grid">

    <div className="grid-card canvas-card">

      <h3>Interaktīvais Visums</h3>

      <EarthPlanet />

    </div>


    {apod && (

      <div className="grid-card apod-card">

        <h3>NASA Dienas Attēls: {apod.title}</h3>

        {apod.media_type === 'image' ? (

          <img src={apod.url} alt={apod.title} className="apod-img" />

        ) : (

          <iframe src={apod.url} title={apod.title} className="apod-video" />

        )}

        <p className="apod-explanation">{apod.explanation}</p>

      </div>

    )}

  </div>

</div>

);

};

```

```
export default Home;
```

frontend/src/pages/News.jsx

JavaScript

```
import { useEffect, useState } from 'react';
```

```
import { Link } from 'react-router-dom';
```

```
import { getNews } from '../services/api';
```

```
import './News.css';
```

```
const News = () => {
```

```
  const [newsList, setNewsList] = useState([]);
```

```
  useEffect(() => {
```

```
    getNews().then(res => setNewsList(res.data)).catch(err => console.error(err));
```

```
  }, []);
```

```
  return (
```

```
    <div className="news-page">
```

```
      <h2>Zinātnes un Kosmosa Ziņas</h2>
```

```
      <div className="news-grid">
```

```
        {newsList.map(item => (
```

```
          <div key={item.id} className="news-card">
```

```
            {item.imageUrl && <img src={item.imageUrl} alt={item.title} />}
```

```
            <div className="news-card-content">
```

```
              <h3>{item.title}</h3>
```

```
              <p>{item.content.substring(0, 150)}...</p>
```

```
              <Link to={` /zinatnes/${item.id}`} className="btn btn-primary btn-sm">Lasīt vairāk</Link>
```

```

        </div>

    </div>

    )}

</div>

</div>

);

};

export default News;

frontend/src/pages/NewsDetail.jsx

JavaScript

import { useEffect, useState } from 'react';

import { useParams, Link } from 'react-router-dom';

import { getNewsItem } from '../services/api';

import './NewsDetail.css';

const NewsDetail = () => {

    const { id } = useParams();

    const [item, setItem] = useState(null);

    useEffect(() => {

        getNewsItem(id).then(res => setItem(res.data)).catch(err => console.error(err));

    }, [id]);

    if (!item) return <div className="loading">Ielādē...</div>;

    return (

```

```

<div className="news-detail-container">

  <Link to="/zinas" className="back-link">&larr; Atpakaļ uz ziņām</Link>

  <h1>{item.title}</h1>

  <div className="news-meta">Pievienots: {new Date(item.createdAt).toLocaleDateString('lv-LV')}</div>

  {item.imageUrl && <img src={item.imageUrl} alt={item.title} className="news-main-img" />}

  <div className="news-content">{item.content}</div>

  {item.sourceUrl && <a href={item.sourceUrl} target="_blank" rel="noreferrer"
className="source-link">Oriģinālais avots</a>}

</div>

);

};

```

```
export default NewsDetail;
```

frontend/src/pages/Forum.jsx

JavaScript

```

import { useEffect, useState } from 'react';

import { Link } from 'react-router-dom';

import { getPosts } from '../services/api';

import { useAuth } from '../context/AuthContext';

import Avatar from '../components/Avatar';

import { timeAgo } from '../utils/timeAgo';

import './Forum.css';

const Forum = () => {

  const [posts, setPosts] = useState([]);

  const { user } = useAuth();

```



```

useEffect(() => {

  getPosts().then(res => setPosts(res.data)).catch(err => console.error(err));

}, []);

return (

  <div className="forum-container">

    <div className="forum-header">

      <h2>Diskusiju Forums</h2>

      {user && <Link to="/forums/izveidot" className="btn btn-primary">Jauna Tēma</Link>}

    </div>

    <div className="post-list">

      {posts.map(post => (

        <div key={post.id} className="post-row">

          <Avatar user={post.author} size={40} />

          <div className="post-info">

            <Link to={` /forums/raksts/${post.id}` } className="post-title">{post.title}</Link>

            <div className="post-meta">

              Kategorija: <span className="cat-tag">{post.category}</span> &bull;

              Autors: {post.author?.firstName} &bull; {timeAgo(post.createdAt)}

            </div>

          </div>

        </div>

      ))}

    </div>

  </div>

);

};

```

```
export default Forum;
```

frontend/src/pages/Materials.jsx

JavaScript

```
import { useEffect, useState } from 'react';
```

```
import { Link } from 'react-router-dom';
```

```
import { getMaterials } from '../services/api';
```

```
import './Materials.css';
```

```
const Materials = () => {
```

```
  const [materials, setMaterials] = useState([]);
```

```
  const [cat, setCat] = useState("");
```

```
  useEffect(() => {
```

```
    getMaterials(cat).then(res => setMaterials(res.data)).catch(err => console.error(err));
```

```
  }, [cat]);
```

```
  return (
```

```
    <div className="materials-container">
```

```
      <h2>Mācību Materiāli</h2>
```

```
      <div className="filter-bar">
```

```
        <button className={`filter-btn ${cat === "" ? 'active' : ""}`} onClick={() =>
setCat("")}>Visi</button>
```

```
        <button className={`filter-btn ${cat === 'Pamati' ? 'active' : ""}`} onClick={() =>
setCat('Pamati')}>Pamati</button>
```

```
        <button className={`filter-btn ${cat === 'Sistēmas' ? 'active' : ""}`} onClick={() =>
setCat('Sistēmas')}>Sistēmas</button>
```

```

    <button className={`filter-btn ${cat === 'Tehnika' ? 'active' : ''}`} onClick={() =>
setCat('Tehnika')}>Tehnika</button>

    <button className={`filter-btn ${cat === 'Astrofotogrāfija' ? 'active' : ''}`} onClick={() =>
setCat('Astrofotogrāfija')}>Astrofotogrāfija</button>

  </div>

  <div className="materials-list">

    {materials.map(m => (

      <div key={m.id} className="material-card">

        <h3>{m.title}</h3>

        <span className="cat-badge">{m.category}</span>

        <p>{m.content.substring(0, 120)}...</p>

        <Link to={`/${material}/${m.id}`} className="btn btn-secondary btn-sm">Atvērt
materiālu</Link>

      </div>

    ))}

  </div>

</div>

);

};

```

```
export default Materials;
```

frontend/src/pages/SolarSystem.jsx

JavaScript

```

import { Canvas } from '@react-three/fiber';

import { OrbitControls, Stars } from '@react-three/drei';

import './SolarSystem.css';

const PlanetVisual = ({ color, size, position, name }) => {

```

```

return (

  <mesh position={position}>

    <sphereGeometry args={[size, 32, 32]} />

    <meshStandardMaterial color={color} roughness={0.8} />

  </mesh>

);

};

const SolarSystem = () => {

  return (

    <div className="solar-system-page">

      <h2>Saules Sistēmas 3D Modelis (Shematisks)</h2>

      <div className="canvas-container" style={{ width: '100%', height: '500px', background:
'#020408', borderRadius: '12px' }}>

        <Canvas camera={{ position: [0, 10, 15], fov: 60 }}>

          <ambientLight intensity={0.6} />

          <pointLight position={[0, 0, 0]} intensity={2} color="#f59e0b" />

          { /* Saule */ }

          <mesh position={[0, 0, 0]}>

            <sphereGeometry args={[1.5, 32, 32]} />

            <meshBasicMaterial color="#f59e0b" />

          </mesh>

          { /* Planētas */ }

          <PlanetVisual color="#9ca3af" size={0.3} position={[3, 0, 0]} name="Merkurs" />

          <PlanetVisual color="#f59e0b" size={0.5} position={[5, 0, 0]} name="Venēra" />

        </div>

      </div>

    )

  )
}

```

```

    <PlanetVisual color="#3b82f6" size={0.6} position={[7, 0, 0]} name="Zeme" />

    <PlanetVisual color="#ef4444" size={0.4} position={[9, 0, 0]} name="Marss" />


    <Stars count={1000} factor={4} speed={1} />

    <OrbitControls />

  </Canvas>

</div>

</div>

);

};

export default SolarSystem;

frontend/src/pages/Profile.jsx

JavaScript

import { useState, useEffect } from 'react';

import { useAuth } from '../context/AuthContext';

import { updateMe, uploadAvatar } from '../services/api';

import Avatar from '../components/Avatar';

import './Profile.css';

const Profile = () => {

  const { user, setUser } = useAuth();

  const [form, setForm] = useState({ firstName: "", lastName: "", bio: "", location: "", website: "", interests: ""
});

  const [file, setFile] = useState(null);

  useEffect(() => {

```

```

if (user) {

  setForm({

    firstName: user.firstName || "",

    lastName: user.lastName || "",

    bio: user.bio || "",

    location: user.location || "",

    website: user.website || "",

    interests: user.interests || ""

  });

}

}, [user]);

```

```

const handleSave = async (e) => {

  e.preventDefault();

  try {

    const res = await updateMe(form);

    setUser(res.data);

    alert('Profils atjaunots!');

  } catch (err) {

    console.error(err);

  }

};

```

```

const handleAvatarUpload = async (e) => {

  const selectedFile = e.target.files[0];

  if (!selectedFile) return;

```

```

const formData = new FormData();

formData.append('avatar', selectedFile);

try {

  const res = await uploadAvatar(formData);

  setUser({ ...user, avatar: res.data.avatar });

} catch (err) {

  alert('Kļūda augšupielādējot avатарu');

}

};

if (!user) return <div className="loading">Lūdzu, autorizējieties...</div>;

return (

  <div className="profile-container">

    <h2>Mans Profils</h2>

    <div className="profile-layout">

      <div className="avatar-section">

        <Avatar user={user} size={120} />

        <input type="file" onChange={handleAvatarUpload} style={{ marginTop: '1rem' }} />

      </div>

      <form onSubmit={handleSave} className="auth-form profile-form">

        <div className="form-group">

          <label>Vārds</label>

          <input type="text" value={form.firstName} onChange={e => setForm({ ...form, firstName:
e.target.value })} />

        </div>

```

```

    <div className="form-group">

      <label>Uzvārds</label>

      <input type="text" value={form.lastName} onChange={e => setForm({ ...form, lastName:
e.target.value })} />

    </div>

    <div className="form-group">

      <label>Bio</label>

      <textarea value={form.bio} onChange={e => setForm({ ...form, bio: e.target.value })}
rows="3" />

    </div>

    <button type="submit" className="btn btn-primary">Saglabāt izmaiņas</button>

  </form>

</div>

</div>

);

};

```

```
export default Profile;
```

frontend/src/pages/PublicProfile.jsx

JavaScript

```

import { useEffect, useState } from 'react';

import { useParams } from 'react-router-dom';

import { getPublicProfile } from '../services/api';

import Avatar from '../components/Avatar';

import './PublicProfile.css';

const PublicProfile = () => {

  const { id } = useParams();

```



```

const [profile, setProfile] = useState(null);

useEffect(() => {

  getPublicProfile(id).then(res => setProfile(res.data)).catch(err => console.error(err));

}, [id]);

if (!profile) return <div className="loading">Ielādē profilu...</div>;

return (

  <div className="public-profile-container">

    <div className="profile-card-header">

      <Avatar user={profile.user} size={100} />

      <h2>{profile.user.firstName} {profile.user.lastName}</h2>

      <span className="role-tag">{profile.user.role}</span>

    </div>

    <div className="profile-body-details">

      {profile.user.bio} && <p className="bio-text"><strong>Par mani:</strong>
      {profile.user.bio}</p>

      <p><strong>Rakstu skaits:</strong> {profile.postCount}</p>

      {profile.user.location} && <p><strong>Atrašanās vieta:</strong> {profile.user.location}</p>

    </div>

  </div>

);

};

export default PublicProfile;

frontend/src/pages/Admin.jsx

```

JavaScript

```
import { useEffect, useState } from 'react';

import { getUsers, updateUserRole, updateUserStatus } from '../services/api';

import './Admin.css';

const Admin = () => {

  const [users, setUsers] = useState([]);

  useEffect(() => {

    getUsers().then(res => setUsers(res.data)).catch(err => console.error(err));

  }, []);

  const toggleRole = async (id, currentRole) => {

    const nextRole = currentRole === 'admin' ? 'user' : 'admin';

    try {

      await updateUserRole(id, nextRole);

      setUsers(users.map(u => u.id === id ? { ...u, role: nextRole } : u));

    } catch (err) {

      console.error(err);

    }

  };

  const toggleStatus = async (id, currentStatus) => {

    const nextStatus = currentStatus === 'blocked' ? 'active' : 'blocked';

    try {

      await updateUserStatus(id, nextStatus);

      setUsers(users.map(u => u.id === id ? { ...u, status: nextStatus } : u));

    }

  };

}
```

```

    } catch (err) {

      console.error(err);

    }

  };

  return (

    <div className="admin-container">

      <h2>Administrators Panel</h2>

      <table className="admin-table">

        <thead>

          <tr>

            <th>Lietotājs</th>

            <th>E-pasts</th>

            <th>Loma</th>

            <th>Statuss</th>

            <th>Darbības</th>

          </tr>

        </thead>

        <tbody>

          {users.map(u => (

            <tr key={u.id}>

              <td>{u.firstName} {u.lastName}</td>

              <td>{u.email}</td>

              <td>{u.role}</td>

              <td>{u.status}</td>

              <td>

```

```

        <button onClick={() => toggleRole(u.id, u.role)} className="btn btn-secondary btn-sm">Mainit lomu</button>

        <button onClick={() => toggleStatus(u.id, u.status)} className={`btn btn-sm ${u.status
        === 'blocked' ? 'btn-success' : 'btn-danger'}`} `>

            {u.status === 'blocked' ? 'Atbloķēt' : 'Bloķēt'}

        </button>

    </td>

</tr>

)}}

</tbody>

</table>

</div>

);

};

```

```
export default Admin;
```

frontend/src/pages/Login.jsx

JavaScript

```

import { useState } from 'react';

import { useNavigate, Link } from 'react-router-dom';

import { login } from '../services/api';

import { useAuth } from '../context/AuthContext';

import './Auth.css';

const Login = () => {

    const [email, setEmail] = useState("");

    const [password, setPassword] = useState("");

    const { loginUser } = useAuth();

```

```

const navigate = useNavigate();

const handleLoginSubmit = async (e) => {
  e.preventDefault();

  try {
    const res = await login({ email, password });
    loginUser(res.data.token, res.data.user);
    navigate('/');
  } catch (err) {
    alert(err.response?.data?.message || 'Pieteikšanās neizdevās');
  }
};

return (
  <div className="auth-container">
    <form onSubmit={handleLoginSubmit} className="auth-form">
      <h2>Pieteikties sistēmā</h2>
      <div className="form-group">
        <label>E-pasts</label>
        <input type="email" value={email} onChange={e => setEmail(e.target.value)} required />
      </div>
      <div className="form-group">
        <label>Parole</label>
        <input type="password" value={password} onChange={e => setPassword(e.target.value)}
required />
      </div>
      <button type="submit" className="btn btn-primary btn-block">Ieiet</button>
    </form>
  </div>
);

```

```

    <p className="auth-redirect">Nav konta? <Link to="/registreties">Reģistrēties šeit</Link></p>

  </form>

</div>

);

};

```

```
export default Login;
```

frontend/src/pages/Register.jsx

JavaScript

```

import { useState } from 'react';

import { useNavigate, Link } from 'react-router-dom';

import { register, verifyRegistration } from '../services/api';

import { useAuth } from '../context/AuthContext';

import './Auth.css';

const Register = () => {

  const [step, setStep] = useState(1);

  const [form, setForm] = useState({ firstName: "", lastName: "", email: "", password: "" });

  const [code, setCode] = useState("");

  const { loginUser } = useAuth();

  const navigate = useNavigate();

  const handleRegisterInit = async (e) => {

    e.preventDefault();

    try {

      await register(form);

      setStep(2);

```

```

    } catch (err) {

      alert(err.response?.data?.message || 'Reģistrācijas kļūda');

    }

  };

```

```

const handleVerifySubmit = async (e) => {

  e.preventDefault();

  try {

    const res = await verifyRegistration({ email: form.email, code });

    loginUser(res.data.token, res.data.user);

    navigate('/');

  } catch (err) {

    alert(err.response?.data?.message || 'Nepareizs kods');

  }

};

```

```

return (

  <div className="auth-container">

    {step === 1 ? (

      <form onSubmit={handleRegisterInit} className="auth-form">

        <h2>Reģistrācija</h2>

        <div className="form-group">

          <label>Vārds</label>

          <input type="text" value={form.firstName} onChange={e => setForm({ ...form, firstName:
e.target.value })} required />

        </div>

        <div className="form-group">

```

```

    <label>Uzvārds</label>

    <input type="text" value={form.lastName} onChange={e => setForm({ ...form, lastName:
e.target.value })} required />

  </div>

  <div className="form-group">

    <label>E-pasts</label>

    <input type="email" value={form.email} onChange={e => setForm({ ...form, email:
e.target.value })} required />

  </div>

  <div className="form-group">

    <label>Parole (min. 8 simboli)</label>

    <input type="password" value={form.password} onChange={e => setForm({ ...form,
password: e.target.value })} required />

  </div>

  <button type="submit" className="btn btn-primary btn-block">Turpināt</button>

  <p className="auth-redirect">Jau ir kots? <Link to="/pieteikties">Pieteikties</Link></p>

</form>

): (

<form onSubmit={handleVerifySubmit} className="auth-form">

  <h2>Apstiprināt e-pastu</h2>

  <p style={{ fontSize: '0.9rem', color: 'var(--text-secondary)', marginBottom: '1rem' }}>Uz
{form.email} tika nosūtīts 6 zīmju kods.</p>

  <div className="form-group">

    <label>Ievadiet saņemto kodu</label>

    <input type="text" value={code} onChange={e => setCode(e.target.value)} required
maxLength="6" style={{ textAlign: 'center', fontSize: '1.2rem', letterSpacing: '4px' }} />

  </div>

  <button type="submit" className="btn btn-success btn-block">Apstiprināt un
pabeigt</button>

```



```

        </form>

    })

</div>

);

};

export default Register;

frontend/src/pages/ForgotPassword.jsx

JavaScript

import { useState } from 'react';

import { forgotPassword } from '../services/api';

import './Auth.css';

const ForgotPassword = () => {

    const [email, setEmail] = useState("");

    const [sent, setSent] = useState(false);

    const handleSubmit = async (e) => {

        e.preventDefault();

        try {

            await forgotPassword({ email });

            setSent(true);

        } catch (err) {

            console.error(err);

        }

    };

```

```

return (

  <div className="auth-container">

    <div className="auth-form">

      <h2>Aizmirsu paroli</h2>

      {!sent ? (

        <form onSubmit={handleSubmit}>

          <div className="form-group">

            <label>Ievadiet sava profila e-pastu</label>

            <input type="email" value={email} onChange={e => setEmail(e.target.value)} required />

          </div>

          <button type="submit" className="btn btn-primary btn-block">Sūtīt instrukcijas</button>

        </form>

      ) : (

        <p style={{ color: 'var(--success)', fontSize: '0.9rem' }}>Ja e-pasts eksistē, atjaunošanas saite
        tika nosūtīta uz jūsu e-pastu.</p>

      )}

    </div>

  </div>

);

};

export default ForgotPassword;

```